OmniLaTeX

The OmniLaTeX Manual

Firmware Version 1.0.0

Release Date: May 9, 2026
Support Contact: https://github.com/WyattAu/OmniLaTeX-template

---

A comprehensive reference manual for the OmniLaTeX document class. Covers every functionality of modern LaTeX through 58 chapters: installation, 26 document types, typography, mathematics, internationalization (18 languages + CJK/RTL), figures, tables, TikZ diagrams, bibliographies, glossaries, code listings, presentations, posters, accessibility, and formal verification via Lean 4.

OmniLaTeX

The OmniLaTeX Manual

Firmware Version 1.0.0

Release Date: May 9, 2026
Support Contact: https://github.com/WyattAu/OmniLaTeX-template

---

A comprehensive reference manual for the OmniLaTeX document class. Covers every functionality of modern LaTeX through 58 chapters: installation, 26 document types, typography, mathematics, internationalization (18 languages + CJK/RTL), figures, tables, TikZ diagrams, bibliographies, glossaries, code listings, presentations, posters, accessibility, and formal verification via Lean 4.

Preface

0.1 Purpose of This Manual

This manual serves as the comprehensive reference for the OmniLaTeX document class–a modular, extensible LaTeX template designed to unify the production of academic and technical documents under a single framework. Rather than maintaining separate templates for theses, articles, reports, presentations, and posters, OmniLaTeX provides a single class with a rich doctype system that adapts its behaviour to the task at hand.
The manual documents every public interface offered by the class: class options, document-type profiles, custom commands, supported packages, and recommended workflows. It is intended to be both a tutorial for newcomers and a definitive reference for experienced users.

0.2 Who OmniLaTeX Is For

OmniLaTeX is designed for anyone who produces technical or scientific documents with LaTeX. The class addresses a common pain point: maintaining separate templates for every document type leads to duplicated effort, inconsistent formatting, and divergent maintenance paths. OmniLaTeX solves this by providing a single, unified framework.

- **Researchers and academics** preparing journal articles, conference papers, and technical reports. OmniLaTeX supports dedicated profiles for articles, inline papers, journals, research proposals, and technical reports–each with appropriate metadata fields, layout defaults, and title page designs.
- **Graduate students** writing theses or dissertations that must conform to institutional formatting requirements. The `doctype=thesis` and `doctype=dissertation` profiles provide structured title pages with advisor, committee, and degree fields. Institutional profiles (e.g. `institution=tuhh`) layer on branding and compliance requirements.
- **Engineers** generating design documents, specifications, manuals, standards, and patents. OmniLaTeX offers dedicated profiles for each, including `doctype=manual`, `doctype=standard`, and `doctype=patent`.
- **Educators** producing homework assignments, examinations, lecture notes, syllabi, and handouts. Each education doctype provides domain-specific environments such as `exercise`, `solution`, `question`, and `answer`.
- **Business professionals** creating CVs, cover letters, memos, invoices, and white papers. OmniLaTeX profiles for business doctypes include appropriate metadata commands and layout defaults.
- **Technical writers** who need a consistent, maintainable template for multi-chapter publications.

Readers are assumed to have basic familiarity with LaTeX (e.g. they know what a preamble is and how to compile a document). No prior experience with KOMA-Script or OmniLaTeX is required.

0.3 What Makes OmniLaTeX Different

OmniLaTeX distinguishes itself from other LaTeX templates through several key design decisions:
26 doctypes, 3 KOMA base classes. Every document type–from a one-page memo to a multi-volume dictionary–maps to one of 26 canonical doctypes. These doctypes are backed by only three KOMA-Script base classes (`scrbook`, `scrreprt`, and `scrartcl`), keeping the underlying complexity minimal while presenting a rich, domain-specific surface to the user. Over 80 user-friendly aliases ensure that common names (e.g. "paper", "resume", "talk") resolve correctly.
16 institutional profiles. Institution-specific branding, title pages, and compliance requirements are handled by pluggable institution modules. The class loads institutional configuration from `config/institutions/` based on the institution class option, without modifying any core files.
Modular architecture. Functionality is organised into independent modules under `lib/`: typography, graphics, layout, code, tables, references, and language. Each module can be enabled or disabled via class options (enablemath, enabletikz, etc.), reducing load time for documents that do not need every feature.
LuaLaTeX-native. OmniLaTeX requires LuaLaTeX and targets TeX Live 2025+. This enables `unicode-math` for modern font handling, `polyglossia` for multilingual support, and Lua-based workarounds for known TeX Live incompatibilities.
Formal verification via Lean 4. OmniLaTeX supports `lean4` code listings and proof rendering, allowing formal verification artifacts to be embedded directly in technical documents alongside LaTeX proofs.

0.4 How to Use This Book

The manual is divided into eleven parts, each covering a major topic area. Chapters are designed to be as self-contained as possible so that readers can jump directly to the section relevant to their current task. Where chapters depend on material presented elsewhere, explicit cross-references are provided.
A recommended reading order for first-time users:

1. Part I (*Foundations*) – installation, quick start, build system, and class options.
2. The chapter corresponding to your document type in Part II (*Document Types*).
3. Remaining parts as needed for typography, layout, figures, references, and advanced features.

The chapter numbering follows a two-digit prefix scheme: `01_…` for front matter, `10_…–19_…` for foundations, `20_…–29_…` for document types, `30_…–39_…` for typography, and so on. Appendix chapters use the prefix `B0_…–B4_….`

0.5 Prerequisites

The following LaTeX packages are loaded automatically by OmniLaTeX core modules and do not require manual installation:

```
% Core (always loaded)
mathtools, unicode-math, polyglossia, tracklang,
booktabs, hyperref, cleveref, tcolorbox, minted,
siunitx, chemmacros, glossaries-extra, biblatex

% Conditional (enabled by class options)
tikz, pgfplots, fontawesome5, setspaceenhanced
```

Table 1: System requirements for OmniLaTeX.

Component	Minimum Version	Notes
LuaLaTeX engine	TeX Live 2025	Required; pdfLaTeX and XeLaTeX are not supported
TeX distribution	TeX Live 2025+	Full installation recommended; `scheme-full` or at minimum `scheme-medium`
Python	3.8+	Required by **minted** for syntax highlighting
Pygments	latest	Python package; install via `pip install Pygments`
Git	any	Required for the Nix-based build system
Nix	2.18+ (optional)	Required for reproducible builds; see Chapter ??

0.6 Scope and Relation to the TUHH Thesis Template

OmniLaTeX originated from the Hamburg University of Technology (TUHH) thesis template but has since evolved into a general-purpose framework. This manual covers the full OmniLaTeX class in its current form. Features that are specific to the TUHH institutional profile are noted where relevant but are not the primary focus.
If you are writing a TUHH thesis, consult the `doctype=thesis` profile documentation in Chapter ?? for institution-specific configuration options.

0.7 Acknowledgments

OmniLaTeX builds upon the work of countless contributors to the TeX and LaTeX ecosystems. Special thanks go to the KOMA-Script team for the `scrbook`, `scrreprt`, and `scrartcl` base classes that form the foundation of every OmniLaTeX document. The `minted`, `biblatex`, `glossaries-extra`, and `tikz` communities have also been instrumental in making this template possible.

0.8 Conventions Used in This Manual

The following typographic conventions are used throughout:

- `Monospace text` denotes LaTeX commands, environments, file names, and code snippets.
- Sans-serif text denotes option names and key-value pairs.
- **Bold text** highlights key terms on their first appearance.
- UI paths indicates clickable menu or interface elements.

All code examples are shown in monospaced listings and are designed to be directly compilable. Where an example is abbreviated for clarity, the omissions are explicitly noted.
Full details on these conventions are given in the next chapter, *Notation & Conventions*.

0.9 Feedback and Contributions

OmniLaTeX is an open-source project. Bug reports, feature requests, and contributions are welcome:

Repository `https://github.com/WyattAu/OmniLaTeX-template`

Issues File bugs and feature requests via GitHub Issues

Pull Requests Contributions follow the standard fork–branch–PR workflow

When reporting issues, please include the OmniLaTeX version (printed in the LaTeX log as `OmniLaTeX v1.x.x`), your TeX Live version, and a minimal working example that reproduces the problem.

Notation & Conventions

0.10 Typographic Conventions

This manual uses consistent typographic formatting to distinguish between different kinds of information. Table ?? provides a summary.

Table 2: Typographic conventions used in this manual.

Appearance	Meaning	Example
`Monospace`	LaTeX commands, code, filenames	`\documentclass`
Sans-serif	Key-value options, package options	language=english
Bold	Key terms (first occurrence)	**doctype profile**
UI element	Menu paths, interface elements	File > Compile
Italics	Emphasis, introduced terminology	*preamble*
Small Caps	Proper names of standards	Unicode

When a command is shown with its mandatory argument, curly braces are included (e.g. `\chapter{Title}`). Optional arguments appear in square brackets (e.g. `\section[short]{Long Title}`). A trailing `*` denotes the starred (unnumbered) variant: `\chapter*{Title}`.

0.11 Package References

Throughout this manual, LaTeX packages are referenced using the `\ctanpackage{}` command, which produces a hyperlink to the package's page on the Comprehensive TeX Archive Network (CTAN). For example, the **minted** package is referenced as shown. Hovering over or clicking the link navigates to `https://ctan.org/pkg/minted`.
This convention allows readers to quickly locate the authoritative documentation for any package mentioned in the text.

0.12 Cross-References

The manual uses the standard LaTeX cross-referencing system. Chapters are numbered consecutively within each part. Equations, figures, tables, and listings are numbered per chapter using the scheme `<chapter>.<counter>`. For example, Equation (3.7) refers to the seventh equation in Chapter 3. Cross-references within the manual use the commands `\ref{}`, `\eqref{}`, `\cref{}`, and `\Cref{}` from the **cleveref** package. Readers should note that `\cref` and `\Cref` automatically insert the appropriate label prefix ("Figure", "Table", "Chapter", etc.).

0.13 Code Examples

All code examples presented in this manual are intended to be directly compilable when placed in a suitable document preamble. Code listings are rendered using the **minted** package, which provides syntax highlighting for a wide range of programming and markup languages.
Where an example is shortened for brevity, the ellipsis marker … is used within the listing and the omission is explicitly described in the surrounding text. Complete source files for the examples are available in the `examples/` directory of the OmniLaTeX repository.
The `minted` package requires Python and the Pygments library to be installed. Ensure that `pip install Pygments` has been run before compiling documents that contain code listings.

0.14 File Path Notation

File paths in this manual use forward slashes regardless of operating system:

Table 3: File path conventions.

Notation	Meaning
`config/fonts.sty`	Relative to repository root
`~/.texlive`	User home directory
`examples/manual/content/`	Directory path (trailing slash)
`main.tex`	File in current directory

Command-line examples use `$` for shell prompts and assume a Bash-compatible shell unless otherwise stated.

0.15 Math Notation

Mathematical content in this manual follows the conventions of the **amsmath** and **unicode-math** packages. Inline mathematics uses `$...$` delimiters; display mathematics uses `equation`, `align`, or `gather` environments.
Variables are set in italic (x, n), vectors in bold italic ($\mathbf{v}$), and operators in upright type ($\sin$, $\log$). Physical constants and named quantities use small capitals where appropriate.

0.16 Warning, Note, and Tip Boxes

The manual uses three types of callout boxes to highlight information:

Table 4: Callout box types.

Box type	Usage
Warning	Common mistakes that cause errors or unexpected output
Note	Important details that may not be obvious
Tip	Suggestions for improved workflow or efficiency

0.17 Keyboard Shortcuts

Keyboard shortcuts are shown using a monospace font with key names joined by `+`:

Table 5: Keyboard shortcut notation.

Notation	Meaning
`Ctrl+C`	Hold Ctrl, then press C
`Ctrl+Shift+S`	Hold Ctrl and Shift, then press S
`Enter`	The Enter (Return) key
`Esc`	The Escape key

Modifier keys are written in the order Ctrl, Shift, Alt, Meta, followed by the primary key.

0.18 Version-Specific Callouts

When a feature requires a minimum version of OmniLaTeX, the manual includes a version callout:

```
% Requires OmniLaTeX v1.10.0 or later
\documentclass[doctype=book, censoring=true]{../../omnilatex}
```

Version requirements are also noted in the text when they affect behaviour:

New in v1.16.0 Features introduced in the current release.

Changed in v1.14.0 Features whose behaviour was modified.

Deprecated since v1.12.0 Features scheduled for removal.

0.19 Glossary Symbols

OmniLaTeX integrates the **glossaries-extra** package to provide consistent symbol and abbreviation management. The following shorthand commands are used throughout this manual:

`\sym{}` References a mathematical symbol entry from the glossary. For instance, `\sym{velocity}` produces the symbol and its associated description as defined in the glossary file.

`\abb{}` References an abbreviation entry. First use expands the abbreviation; subsequent uses display the short form. For example, `\abb{LaTeX}` expands to "LaTeX" on first use and "LaTeX" thereafter.

`\num{}` Formats a number according to the locale settings of the document. This is particularly useful in multilingual documents where number formatting conventions differ.

The glossary definitions themselves are maintained in dedicated `.tex` files under the `glossaries/` directory and are loaded via `\input{}` in the document preamble. Detailed instructions for creating and managing glossary entries can be found in Chapter ??.

Contents

Contents

List of Figures

List of Tables

About This Manual

This manual documents OmniLaTeX v1.16.0, a modular LaTeX document class built on KOMA-Script and LuaLaTeX. It was typeset using OmniLaTeX itself with the `doctype=manual` profile.

How This Manual Is Organised

The manual is structured into numbered parts that group related chapters:

Part I – Foundations Installation, quick start, build system, and class options. Start here if you are new to OmniLaTeX.

Part II – Document Types Dedicated chapters for academic, business, education, visual, and reference doctypes. Each chapter covers the profile's metadata commands, title page design, and recommended usage.

Part III – Typography Fonts, text formatting, mathematics, CJK mathematics, derivatives, and physical math notation.

Part IV – Internationalisation The multilingual system, CJK scripts, RTL scripts, and translation keys.

Part V – Layout Page layout, sectioning, title pages, and institutional profiles.

Part VI – Figures and Tables Figures, tables, TikZ engineering diagrams, and pgfplots.

Part VII – References Bibliography, citation styles, glossaries, and cross-references.

Part VIII – Code and Presentations Code listings, colour themes, boxes, censoring, accessibility, posters, and presentations.

Part IX – Build and CI/CD Build modes, Git integration, Docker, Nix, and CI/CD pipelines.

Part X – Advanced Lua scripting and Lean 4 formal verification.

Part XI – Architecture The module system, class internals, and extension points.

Navigating This Manual

- The **Table of Contents** (preceding this section) provides a complete overview of all chapters and sections.
- The **List of Figures**, **List of Tables**, **List of Examples**, and **List of Listings** provide page references to all numbered floats and code blocks.
- **Cross-references** within the text link to the relevant chapter, section, equation, figure, or table. Clicking any coloured reference navigates directly to the target.
- The **Index** at the end of the manual provides an alphabetical lookup of all key terms, commands, and concepts.
- The **Glossary** collects all symbols, abbreviations, and technical terms used throughout the manual.

Document Metadata

Typographic Conventions

This manual uses consistent typographic formatting to distinguish between different kinds of information. Full details are given in Chapter *Notation & Conventions*; a brief summary follows:

Table 6: Manual publication metadata.

Field	Value
Document class	OmniLaTeX v1.16.0
Doctype	manual (`scrreprt`)
Base classes	KOMA-Script `scrbook`, `scrreprt`, `scrartcl`
Engine	LuaLaTeX (TeX Live 2025+)
Primary font	Libertinus (serif, sans, mono)
Math font	Libertinus Math (`unicode-math`)
Code listings	**`minted`**
Tables	**`booktabs`** + **`tabularray`**
Bibliography	**`biblatex`** + Biber
Glossary	**`glossaries-extra`**
Graphics	**`tikz`** + **`pgfplots`**

Table 7: Typographic conventions summary.

Appearance	Usage
`Monospace`	LaTeX commands, code, filenames
Sans-serif	Key-value options, package options
Bold	Key terms on first occurrence
Italics	Emphasis, introduced terminology
Small Caps	Proper names of standards

Code Examples

All code examples in this manual use the **minted** package for syntax highlighting and are designed to be directly compilable. Each example is enclosed in a `minted` environment with the language specified. Where an example is shortened for brevity, the ellipsis marker … is used and the omission is described in the surrounding text.
The `minted` package requires Python and the Pygments library to be installed. Ensure that `pip install Pygments` has been run before compiling documents that contain code listings.

Contribution and Feedback

OmniLaTeX is an open-source project and contributions are welcome. The canonical source is maintained on GitHub:

Source `https://github.com/WyattAu/OmniLaTeX-template`

Issues Use GitHub Issues for bug reports and feature requests

Patches Fork, create a feature branch, and submit a pull request

When reporting issues with this manual specifically, please include the chapter and section number where the problem occurs, along with the OmniLaTeX version printed in the build log.

Part I

Foundations

1 Installation

OmniLaTeX can be set up using several methods, each suited to a different workflow. This chapter covers the recommended Docker approach, the Nix alternative, a manual TeX Live installation, and a one-click DevContainer setup.

1.1 Prerequisites

Before choosing an installation method, review the requirements for each approach:

Docker (recommended) Docker Engine 20.10 or later. The OmniLaTeX image bundles a complete TeX Live installation, Python tooling, fonts, and all external dependencies. No separate TEX installation is required on the host.

Nix (alternative) Nix with Flakes enabled. The `flake.nix` declares all TEX packages explicitly, producing a fully reproducible development shell.

TeX Live 2026 (manual) A full TeX Live installation including LuaLATEX, Biber, `bib2gls`, Inkscape, Gnuplot, and Python 3 with Pygments. Most users should prefer Docker or Nix.

All methods require Git to clone the repository:

```
git clone https://github.com/WyattAu/OmniLaTeX-template.git
cd OmniLaTeX-template
```

1.2 Docker Method

The Docker image is the recommended way to use OmniLaTeX. It provides a self-contained build environment with TeX Live, system fonts (Libertinus, Monaspace, Atkinson Hyperlegible Next), and all auxiliary tools pre-installed.

1.2.1 Pulling the Image

```
docker pull ghcr.io/wyattau/omnilatex-docker:latest
```

The image is published to the GitHub Container Registry (ghcr.io). The `latest` tag tracks the most recent TeX Live release.

1.2.2 Verifying the Installation

Confirm the image works by invoking `latexmk` through the container entrypoint:

```
docker run --rm -v "$(pwd):/workspace" \
    ghcr.io/wyattau/omnilatex-docker:latest --version
```

The entrypoint script at `/usr/local/bin/entrypoint.sh` forwards arguments to `latexmk` by default. This means you can compile any document by passing its filename directly.

1.2.3 Running Your First Build

Mount the repository into the container and run the full build:

```
docker run --rm -v "$(pwd):/workspace" \
    ghcr.io/wyattau/omnilatex-docker:latest \
    python3 build.py build-all --mode dev
```

This compiles the root document and all example projects. The resulting PDFs are written to the `build/` directory on the host.

1.2.4 Bypassing the Entrypoint

To execute arbitrary commands inside the container (e.g. running tests or opening a shell), override the entrypoint:

```
docker run --rm -it --entrypoint bash \
    -v "$(pwd):/workspace" \
    ghcr.io/wyattau/omnilatex-docker:latest
```

Inside the shell, all TeX Live binaries, Python packages, and fonts are on `PATH`.

1.3 Nix Method

The `flake.nix` at the repository root declares the complete OmniLaTeX development environment as a Nix flake. This method guarantees reproducibility: every collaborator gets identical package versions.

1.3.1 Entering the Dev Shell

```
nix develop .#
```

This drops you into a shell with TeX Live (via `texlive.combine`), Python 3 with Pygments and Rich, Inkscape, Gnuplot, and Lean 4 available.
The TeX Live scheme is `scheme-medium` plus explicit package declarations extracted from every `\usepackage` and `\RequirePackage` call in `lib/*.sty` and `omnilatex.cls`. This avoids the ~4 GB `scheme-full` while covering all OmniLaTeX dependencies.

1.3.2 Building from the Nix Shell

```
python3 build.py build-all
```

For Lean 4 formal verification targets:

```
lake build
```

1.3.3 Building Individual Examples with Nix

The flake exposes per-example packages. To build a single example as a Nix derivation:

```
nix build .#examples-article
```

The resulting PDF is placed at `result/omnilatex-article.pdf`.

1.4 TeX Live Method

After OmniLaTeX is published on CTAN, the class file and all bundled style files can be installed via the TeX Live package manager:

```
tlmgr install omnilatex
```

This installs `omnilatex.cls` and the `lib/` style library into the local TeX Live tree. You will still need to install the external tools listed in Section ?? separately.
For development (using the class from the repository without a CTAN release), set the `TEXINPUTS` environment variable:

```
export TEXINPUTS="/path/to/OmniLaTeX-template:"
```

1.5 DevContainer

VS Code users can open the repository directly as a DevContainer. The configuration in `.devcontainer/devcontainer.json` launches the OmniLaTeX Docker image with recommended extensions pre-installed:

- `ms-python.python` – Python language support.
- `james-yu.latex-workshop` – LaTeX compilation recipes configured for dev and prod modes.
- `streetsidesoftware.code-spell-checker` – Spell checking.

To start:

1. Open the repository in VS Code.
2. When prompted, select Reopen in Container.
3. VS Code builds the container and installs extensions automatically.

The `postCreateCommand` runs `python3 build.py preflight` to verify the environment on first launch.

1.6 Verification

After installation, verify that everything works by running the full test suite:

```
python3 build.py test
```

To check all example documents compile successfully:

```
python3 build.py build-examples --mode dev
```

A successful run produces PDFs for all 43 examples in the `build/` directory. The build orchestrator checks for PDF existence (not exit codes) to determine success, since some LaTeX warnings do not constitute failures.
For per-example timing information:

```
python3 build.py build-all --timings
```

1.7 Troubleshooting

Table ?? lists the most common installation problems and their solutions.

Table 1.1: Common installation issues and resolutions.

Symptom	Resolution
Font not found (Libertinus, Monaspace)	Use the Docker image, which bundles all custom fonts. For Nix, install fonts system-wide and regenerate the fontconfig cache with `fc-cache -f -v`.
Permission denied on `build/`	Docker creates files as UID/GID 1000 by default. If your host UID differs, pass `-e USER_ID=$(id -u)` or adjust volume permissions.
Out of memory during compilation	TeX Live with **tikz** and **pgfplots** can require >2 GB of RAM. Increase Docker's memory limit in Desktop settings, or close other applications.
`pygmentize` not found	The **minted** package requires Python's Pygments library. Install it with `pip install pygments` or use the Docker image where it is pre-installed.
`bib2gls` not found	Required by **glossaries-extra**. Install via `tlmgr install bib2gls` or use Docker/Nix.
`luaotfload` font cache error	Run `luaotfload-tool --update` to regenerate the font name database. This is done automatically in the Docker image.

2 Quick Start

This chapter walks through creating, configuring, and building an OmniLaTeX document from scratch. By the end, you will have a complete article with a figure, a table, and a bibliography entry.

2.1 Minimal Document

The smallest possible OmniLaTeX document requires only five lines:

```
\documentclass{omnilatex}
\begin{document}
\section{Hello World}
This is a minimal OmniLaTeX document.
\end{document}
```

Each line serves a purpose:

1. `\documentclass{omnilatex}` – Loads the OmniLaTeX class. With no options specified, all defaults are applied: doctype=article, language=english, and the default KOMA-Script base class (`scrartcl`).
2. `\begin{document}` – Marks the start of the document body. The preamble (everything before this line) is where you place configuration, package loading, and metadata.
3. `\section{Hello World}` – A top-level section heading. OmniLaTeX inherits KOMA-Script's sectioning commands, which support optional short titles for the table of contents.
4. The paragraph text – Rendered in the default Libertinus Serif font with **microtype** optimisations applied automatically.
5. `\end{document}` – Ends the document body and triggers finalisation (writing the table of contents, glossary, bibliography, etc.).

Compile with:

```
latexmk -lualatex main.tex
```

2.2 Adding Options

OmniLaTeX accepts key-value options via the `\documentclass` command. Options are comma-separated in square brackets.

2.2.1 Language

```
\documentclass[language=german]{omnilatex}
```

This loads **polyglossia** and sets German as the document language, activating hyphenation patterns and localised strings (e.g. "Inhaltsverzeichnis" instead of "Table of Contents"). Supported languages include English, German, French, Japanese, Chinese, Korean, Arabic, and Hebrew.

2.2.2 Document Type

```
\documentclass[doctype=report]{omnilatex}
```

The doctype option selects the document profile. Each doctype configures the KOMA-Script base class, page layout, and feature set. Available doctypes include article, report, thesis, book, presentation, poster, and cv. The full doctype system is documented in Chapter ??.

2.2.3 Combining Options

Options can be combined freely:

```
\documentclass[
  doctype=thesis,
  language=english,
  loadGlossaries,
]{omnilatex}
```

The loadGlossaries option activates the **glossaries-extra** integration, loading the symbol and abbreviation management subsystem.

2.3 Your First Real Document

The following example demonstrates a complete OmniLaTeX article with a title, author, abstract, sections, a figure, a table, and a bibliography entry.

```
\documentclass[
  doctype=article,
  language=english,
]{omnilatex}

\title{An Analysis of Thermal Conductivity in Nanofluids}
\author{Jane Doe}
\date{\today}

\begin{document}
\maketitle

\begin{abstract}
  This article presents experimental measurements of thermal
  conductivity in water-based alumina nanofluids at volume fractions
  ranging from 0.5\% to 4.0\%.  Results show a non-linear enhancement
  that deviates from the Maxwell model at higher concentrations.
\end{abstract}

\section{Introduction}
Nanofluids -- suspensions of nanometre-sized particles in base fluids
-- have attracted significant interest for heat transfer enhancement
\cite{choi1995nanofluids}.

\section{Methodology}
Experiments were conducted using a transient hot-wire apparatus.  The
test section and measurement procedure are shown in
\Cref{fig:apparatus}.

\begin{figure}[htbp]
  \centering
  \fbox{\parbox{0.6\textwidth}{\centering\vspace{2cm}%
```

```
    Schematic of the transient hot-wire apparatus\vspace{2cm}}}
  \caption{Experimental apparatus for thermal conductivity measurement.}
  \label{fig:apparatus}
\end{figure}

\section{Results}
\Cref{tab:results} summarises the measured thermal conductivity ratios.

\begin{table}[htbp]
  \centering
  \caption{Thermal conductivity enhancement at $T = 25\,\si{\degreeCelsius}$}
  \label{tab:results}
  \begin{tabular}{@{} S[table-format=1.1] S[table-format=1.3] @{}}
      \toprule
      {$\phi$ [\%]} & {$k_{\mathrm{nf}} / k_{\mathrm{bf}}$} \\
      \midrule
      0.5 & 1.023 \\
      1.0 & 1.051 \\
      2.0 & 1.098 \\
      4.0 & 1.142 \\
      \bottomrule
  \end{tabular}
\end{table}

\begin{thebibliography}{1}
  \bibitem{choi1995nanofluids}
    S.~U.~S.~Choi, ``Enhancing thermal conductivity of fluids with
    nanoparticles,'' \emph{ASME Publications FED}, vol.~231, pp.~99--105,
    1995.
\end{thebibliography}
\end{document}
```

Key observations about this example:

- The `S` column type from **`siunitx`** is used for aligned numeric columns with automatic unit formatting.
- `\si{\degreeCelsius}` provides proper typesetting of the degree Celsius symbol.
- `\Cref{}` from **`cleveref`** automatically prepends “Figure” or “Table” to cross-references.
- The abstract environment is provided by KOMA-Script and styled automatically by the OmniLaTeX class.

2.4 Building

OmniLaTeX provides a Python-based build orchestrator (`build.py`) that replaces traditional Makefiles. To build the example article:

```
python3 build.py build-example article --mode dev
```

This compiles the article through `latexmk` with three passes (sufficient for cross-references in development mode) and places the output PDF in `build/examples/article/main.pdf`.
To build from within Docker:

```
docker run --rm -v "$(pwd):/workspace" \
    ghcr.io/wyattau/omnilatex-docker:latest \
    python3 build.py build-example article --mode dev
```

2.5 Next Steps

With the basics covered, the following chapters provide deeper dives into specific topics:

- **??** – Build commands, modes, caching, and performance optimisation.
- **??** – All class options and their effects.
- **??** – The full doctype profile system.
- **??** – Detailed guides for each document type (articles, theses, presentations, posters, etc.).
- **??** – Fonts, text formatting, mathematics, and CJK typesetting.

3 Build System

OmniLaTeX uses a Python-based build orchestrator, `build.py`, to manage compilation of all documents in the repository. At 2060 lines, it replaces traditional Makefiles with explicit support for multiple build modes, incremental builds via source-hash caching, parallel compilation, and rich terminal output.

3.1 Overview

The build system is designed around several principles:

- **Explicit modes.** Every build selects a mode (dev, prod, or ultra) that determines the number of LaTeX passes, bibliography processing, and glossary generation.
- **Incremental by default.** A source-hash cache in `build/build_cache.json` skips unchanged documents, reducing rebuild times from minutes to seconds.
- **Parallel execution.** Examples are built concurrently using a thread pool, with progress displayed via the `rich` library or a TQDM fallback.
- **Success via PDF existence.** The orchestrator considers a build successful when a PDF file is produced, not when `latexmk` returns exit code zero. This accommodates LaTeX warnings that do not indicate actual failures.

3.2 Build Commands

Table ?? lists all top-level commands accepted by `build.py`.
Usage examples:

```
python3 build.py build-all
python3 build.py build-example article presentation
python3 build.py clean
python3 build.py list
```

3.3 Global Flags

The following flags apply to all build commands:

3.4 Build Modes

Each build mode controls how many LaTeX passes are executed and whether auxiliary processors (Biber, `bib2gls`) are invoked.

3.4.1 Dev Mode

The default mode. Runs three LaTeX passes with Biber and `bib2gls` processing. Three passes are sufficient to resolve most cross-references and citations during active editing. Use this mode for day-to-day development.

Table 3.1: Build orchestrator commands.

Command	Description
`build-all`	Alias for the default build. Compiles the root document and all examples.
`build-examples`	Build all example projects in `examples/` concurrently.
`build-example`	Build one or more named examples. Accepts positional arguments: `build.py build-example article thesis`.
`build-root`	Build only the root document (`main.tex` at the repository root).
`clean`	Full cleanup. Removes the entire `build/` directory, `_minted/` caches, and `svg-inkscape/` caches.
`clean-aux`	Remove auxiliary files (`.aux`, `.log`, `.toc`, `.bbl`, etc.) while preserving output PDFs.
`list`	List all available examples with their status (built/not built).
`preflight`	Run preflight checks: verify `latexmk`, `biber`, `bib2gls`, `pygmentize`, Inkscape, and Gnuplot are available on `PATH`.
`test`	Run the Python test suite (pytest-based). Does not require a full LaTeX installation for unit tests; integration tests require Docker.

Table 3.2: Global build flags.

Flag	Description
`--mode {dev,prod,ultra}`	Select the build mode. Defaults to dev. See ?? for details.
`--force`	Ignore the source-hash cache and rebuild everything from scratch. Equivalent to passing `-g` to `latexmk`.
`--timings`	Print per-example wall-clock times after the build completes. Useful for identifying slow examples.
`--verbose`	Enable verbose output from `latexmk` and subprocesses. Can also be set via the `OMNILATEX_VERBOSE=1` environment variable.
`--source-date-epoch`	Set the `SOURCE_DATE_EPOCH` environment variable for reproducible builds. Accepts a Unix timestamp.

Table 3.3: Build mode comparison.

Feature	dev	prod	ultra
LaTeX passes	3	Full	2
Biber (bibliography)	Yes	Yes	No
bib2gls (glossary)	Yes	Yes	No
Target use case	Editing	Release	Draft
Typical wall time	~5 min	~14 min	~2 min

```
python3 build.py build-example article --mode dev
```

3.4.2 Prod Mode

Runs `latexmk` in its default mode, which performs as many passes as needed until all references, citations, and glossary entries are fully resolved. This is the mode used for final publication builds.

```
python3 build.py build-all --mode prod
```

3.4.3 Ultra Mode

A minimal two-pass build with no bibliography or glossary processing. Designed for quick draft previews where reference accuracy is not critical.

```
python3 build.py build-example thesis --mode ultra
```

3.5 Incremental Builds

The build system avoids redundant compilation through source-hash caching.

3.5.1 How It Works

Before building an example, `build.py` computes a SHA-256 hash of all `.tex`, `.bib`, `.sty`, and `.cls` files in the example directory and its dependencies. This hash is stored in `build/build_cache.json` alongside the example name.
On subsequent builds, the current hash is compared against the cached value:

- **Cache hit** – The hash matches. The example is skipped entirely, and the existing PDF is reused.
- **Cache miss** – The hash differs (or no entry exists). The example is rebuilt, and the cache is updated with the new hash.

3.5.2 Bypassing the Cache

Pass `--force` to ignore the cache and rebuild all targets:

```
python3 build.py build-all --force
```

This is equivalent to removing `build/build_cache.json` manually. The `--force` flag also passes `-g` to `latexmk`, which forces a full recompilation even if `latexmk`'s own dependency tracking considers the target up to date.

3.6 latexmk Configuration

The `.latexmkrc` file at the repository root configures `latexmk` behaviour for all OmniLaTeX documents:

- **Engine.** LuaLaTeX is the default and only supported engine. OmniLaTeX relies on LuaTeX features including **`fontspec`**, **`unicode-math`**, and **`luatexja`** for CJK support.
- **Interaction mode.** `-interaction=nonstopmode` is used for all builds. Errors are reported in the log but do not pause compilation.
- **Output directory.** PDFs and auxiliary files are written to `build/`, keeping the source tree clean.
- **Minted cache.** The `_minted/` directory is configured as a cache for **`minted`** output to avoid redundant Pygments invocations.

The entrypoint script in the Docker image also reads `.latexmkrc` from the workspace, so the same configuration applies in containerised builds.

3.7 Parallel Builds

Example builds execute concurrently using Python's `ThreadPoolExecutor`. The default concurrency depends on the environment:

- **CI environments** (GitHub Actions, GitLab CI): 4 concurrent workers.
- **Local builds**: `os.cpu_count()` or 4, whichever is lower.

The job count can be overridden with the `--jobs` flag:

```
python3 build.py build-examples --jobs 8
```

Progress is displayed using the `rich` library when available, showing a live table with per-example status (building/done/failed). If `rich` is not installed, a simple TQDM progress bar is used as a fallback. Thread safety is ensured by creating the `build/examples/` directory early and isolating per-example output into separate subdirectories.

3.8 Performance

3.8.1 Full Build Timing

A complete `build-all` in prod mode compiles the root document and all 43 examples in approximately 14 minutes on a modern machine. The same build in ultra mode completes in roughly 2 minutes.

3.8.2 Per-Example Timings

Use the `--timings` flag to see wall-clock time for each example:

```
python3 build.py build-all --mode prod --timings
```

This prints a summary table after all builds complete, sorted by duration. Examples using **`tikz`** or **`pgfplots`** computations are typically the slowest, as the LaTeX engine performs numerical calculations in-document rather than delegating to an external tool.

3.8.3 Performance Tips

- Use dev mode during editing – three passes are sufficient for most cross-reference resolution.
- Use ultra mode for rapid iteration when bibliography and glossary accuracy is not needed.
- Leverage incremental builds: only modified examples are recompiled.
- For **pgfplots** heavy documents, pre-compute data externally and use `\pgfplotstableread` instead of in-document calculations.
- Increase `--jobs` on machines with many cores, but note that each LuaLaTeX instance can consume significant memory.

4 Class Options

OmniLaTeX accepts key-value options through the standard `\documentclass` interface. This chapter documents all 15 class options, their defaults, valid values, and interactions.

4.1 Loading Options

Options are passed as comma-separated entries in square brackets before the class name:

```
\documentclass[doctype=article, language=german, loadGlossaries]{omnilatex}
```

Two syntactic forms are supported:

- **Boolean flags** – standalone keywords that enable a feature. Example: loadGlossaries, todonotes, a5.
- **Key-value pairs** – written as `key=value`. Example: language=english, doctype=thesis, institution=tuhh.

Options are processed by the **kvoptions** package (`family=omnilatex`, `prefix=omnilatex@`). Unrecognised options are forwarded to the underlying KOMA-Script base class via `\DeclareDefaultOption`.
The option processing order is:

1. `kvoptions` parses all declared options.
2. The doctype value is normalised to lowercase and matched against the alias table in `omnilatex.cls`.
3. The titlestyle value is normalised and validated.
4. Matched KOMA class options are combined with any user-supplied options and passed to `\LoadClass`.

4.2 Option Reference

Table ?? lists all 15 class options.

4.3 Option Details

4.3.1 Document Metadata

4.3.1.1 language

Sets the document language for **polyglossia** (via **tracklang**). Activates language-specific hyphenation patterns, localised strings (e.g. “Inhaltsverzeichnis” for German), and automatic CJK or RTL support when applicable.

Table 4.1: Complete class option reference.

Option	Type	Default	Description	Chapter
language	string	`english`	Document language for hyphenation and localisation	??
doctype	string	`thesis`	Document profile selecting KOMA base class and layout	??
institution	string	`none`	Institutional template (e.g. `tuhh`)	??
titlestyle	string	`book`	Title page layout variant	??
censoring	bool	false	Enable `\censored` command	??
loadGlossaries	bool	false	Load **`glossaries-extra`** integration	??
todonotes	bool	false	Load **`todonotes`** markup commands	–
enablefonts	bool	false	Enable extended font selection module	??
enablegraphics	bool	false	Enable standalone graphics module	??
enablemath	bool	true	Load **`unicode-math`** and math support	??
enabletikz	bool	true	Load **`tikz`** core and graphics module	??
enableengineering	bool	true	Load engineering drawing libraries (circuits, gantts)	??
enablecode	bool	true	Load **`minted`** code listing support	??
enabletables	bool	true	Load **`booktabs`** and table module	??
a5	void	–	Switch to A5 paper with 10 pt font	??

Valid values: `english`, `german`, `french`, `japanese`, `chinese`, `simplifiedchinese`, `traditionalchinese`, `korean`, `arabic`, `hebrew`, `persian`.
Handled by: `lib/language/omnilatex-i18n.sty`, with automatic conditional loading of `omnilatex-cjk.sty` or `omnilatex-rtl.sty`.

```
\documentclass[language=german]{omnilatex}
```

4.3.1.2 doctype

Selects the document profile that determines the KOMA-Script base class, page layout, sectioning depth, and feature set. Over 80 user-friendly aliases map to 26 canonical doctypes backed by 3 KOMA classes. The full system is documented in Chapter ??.
Default: `thesis`. Handled by: `omnilatex.cls` lines 111–147.

```
\documentclass[doctype=article]{omnilatex}
\documentclass[doctype=technicalreport]{omnilatex}
```

4.3.1.3 institution

Loads an institutional configuration package that provides branding, logo placement, and institution-specific title page commands. The class first checks for a local `config/institution.sty`, then falls back to shared configurations under `config/institutions/`.
Valid values: `none` (default), `tuhh`, or any directory name under `config/institutions/`.

```
\documentclass[institution=tuhh]{omnilatex}
```

4.3.1.4 titlestyle

Selects the title page layout variant. Valid values: `book`, `thesis`, `article`, `simple`, `tuhh`. Unknown values produce an informational message and fall back to the default layout.

```
\documentclass[titlestyle=tuhh]{omnilatex}
```

4.3.2 Feature Flags

4.3.2.1 censoring

Enables the `\censored{text}` command, which replaces text with black rectangles while preserving layout. Useful for redacted drafts.

```
\documentclass[censoring]{omnilatex}
```

4.3.2.2 loadGlossaries

Loads the **glossaries-extra** integration module (`lib/references/omnilatex-glossary.sty`), enabling `\newacronym`, `\newglossaryentry`, and `\printglossaries`.

```
\documentclass[loadGlossaries]{omnilatex}
```

4.3.2.3 todonotes

Loads **todonotes**, providing `\todo{}`, `\missingfigure{}`, and inline annotation commands.

```
\documentclass[todonotes]{omnilatex}
```

4.3.2.4 enablefonts

Enables extended font selection beyond the default Libertinus family. Without this option, only the core font module is loaded.

```
\documentclass[enablefonts]{omnilatex}
```

4.3.2.5 enablegraphics

Enables the standalone graphics module (separate from the TikZ bundle). Most documents do not need this as graphics support is included with enabletikz.

```
\documentclass[enablegraphics]{omnilatex}
```

4.3.2.6 enablemath

Loads **unicode-math** and the math support module (`lib/typography/omnilatex-math.sty`). Enabled by default. Disable for documents that contain no mathematics to reduce load time.

```
\documentclass[enablemath=false]{omnilatex}
```

4.3.2.7 enabletikz

Loads **tikz** core libraries and the graphics module (`lib/graphics/omnilatex-graphics.sty`, `lib/graphics/omnilatex-tikz-core.sty`). Enabled by default.

```
\documentclass[enabletikz=false]{omnilatex}
```

4.3.2.8 enableengineering

Loads engineering-specific TikZ libraries (`lib/graphics/omnilatex-tikz-engineering.sty`) including circuit diagrams, Gantt charts, and technical drawings. Enabled by default. Requires enabletikz.

```
\documentclass[enableengineering=false]{omnilatex}
```

4.3.2.9 enablecode

Loads the code listing module (`lib/code/omnilatex-listings.sty`) built on **minted**. Enabled by default.

```
\documentclass[enablecode=false]{omnilatex}
```

4.3.2.10 enabletables

Loads the table module (`lib/tables/omnilatex-tables.sty`) providing **booktabs**, **siunitx** column types, and **multirow**/**multicol** support. Enabled by default.

```
\documentclass[enabletables=false]{omnilatex}
```

4.3.3 Layout

4.3.3.1 a5

A void option (no value) that switches the page size to A5 and sets the font size to 10 pt via `\PassOptionsToPackage`. Internally passes `paper=a5` to **typearea** and adds `fontsize=10pt` to the user class options.

```
\documentclass[a5]{omnilatex}
```

4.4 Common Combinations

Academic thesis `doctype=thesis, loadGlossaries, todonotes` – Full thesis profile with glossary and annotation support for draft review.

Journal article `doctype=journal, language=english` – Compact article layout with titlepage=true and parskip=half.

German report `doctype=report, language=german, loadGlossaries` – Report layout with German localisation and glossary integration.

Minimal draft `doctype=article, enablemath=false, enabletikz=false, enablecode=false` – Strips all optional modules for fastest compilation.

Conference poster `doctype=poster, enabletikz=true, enableengineering=true` – A1 poster layout with full TikZ and engineering drawing support.

TUHH thesis `doctype=thesis, institution=tuhh, titlestyle=tuhh, loadGlossaries` – Institution-branded thesis with TUHH title page and glossary.

5 Document Type System

OmniLaTeX provides a document type system that maps user-friendly names to KOMA-Script base classes. Over 80 aliases resolve to 26 canonical doctypes, each backed by one of three KOMA classes (`scrartcl`, `scrreprt`, `scrbook`).

5.1 Architecture

The doctype system uses a three-tier resolution pipeline:

1. **User alias** – The string passed to `doctype=` (e.g. `paper`, `talk`, `tech-report`).
2. **Canonical name** – The internal doctype identifier (e.g. `article`, `presentation`, `technicalreport`).
3. **KOMA class** – The underlying KOMA-Script base class (`scrartcl`, `scrreprt`, or `scrbook`) with associated default options.

This design lets users write intuitive names while the class handles the complexity of KOMA-Script configuration. For example, `doctype=thesis` resolves through `thesis` → `scrbook` with options `listof=totoc,bibliography=totoc,chapterprefix=true,open=right`.
Resolution is implemented via `\omnilatex@matchdoctypes` in `omnilatex.cls`, which iterates comma-separated alias lists and calls `\omnilatex@setdoctype` on the first match. The doctype string is normalised to lowercase before matching. Unrecognised doctypes trigger a warning and fall back to `book`.

5.2 KOMA Class Base

Each KOMA class carries a distinct set of default options, defined in `omnilatex.cls`:
Two additional option sets exist for specialised doctypes:

- `\omnilatex@inlinepaperoptions` – Same as article options with `titlepage=false`.
- `\omnilatex@journaloptions` – Same as article options with `titlepage=true`.
- `\omnilatex@cvoptions` – `numbers=noenddot, parskip=full, fontsize=11pt`.

5.3 Complete Alias Table

The following tables list all aliases grouped by their canonical doctype and KOMA base class.

Table 5.1: KOMA class base options.

KOMA Class	Options Macro	Default Options
scrbook	\omnilatex@bookoptions	listof=totoc, bibliography=totoc, chapterprefix=true, open=right, numbers=noenddot
scrreprt	\omnilatex@reportoptions	listof=totoc, bibliography=totoc, chapterprefix=false, open=any, numbers=noenddot
scrartcl	\omnilatex@articleoptions	bibliography=totoc, numbers=noenddot, parskip=half

Table 5.2: Aliases resolving to scrbook.

Canonical	Aliases
book	book, books
thesis	thesis, theses
dissertation	dissertation, dissertations
dictionary	dictionary, dictionaries, lexicon, lexicons

Table 5.3: Aliases resolving to scrreprt.

Canonical	Aliases
manual	manual, manuals, guide, guides, handbook, handbooks
technicalreport	report, reports, technicalreport, technical-report, technicalreports, technical-reports, techreport, te
standard	standard, standards
patent	patent, patents
research-proposal	research-proposal, researchproposal, research-proposals

Table 5.4: Aliases resolving to `scrartcl`.

Canonical	Aliases
article	article, articles, paper, papers
inlinepaper	inlinepaper, inlinepapers, inline-research, inline-research-paper
journal	journal, journals, magazine, magazines
cv	cv, resume, resumes, curriculumvitae
cover-letter	cover-letter, coverletter
poster	poster, posters
presentation	presentation, presentations, slides, talk, talks
letter	letter, letters
homework	homework, homeworks
exam	exam, exams, examination, examinations
lecture-notes	lecture-notes, lecturenotes, lecture-note
syllabus	syllabus, syllabi
handout	handout, handouts
memo	memo, memos, memorandum, memorandums
white-paper	white-paper, whitepapers
invoice	invoice, invoices
recipe	recipe, recipes

5.3.1 Aliases mapping to `scrbook`

5.3.2 Aliases mapping to `scrreprt`

5.3.3 Aliases mapping to `scrartcl`

5.4 Canonical Doctypes

Table ?? lists all 26 canonical doctypes with their KOMA base class and category. Each canonical doctype has a corresponding `.sty` profile in `config/document-types/`.

5.5 Creating Custom Doctypes

New doctypes are added by creating a `.sty` file in `config/document-types/`. The class automatically loads the profile whose filename matches the canonical doctype name.

5.5.1 Step-by-step

1. Create `config/document-types/myreport.sty`.
2. Define the document type metadata:

```
\NeedsTeXFormat{LaTeX2e}
\ProvidesPackage{config/document-types/myreport}%
    [2026/01/01 OmniLaTeX myreport profile]
\documenttype{My Custom Report}
```

3. Set layout and KOMA options:

```
\documentfontsize{11pt}
\documentlayout{DIV=12,headinclude=false,footinclude=false}
\KOMAoptions{titlepage=true}
\setcounter{secnumdepth}{3}
\setcounter{tocdepth}{2}
```

4. Configure document appearance:

```
\documentcolormode{color}
\documentlinespacing{onehalf}
\documentparspacing{half}
\documentfontmode{serif}
```

5. Optionally define a custom `\maketitle`:

```
\makeatletter
\renewcommand{\maketitle}{%
    \begin{titlepage}
        \centering
        {\huge\bfseries\@title}\par\vspace{1cm}
        {\large\@author}\par\vspace{0.5cm}
        {\normalsize\@date}
    \end{titlepage}
}
\makeatother
```

6. Use with `\documentclass[doctype=myreport]{omnilatex}`. Optionally add aliases in `omnilatex.cls` via `\omnilatex@matchdoctypes`.

5.5.2 Minimal Example

A minimal custom doctype profile:

Table 5.5: All 26 canonical doctypes.

Canonical	KOMA Base	Category	Profile File
article	scrartcl	Academic	article.sty
inlinepaper	scrartcl	Academic	inlinepaper.sty
journal	scrartcl	Academic	journal.sty
cv	scrartcl	Business	cv.sty
cover-letter	scrartcl	Business	cover-letter.sty
letter	scrartcl	Business	letter.sty
memo	scrartcl	Business	memo.sty
invoice	scrartcl	Business	invoice.sty
white-paper	scrartcl	Business	white-paper.sty
homework	scrartcl	Education	homework.sty
exam	scrartcl	Education	exam.sty
lecture-notes	scrartcl	Education	lecture-notes.sty
syllabus	scrartcl	Education	syllabus.sty
handout	scrartcl	Education	handout.sty
poster	scrartcl	Visual	poster.sty
presentation	scrartcl	Visual	presentation.sty
recipe	scrartcl	Personal	recipe.sty
manual	scrreprt	Reference	manual.sty
technicalreport	scrreprt	Academic	technicalreport.sty
standard	scrreprt	Technical	standard.sty
patent	scrreprt	Technical	patent.sty
research-proposal	scrreprt	Academic	research-proposal.sty
book	scrbook	Publishing	book.sty
thesis	scrbook	Academic	thesis.sty
dissertation	scrbook	Academic	dissertation.sty
dictionary	scrbook	Reference	dictionary.sty

```
\NeedsTeXFormat{LaTeX2e}
\ProvidesPackage{config/document-types/brief}
    [2026/01/01 OmniLaTeX brief profile]

\documenttype{Internal Brief}
\documentfontsize{11pt}
\documentlayout{DIV=12}
\KOMAoptions{titlepage=false}
\setcounter{secnumdepth}{2}
\setcounter{tocdepth}{1}

\documentcolormode{color}
\documentlinespacing{single}
\documentparspacing{half}
\documentfontmode{sans}

\makeatletter
\renewcommand{\maketitle}{%
    \begin{center}
        {\Large\bfseries\@title}\par\medskip
        {\normalsize\@author\quad|\quad\@date}
    \end{center}
    \bigskip\noindent\rule{\textwidth}{0.4pt}\par\medskip
}
\makeatother

\endinput
```

5.6 Formal Verification

The doctype-to-class mapping is formally verified in Lean 4. Two proof files establish key properties of the system.

5.6.1 DoctypeClassMapping.lean

Located at `specs/proofs/OmniLaTeXProofs/DoctypeClassMapping.lean`, this file defines three lists of doctype strings grouped by KOMA class and proves the following theorems:

`doctype_class_deterministic` Every doctype maps to at most one KOMA class (determinism via `Option.some.inj`).

`valid_doctype_has_class` A representative sample of 10 doctypes spanning all three classes each resolves to a non-`none` value (proved by `decide`).

`scrartcl_count` `scrartclDoctypes.length = 34`.

`scrbook_count` `scrbookDoctypes.length = 6`.

`scrreprt_count` `scrreprtDoctypes.length = 20`.

`class_covering` The total list length equals the sum of all three class-specific lists ($34 + 6 + 20 = 60$).

`scrartcl_is_largest` `scrartcl` has strictly more doctypes than both `scrbook` ($34 > 6$) and `scrreprt` ($34 > 20$), proved via `omega`.

5.6.2 DocumentSettings.lean

Located at `specs/proofs/OmniLaTeXProofs/DocumentSettings.lean`, this file models the 26 canonical doctypes as an inductive type and proves structural properties of the class partition:

doctype_class_deterministic Each canonical doctype resolves to exactly one KOMA class (functional correctness).

class_partition The 26 doctypes partition into article (15), report (6), and book (5), with all three sets non-empty: $15 + 6 + 5 = 26$.

article_is_largest The article class contains strictly more doctypes than report ($15 > 6$) and book ($15 > 5$).

doctype_count Total canonical doctype count is 26.

Part II

Document Types

6 Articles & Short Documents

This chapter covers the six `scrartcl`-based doctypes in OmniLaTeX. These document types are optimised for single-column or two-column works that do not require chapters: journal submissions, internal papers, business correspondence, and white papers. All share a common KOMA-Script base class with `parskip=half`, `numbers=noenddot`, and automatic bibliography inclusion in the table of contents.

6.1 Common Features

Every article-type doctype inherits the following from the `scrartcl` base class:

- **Sectioning up to `\subsubsection`** – No chapter or part commands; numbered sections, subsections, and subsubsections.
- **Title page control** – Each doctype sets `titlepage` to `true` or `false` depending on its conventions.
- **Single-sided by default** – Article-type documents print single-sided unless the user explicitly passes `twoside`.
- **Bibliography in TOC** – `bibliography=totoc` is included in the default `scrartcl` options.
- **Paragraph spacing** – `parskip=half` provides visual paragraph separation without indentation.

6.2 Choosing the Right Article Type

Table 6.1: Article-type doctype decision guide.

Doctype	Title Page	Best For
article	Yes	Academic or research papers with journal-style layout
journal	Yes	Formal journal submissions with volume/issue metadata
inline-paper	No	Compact preprints resembling arXiv two-column format
white-paper	Yes	Business or technical white papers with callout boxes
letter	No	Formal correspondence with sender/recipient blocks
cover-letter	No	Job application cover letters

6.3 Doctype Profiles

6.3.1 article

The `article` doctype provides a full title page with journal header, abstract, keywords, DOI, and affiliation footer. It uses 11 pt serif font with one-and-a-half line spacing.

Table 6.2: Custom commands for `article`.

Command	Description
`\subtitle{...}`	Sets the abstract text shown on the title page
`\keywords{...}`	Comma-separated keyword list
`\doi{...}`	Digital object identifier
`\contact{...}`	Correspondence contact information
`\affiliation{...}`	Author affiliation(s)

Use `article` for academic papers, conference submissions, or any research document that benefits from a formal title page with metadata fields.

```
\documentclass[doctype=article, language=english]{../../omnilatex}
\title{An Analysis of Thermal Conductivity in Nanofluids}
\author{Jane Doe}
\keywords{nanofluids, thermal conductivity, alumina}
\doi{10.1234/example.2025}
\affiliation{Department of Physics, Example University}
\begin{document}
\maketitle
\section{Introduction}
Nanofluids have attracted significant interest for heat transfer.
\end{document}
```

6.3.2 journal

The `journal` doctype extends the article layout with full journal metadata: volume, issue, page range, article type, highlights, and received/accepted/published dates. The title page includes a right-aligned journal header and a highlights list.
Use `journal` when submitting to a specific journal or when you need volume/issue tracking and peer-review date stamps.

```
\documentclass[doctype=journal]{../../omnilatex}
\title{Non-linear Enhancement in Alumina Nanofluids}
\author{Jane Doe}
\journalname{Journal of Thermal Sciences}
\journalvolume{42}\journalissue{3}\journalyear{2025}
\journalpages{115--128}
\articletype{Original Research}
\receiveddate{2025-01-15}\accepteddate{2025-04-20}
\publisheddate{2025-05-01}
\begin{document}
\maketitle
\section{Introduction}
\end{document}
```

Table 6.3: Custom commands for `journal`.

Command	Description
`\journalname{...}`	Journal name
`\journalvolume{...}`	Volume number
`\journalissue{...}`	Issue number
`\journalpages{...}`	Page range (e.g. `1--12`)
`\journalyear{...}`	Publication year
`\articletype{...}`	Type (Original Research, Review, etc.)
`\highlights{...}`	Comma-separated highlight list
`\receiveddate{...}`	Date manuscript was received
`\acceptеddate{...}`	Date manuscript was accepted
`\publisheddate{...}`	Date manuscript was published

6.3.3 inline-paper

The `inline-paper` doctype produces a compact, arXiv-style layout with no separate title page. The title, authors, and abstract are rendered inline at the top of the first page in a single column, then the body switches to two-column format via `\twocolumn`. It uses 10 pt font with `DIV=16` for tighter margins.

Table 6.4: Custom commands for `inline-paper`.

Command	Description
`\subtitle{...}`	Abstract text displayed inline
`\keywords{...}`	Comma-separated keywords
`\affiliation{...}`	Author affiliation(s)
`\contact{...}`	Correspondence e-mail or address
`\supplementary{...}`	Supplementary material link

Use `inline-paper` for preprints, arXiv submissions, or any document where a compact two-column layout is preferred over a full title page.

```
\documentclass[doctype=inline-paper]{../../omnilatex}
\title{Compact Preprint Title}
\author{Author One \and Author Two}
\affiliation{Department of Physics, Example University}
\subtitle{This is the abstract text for the inline paper.}
\keywords{preprint, arXiv, compact layout}
\begin{document}
\maketitle
\section{Introduction}
The body text flows in two columns after the inline header.
\end{document}
```

6.3.4 white-paper

The `white-paper` doctype is designed for business or technical white papers. It provides a branded title page with organisation, version, and classification metadata, plus two custom environments: `whitepaperabstract` for an executive summary and `keytakeaways` for a highlighted conclusions box. The `\callout{title}{text}` command creates inline callout boxes using **tcolorbox**.

Table 6.5: Custom commands and environments for `white-paper`.

Name	Type	Description
`\wptitle{...}`	Command	Override the displayed title
`\wporganization{...}`	Command	Publishing organisation
`\wpversion{...}`	Command	Document version (default: Draft)
`\wpconfidential{...}`	Command	Classification level
`\wpcontact{...}`	Command	Contact information
`\wpdate{...}`	Command	Custom publication date
`whitepaperabstract`	Env.	Executive summary block
`keytakeaways`	Env.	Green-framed key findings box
`\callout`	Command	Inline blue callout box

Use `white-paper` for technology assessments, policy briefs, or industry reports that benefit from structured abstracts and highlighted findings.

```
\documentclass[doctype=white-paper]{../../omnilatex}
\title{AI in Software Development}
\wporganization{Acme Research}
\wpversion{1.0}\wpdate{May 2025}
\begin{document}
\maketitle
\begin{whitepaperabstract}
This white paper examines the impact of AI on software engineering.
\end{whitepaperabstract}
\section{Introduction}
\callout{Key Finding}{Teams using AI tools ship 60\% faster.}
\begin{keytakeaways}
AI accelerates development when paired with automated testing.
\end{keytakeaways}
\end{document}
```

6.3.5 letter

The `letter` doctype produces a formal letter with no section numbering, no table of contents, and no page numbers. The `\maketitle` command renders a sender/recipient block, date, subject line, and salutation. The `\letterclosingblock` command prints the closing and signature.
Use `letter` for formal business correspondence, complaint letters, or any structured letter format.

```
\documentclass[doctype=letter]{../../omnilatex}
\lettersender{Jane Doe\\123 Main St\\Springfield, IL 62701}
\letterrecipient{Mr.\ John Smith\\Acme Corp\\456 Oak Ave}
\lettersubject{Proposal for Phase~II Collaboration}
\letterdate{May 7, 2025}
```

Table 6.6: Custom commands for `letter`.

Command	Description
`\lettersender{...}`	Sender name and address block
`\letterrecipient{...}`	Recipient name and address
`\lettersubject{...}`	Subject line
`\letterdate{...}`	Date (default: `\today`)
`\letteropening{...}`	Salutation (default: Dear Sir or Madam)
`\letterclosing{...}`	Closing (default: Sincerely)
`\letterclosingblock`	Prints closing and sender signature

```
\begin{document}
\maketitle
Thank you for your continued partnership.  We propose the following
scope for Phase~II of our collaboration.
\letterclosingblock
\end{document}
```

6.3.6 cover-letter

The `cover-letter` doctype is a minimal template for job application cover letters. It disables `\maketitle` entirely and provides metadata commands for sender and recipient information. Section numbering and TOC depth are set to zero.

Table 6.7: Custom commands for `cover-letter`.

Command	Description
`\coverletterrecipient{...}`	Recipient name
`\coverletterposition{...}`	Position applied for
`\coverlettercompany{...}`	Company name
`\coverletteraddress{...}`	Company street address
`\coverlettercity{...}`	City, state, ZIP
`\coverletterdate{...}`	Date (default: `\today`)
`\coverlettersendername{...}`	Sender full name
`\coverlettersenderaddress{...}`	Sender street address
`\coverlettersendercity{...}`	Sender city, state, ZIP
`\coverlettersenderphone{...}`	Sender phone number
`\coverlettersenderemail{...}`	Sender e-mail address

Use `cover-letter` for job applications. The template is deliberately minimal–you compose the letter body directly in the document environment using the metadata commands in a `config/document-settings.sty` file.

```
\documentclass[doctype=cover-letter]{../../omnilatex}
```

```
\coverletterposition{Senior Engineer}
\coverlettercompany{Acme Corp}
\coverlettersendername{Jane Doe}
\coverlettersenderemail{jane@example.com}
\begin{document}
Dear Hiring Manager,
I am writing to express my interest in the Senior Engineer role.
\end{document}
```

7 Reports & Long Documents

This chapter covers five `scrreprt`-based doctypes. These document types support chapters via the KOMA-Script report class, making them suitable for multi-section technical works, proposals, and reference documents. All share `listof=totoc`, `bibliography=totoc`, `chapterprefix=false`, `open=any`, and `numbers=noenddot` from the default `scrreprt` options.

7.1 Common Features

Every report-type doctype provides:

- **Chapter-level sectioning** – Full `\chapter` command for top-level divisions, with sections and subsections below.
- **Open on any page** – `open=any` allows chapters to start on even or odd pages, saving space.
- **Lists in TOC** – `listof=totoc` automatically includes the list of figures, list of tables, and list of algorithms in the table of contents.
- **No chapter prefix** – `chapterprefix=false` omits the "Chapter N" prefix from chapter headings.

7.2 Choosing the Right Report Type

Table 7.1: Report-type doctype decision guide.

Doctype	Best For
report / technical-report	Internal or client-facing technical documents with report numbers
research-proposal	Grant applications with PI, budget, and programme metadata
standard	Formal standards documents with designation and ICS codes
patent	Patent specifications with legal-style sectioning
manual	Product manuals, guides, or handbooks

7.3 Doctype Profiles

7.3.1 report / technical-report

The `technicalreport` doctype (aliased as `report`, `tech-report`, etc.) produces a formal technical report with a title page containing report number, revision, author, client, project name, confidentiality level, and a personnel table. It is single-sided with chapter-level sectioning.

Table 7.2: Custom commands for `technicalreport`.

Command	Description
`\reportnumber{...}`	Report identifier (default: TR-000)
`\revision{...}`	Revision letter (default: A)
`\preparedby{...}`	Preparing group or author
`\client{...}`	Client organisation
`\projectname{...}`	Associated project name
`\confidentiality{...}`	Classification (default: Internal Use Only)
`\sponsor{...}`	Executive sponsor name
`\leadengineer{...}`	Lead engineer name
`\doccontrol{...}`	Document control contact
`\distribution{...}`	Distribution statement

Use `technicalreport` for engineering reports, consultant deliverables, or any document that tracks report numbers and revisions.

```
\documentclass[doctype=technical-report]{../../omnilatex}
\title{Structural Analysis of Bridge Deck Section B-7}
\reportnumber{TR-2025-042}\revision{C}
\preparedby{Structural Analysis Group}
\client{Department of Transportation}
\projectname{Highway 101 Overpass Rehabilitation}
\confidentiality{Public Release}
\begin{document}
\maketitle
\tableofcontents
\chapter{Executive Summary}
This report presents the findings of the structural analysis.
\end{document}
```

7.3.2 research-proposal

The `research-proposal` doctype is tailored for grant and fellowship applications. The title page displays the funding programme, call reference, principal and co-investigators, organisation, duration, budget, and start/end dates. It uses `chapterprefix=false` and `open=any` for a compact layout. Use `research-proposal` for ERC, NSF, DFG, or similar funding applications that require structured metadata on the title page.

```
\documentclass[doctype=research-proposal]{../../omnilatex}
\title{Machine Learning for Predictive Maintenance}
\propprogram{Horizon Europe}
\propcall{HORIZON-CL4-2025-01}
\propinvestigator{Dr.\ Jane Doe}
\propcoinvestigators{Dr.\ A.~Smith, Dr.\ B.~Mueller}
\proporganization{Technical University}
\propduration{36 months}
\propbudget{EUR 1,500,000}
\propstartdate{2026-01-01}\propenddate{2028-12-31}
\begin{document}
\maketitle
```

Table 7.3: Custom commands for `research-proposal`.

Command	Description
`\propabstract{...}`	Proposal abstract or summary
`\propprogram{...}`	Funding programme name
`\propcall{...}`	Call for proposals identifier
`\propinvestigator{...}`	Principal Investigator name
`\propcoinvestigators{...}`	Co-Investigator names
`\proporganization{...}`	Host institution or organisation
`\propduration{...}`	Project duration (default: 24 months)
`\propbudget{...}`	Total requested budget
`\propstartdate{...}`	Project start date
`\propenddate{...}`	Project end date

```
\chapter{Project Description}
This proposal aims to develop ML-based predictive maintenance.
\end{document}
```

7.3.3 standard

The `standard` doctype is designed for formal standards documents resembling ISO or IEC publications. The title page shows the standards series, designation number, status, scope, committee, publication date, superseded document, and ICS codes. Section numbering uses plain Arabic numerals without chapter prefixes (e.g. 1, 1.1, 1.1.1).

Table 7.4: Custom commands for `standard`.

Command	Description
`\standardseries{...}`	Standards series name
`\standarddesignation{...}`	Designation number (default: F-000)
`\standardstatus{...}`	Status (Draft, Published, Withdrawn)
`\standardscope{...}`	Scope statement displayed on title page
`\standardcommittee{...}`	Preparing committee name
`\standardsupersedes{...}`	Superseded standard designation
`\standardics{...}`	ICS classification codes
`\standardkeywords{...}`	Subject keywords
`\standarddisclaimer{...}`	Legal disclaimer text

Use `standard` for internal standards, specifications, or normative documents that require designation tracking and formal status indicators.

```
\documentclass[doctype=standard]{../../omnilatex}
\title{Thermal Performance of Building Envelopes}
```

```
\standardseries{Omni Standards Series}
\standarddesignation{B-204}
\standardstatus{Published}
\standardscope{This standard specifies methods for measuring
  the thermal transmittance of building envelope components.}
\standardcommittee{TC 163 -- Building Environment}
\standardics{91.120.10}
\begin{document}
\maketitle
\section{Scope}
This standard applies to opaque and transparent building elements.
\end{document}
```

7.3.4 patent

The `patent` doctype provides a minimal report-class layout for patent specifications. It sets `titlepage=true`, single-sided, with sectioning depth limited to two levels (`secnumdepth=2`, `tocdepth=1`). No custom commands are defined–the doctype relies on standard KOMA-Script commands and the user's own preamble.
Use `patent` for drafting patent applications or publishing patent descriptions where the formal structure (title page, sections, claims) is composed directly by the author.

```
\documentclass[doctype=patent]{../../omnilatex}
\title{Method and Apparatus for Efficient Heat Recovery}
\author{Jane Doe}\date{May 2025}
\begin{document}
\maketitle
\chapter{Field of the Invention}
This invention relates to heat recovery systems for industrial processes.
\chapter{Background}
Existing heat recovery methods suffer from thermal losses exceeding 30\%.
\chapter{Summary}
The present invention provides a heat recovery apparatus with improved
thermal efficiency through a counter-flow heat exchanger design.
\end{document}
```

7.3.5 manual

The `manual` doctype is optimised for product manuals, user guides, and handbooks. It uses a sans-serif font at 11 pt with single line spacing and `DIV=14` for comfortable reading. The front page displays a brand name, document title, firmware or software version, release date, support contact, and a summary paragraph. No separate title page is generated; the front matter appears on the first page with `titlepage=false`.
Use `manual` for hardware manuals, software documentation, or any reference guide that needs version tracking and support contact information.

```
\documentclass[doctype=manual]{../../omnilatex}
\title{Model X-2000 User Guide}
\manualbrand{Acme Devices}
\manualversion{3.2.1}
\manualsupport{support@acme.com}
\manualsummary{This guide covers installation, configuration,
  and operation of the Model X-2000.}
\begin{document}
\maketitle
\tableofcontents
\chapter{Getting Started}
```

Table 7.5: Custom commands for `manual`.

Command	Description
`\manualbrand{...}`	Product or brand name (default: OmniLaTeX Devices)
`\manualversion{...}`	Firmware/software version (default: 1.0)
`\manualreleasedate{...}`	Release date (default: `\today`)
`\manualsupport{...}`	Support contact information
`\manualsummary{...}`	One-paragraph manual summary

```
Unpack the device and verify all components against the parts list.
\end{document}
```

8 Theses & Dissertations

This chapter covers four scrbook-based doctypes for long-form academic works: thesis, dissertation, thesis-tuhh, and dictionary. The manual doctype is also scrbook-backed and is documented in Chapter ??.
All book-type doctypes inherit listof=totoc, bibliography=totoc, chapterprefix=true, open=right, and numbers=noenddot. They support \frontmatter, \mainmatter, and \backmatter for distinct typographic treatment of preliminary pages, body chapters, and appendices.

8.1 Thesis vs. Dissertation

Table 8.1: Comparison of thesis and dissertation doctypes.

	thesis	dissertation
KOMA base	scrbook	scrbook
Font size	12 pt	Default (11 pt)
Sides	Two-sided	Two-sided
secnumdepth	2	3
Line spacing	One-and-a-half	Default
Paragraph spacing	None	Default
Supervisor model	Single advisor	Primary + secondary
Committee	\thesiscommittee	\dissertationcommittee
Degree cmd	\thesisdegree	\dissertationdegree
Keywords	No	Yes (\keywords)

The thesis doctype is designed for master's theses with a single advisor, shallower sectioning depth, and larger 12 pt font. The dissertation doctype targets doctoral works with deeper sectioning, dual supervisor support, and keyword fields for library cataloguing.

8.2 Doctype Profiles

8.2.1 thesis

The thesis doctype produces a formal thesis title page with institution, department, degree, advisor, committee, and defense date. It uses 12 pt serif font, one-and-a-half line spacing, and no paragraph indentation. The document is two-sided with binding correction support via the BCOR option.
Use thesis for master's theses or similar graduate-level documents that require a formal academic title page.

Table 8.2: Custom commands for `thesis`.

Command	Description
`\thesisinstitution{...}`	University or institution name
`\thesisdepartment{...}`	Department or school name
`\thesisdegree{...}`	Degree (default: Master of Science)
`\thesisadvisor{...}`	Thesis advisor name
`\thesiscommittee{...}`	Committee members (comma-separated)
`\defensedate{...}`	Defense date (default: `\today`)
`\thesislocation{...}`	City and country

```
\documentclass[doctype=thesis, language=english]{../../omnilatex}
\title{Numerical Methods for Turbulent Flow Simulation}
\author{Jane Doe}
\thesisinstitution{Technical University}
\thesisdepartment{Mechanical Engineering}
\thesisdegree{Master of Science}
\thesisadvisor{Prof.\ Dr.\ A.~Mueller}
\thesiscommittee{Prof.\ B.~Smith, Prof.\ C.~Lee}
\defensedate{2025-07-15}
\thesislocation{Hamburg, Germany}
\begin{document}
\maketitle
\frontmatter
\tableofcontents
\mainmatter
\chapter{Introduction}
\end{document}
```

8.2.2 dissertation

The `dissertation` doctype extends the thesis format with a dual supervisor model (primary and secondary), deeper sectioning (`secnumdepth=3`), and keyword support for repository metadata. The title page includes institution, department, degree, both supervisors, committee, submission date, and keywords.
Use `dissertation` for PhD or doctoral works that require dual supervisor tracking and keyword metadata.

```
\documentclass[doctype=dissertation]{../../omnilatex}
\title{Data-Driven Modelling of Urban Microclimates}
\author{Jane Doe}
\dissertationinstitution{Technical University}
\dissertationdepartment{Environmental Engineering}
\dissertationdegree{Doctor of Engineering}
\firstsupervisor{Prof.\ Dr.\ A.~Mueller}
\secondsupervisor{Prof.\ Dr.\ B.~Smith}
\dissertationcommittee{Dr.\ C.~Lee, Dr.\ D.~Park}
\submissiondate{2025-06-01}
\keywords{urban microclimate, data-driven modelling, CFD}
\begin{document}
\maketitle
\frontmatter
\chapter{Abstract}
```

Table 8.3: Custom commands for `dissertation`.

Command	Description
`\dissertationdegree{...}`	Degree (default: Doctoral Dissertation)
`\dissertationdepartment{...}`	Department or faculty
`\dissertationinstitution{...}`	University name
`\firstsupervisor{...}`	Primary supervisor
`\secondsupervisor{...}`	Secondary or co-supervisor
`\dissertationcommittee{...}`	Advisory committee members
`\submissiondate{...}`	Submission date (default: `\today`)
`\keywords{...}`	Comma-separated keywords

```
\tableofcontents
\mainmatter
\chapter{Introduction}
\end{document}
```

8.2.3 thesis-tuhh

The `thesis-tuhh` doctype combines the generic `thesis` profile with the `institution=tuhh` option for institution-specific branding at Hamburg University of Technology (TUHH). It is not a separate profile file; rather, it is activated by combining options:

```
\documentclass[
    doctype=thesis,
    institution=tuhh,
    titlestyle=TUHH,
    loadGlossaries,
    todonotes,
]{../../omnilatex}
```

Key features of the TUHH configuration include:

- **Title page branding** – The `titlestyle=TUHH` option selects a custom title page layout with the TUHH logo, bilingual headers, and the institutional colour scheme.
- **Bilingual logos** – Both German and English TUHH logos are available, selected automatically based on the `language` option.
- **TUHHuman TikZ picture** – A TikZ-drawn mascot (`TUHHuman`) is defined in the TUHH institution package and can be placed on the title page or in colophon sections.
- **Custom title page** – The TUHH institution package (`config/institutions/tuhh.sty`) overrides the default `\maketitle` with a layout that includes the TUHH wordmark, faculty name, and thesis metadata in the institutional style.

The complete TUHH thesis example is located at `examples/thesis-tuhh/` and demonstrates front matter (abstract, task description, authorship declaration), main matter chapters with code listings and figures, and back matter with appendices.
Use `thesis-tuhh` for any thesis produced at TUHH that requires official institutional branding on the title page.

```
\documentclass[
    language=english,
    institution=tuhh,
    titlestyle=TUHH,
    censoring=true,
    loadGlossaries,
    todonotes,
]{../../omnilatex}
\RequirePackage{config/document-settings}
\addbibresource{bib/bibliography.bib}
\begin{document}
\import{content/}{frontmatter}
\import{content/}{mainmatter}
\import{content/}{backmatter}
\end{document}
```

8.2.4 dictionary

The `dictionary` doctype produces a reference dictionary or glossary book. It uses a compact 10 pt font with `DIV=13` for a dense, reference-style layout. The title page includes series name, subtitle, edition, publisher, ISBN, and subject fields. The document is two-sided with shallow sectioning (`secnumdepth=1`, `tocdepth=1`).

Table 8.4: Custom commands for `dictionary`.

Command	Description
`\dictionaryseries{...}`	Series collection name
`\dictionarysubtitle{...}`	Subtitle or tagline
`\dictionaryedition{...}`	Edition (default: First Edition)
`\dictionarypublisher{...}`	Publisher name
`\dictionaryisbn{...}`	ISBN
`\dictionarysubjects{...}`	Subject categories
`\dictionarynote{...}`	Editorial note or disclaimer

Use `dictionary` for compiling terminology collections, technical lexicons, or domain-specific reference books.

```
\documentclass[doctype=dictionary]{../../omnilatex}
\title{Dictionary of Control Engineering}
\author{Jane Doe}
\dictionaryseries{OmniLexicon Collection}
\dictionarysubtitle{A quick reference for control systems terminology}
\dictionaryedition{Second Edition}
\dictionarypublisher{Acme Academic Press}
\dictionaryisbn{978-0-00-000002-4}
\dictionarysubjects{Control Theory, Automation}
\begin{document}
\maketitle
\chapter{A}
\section{Actuator}
A device that converts a control signal into mechanical motion.
\section{Adaptive Control}
A control strategy that adjusts parameters in real time.
\end{document}
```

8.2.5 manual (book-class)

The `manual` doctype uses `scrreprt` and is documented in Chapter **??**. When used as part of a book-style workflow with `\frontmatter` and `\backmatter`, it integrates naturally with the book-class ecosystem. See **??** for its custom commands.

9 Academic Documents

This chapter covers five doctypes designed for teaching and learning contexts: `homework`, `exam`, `lecture-notes`, `syllabus`, and `handout`. All use `scrartcl` with serif fonts, one-and-a-half line spacing, and half paragraph spacing.

9.1 Homework

The `homework` doctype produces a title page with course name, assignment number, student details, due date, and total points. It suppresses the TOC and sets `secnumdepth=0` for a clean, assignment-style layout.

```
\documentclass[doctype=homework]{../../omnilatex}
\title{Linear Algebra Problem Set}
\hwcourse{MATH 201}
\hwnumber{Homework 3}
\hwauthor{Prof.\ A.\ Smith}
\hwduedate{2025-11-15}
\hwinstructor{Prof.\ A.\ Smith}
\hwstudentname{Jane Doe}
\hwstudentid{12345678}
\hwtotalpoints{50}
\begin{document}
\maketitle
\begin{exercise}[10]
    Prove that every vector space has a basis.
\end{exercise}
\begin{solution}
    By Zorn's Lemma, \ldots
\end{solution}
\end{document}
```

9.1.1 Custom Commands

9.1.2 Environments

Two environments are provided for structuring problem sets:

- `exercise[points]` – Numbers exercises automatically and optionally displays a point value in parentheses.
- `solution` – Wraps the solution text. Content is only visible when solutions are enabled (see below).

9.1.3 Solution Control

A boolean flag controls whether solutions are rendered:

```
\showsolutions   % render all solution environments
\hidesolutions   % hide all solution environments (default)
```

Table 9.1: Homework metadata commands.

Command	Description
`\hwtitle{...}`	Assignment title (defaults to `\title`)
`\hwcourse{...}`	Course name (default: Course Name)
`\hwauthor{...}`	Author name (defaults to `\author`)
`\hwduedate{...}`	Due date (defaults to `\date`)
`\hwinstructor{...}`	Instructor name (default: Instructor Name)
`\hwnumber{...}`	Assignment label (default: Homework 1)
`\hwstudentname{...}`	Student name (default: empty)
`\hwstudentid{...}`	Student ID (default: empty)
`\hwtotalpoints{...}`	Total points (default: empty)

When `\hidesolutions` is active (the default), content inside `solution` environments is silently discarded via `\setbox`, producing a clean student-facing handout.

9.2 Examinations

The `exam` doctype formats formal examination papers with a title page showing exam code, date, duration, total marks, department, and custom instructions. It uses `pagestyle=empty` and suppresses section numbering.

```
\documentclass[doctype=exam]{../../omnilatex}
\title{Final Examination}
\examcode{MATH301-F2025}
\examdate{2025-12-18}
\examduration{3 hours}
\totalmarks{100}
\department{Department of Mathematics}
\begin{document}
\maketitle
\begin{question}[25]
    State and prove the Fundamental Theorem of Calculus.
\end{question}
\begin{answer}[4cm]
    % blank space for student response
\end{answer}
\end{document}
```

Two environments structure exam content: `question[marks]` auto-numbers questions and displays an optional mark allocation, while `answer[height]` creates an `addmargin` of configurable height for student responses.

9.3 Lecture Notes

The `lecture-notes` doctype is designed for textbook-style handouts with a wide left margin for student annotations. It sets `secnumdepth=2`, includes a header with course name and lecture number, and provides theorem-like environments.

Table 9.2: Examination commands.

Command	Description
`\examtitle{...}`	Exam title (defaults to `\title`)
`\examcode{...}`	Exam code (default: EXAM000)
`\examdate{...}`	Exam date (defaults to `\date`)
`\examduration{...}`	Duration (default: 2 hours)
`\totalmarks{...}`	Total marks (default: 100)
`\department{...}`	Department name (default: Department of Mathematics)
`\examinstructions{...}`	Custom instructions

9.3.1 Metadata Commands

Table 9.3: Lecture notes commands.

Command	Description
`\lecturetitle{...}`	Lecture title (defaults to `\title`)
`\lecturenumber{...}`	Lecture number (default: 1)
`\coursename{...}`	Course name (default: Course Name)
`\lecturedate{...}`	Date (defaults to `\date`)
`\lecturer{...}`	Lecturer name (default: Lecturer Name)
`\semester{...}`	Semester (default: Fall 2025)

9.3.2 Theorem-Like Environments

Numbered environments share a common counter: `theorem`, `lemma`, `corollary`, and `definition`. Unnumbered environments include `example`, `remark`, and `proof` (which appends a □).

9.3.3 Wide Left Margin

The doctype calls `\setCustomMargins{3cm}{2cm}{2.5cm}{2.5cm}`, creating a 3 cm left margin ideal for handwritten annotations during lectures.

9.4 Syllabus

The `syllabus` doctype generates a course syllabus with a detailed title page listing instructor contact information, meeting details, prerequisites, and course website.

9.4.1 Environments and Helpers

- `objectives` – Wraps a numbered list of learning objectives.

Table 9.4: Syllabus commands.

Command	Description
`\coursenumber{...}`	Course code (default: CS 000)
`\coursetitle{...}`	Course title (default: Course Title)
`\semester{...}`	Semester (default: Fall 2025)
`\instructor{...}`	Instructor name
`\instructoroffice{...}`	Office location
`\instructoremail{...}`	Email address
`\instructorhours{...}`	Office hours
`\meetingtime{...}`	Class meeting time
`\meetinglocation{...}`	Classroom location
`\prerequisites{...}`	Prerequisites (default: None)
`\coursewebsite{...}`	Course URL

- `gradingpolicy` – Produces a `booktabs` table with Component/Weight columns and an automatic Total row.
- `\gradingitem{component}{weight}` – Adds a row inside the `gradingpolicy` environment.
- `\scheduleentry{week}{topic}{notes}` – Adds a row for weekly schedule tables.

```
\begin{gradingpolicy}
    \gradingitem{Homework}{30}
    \gradingitem{Midterm Exam}{25}
    \gradingitem{Final Exam}{35}
    \gradingitem{Participation}{10}
\end{gradingpolicy}
```

9.5 Handout

The `handout` doctype produces compact, two-column course handouts with a running header showing course name, handout number, title, author, and date. It uses 15 mm margins and `parskip=half`.

Table 9.5: Handout commands.

Command	Description
`\handouttitle{...}`	Handout title (defaults to `\title`)
`\handoutcourse{...}`	Course name (default: Course Name)
`\handoutnumber{...}`	Handout number (default: 1)
`\handoutdate{...}`	Date (defaults to `\date`)
`\handoutauthor{...}`	Author (defaults to `\author`)

9.5.1 Highlight Boxes

Two tcolorbox-based commands draw attention to key material:

- `\keyconcept{title}{text}` – A titled box with a blue frame (`cBlue!70!black`) and light background (`Air!40`), rendered via the `keyconceptbox` tcolorbox.
- `\remember{text}` – An inline box with an orange frame (`mOrange!80!black`) and yellow background (`mYellow!20`), prefixed with “Remember:”.

9.5.2 Two-Column Mode

The doctype enables `twocolumn=true` via KOMA-Script, so all body content flows in two columns automatically. Use `\keyconcept` and `\remember` for single-column callouts that span the full column width.

10 Business & Personal Documents

This chapter covers five doctypes for professional and personal use: `cv`, `cover-letter`, `invoice`, `memo`, and `recipe`. These profiles demonstrate OmniLaTeX's versatility beyond academic writing.

10.1 CV / Resume

The `cv` doctype produces a single-page curriculum vitae with a compact layout: 10 pt sans-serif font, empty page style, and minimal item spacing (`documentitemspacing=compact`). It loads `fontawesome5`, `tikz`, and `adjustbox` for icons and optional circular profile photos.

```
\documentclass[doctype=cv]{../../omnilatex}
\title{Jane Doe}
\cvrole{Senior Software Engineer}
\cvcontact{jane@example.com}
\cvlocation{San Francisco, CA}
\cvphone{+1\,555\,012\,3456}
\cvlinks{\href{https://github.com/janedoe}{github.com/janedoe}
    \textbullet{}
    \href{https://janedoe.dev}{janedoe.dev}}
\cvsummary{Experienced engineer with 10+ years in distributed systems.}
\begin{document}
\maketitle
\section{Experience}
\begin{itemize}
    \item \textbf{Staff Engineer} -- Acme Corp, 2020--present
    \item \textbf{Senior Engineer} -- Startup Inc, 2016--2020
\end{itemize}
\section{Education}
\begin{itemize}
    \item \textbf{M.Sc.\ Computer Science} -- University, 2016
\end{itemize}
\end{document}
```

10.1.1 Custom Commands

The `\maketitle` command renders the header with name, role, location, phone, contact, and links. When `\cvimage` is set, a circular photo (2 cm diameter via TikZ clipping) appears to the left of the header block. Each `\section` is followed by a horizontal rule for visual separation.

10.1.2 cv-twopage Variant

The `examples/cv-twopage/` example demonstrates a two-page CV using `scrbook` instead of `scrartcl`. It uses the same `cv` doctype commands but allows a longer document with chapter structure. See `examples/cv-twopage/config/document-settings.sty` for the full configuration.

10.2 Cover Letter

The `cover-letter` doctype formats a formal cover letter with 11 pt serif font, one-and-a-half line spacing, and a plain page style. The `\maketitle` command is disabled; metadata commands are used to

Table 10.1: CV metadata commands.

Command	Description
`\cvrole{...}`	Job title or role
`\cvcontact{...}`	Contact line (email or phone)
`\cvlocation{...}`	City and region
`\cvphone{...}`	Phone number
`\cvlinks{...}`	Free-form links and social URLs
`\cvsummary{...}`	Professional summary (rendered below header rule)
`\cvimage{...}`	Path to a profile photo (rendered as circular crop)
`\cvbulletspacing{...}`	Set `\itemsep` for list items

populate the letter's address block manually.

```
\documentclass[doctype=cover-letter]{../../omnilatex}
\coverletterrecipient{Dr.\ A.\ Smith}
\coverletterposition{Senior Engineer}
\coverlettercompany{Acme Corp}
\coverletteraddress{123 Main St}
\coverlettercity{San Francisco, CA 94102}
\coverletterdate{2025-11-01}
\coverlettersendername{Jane Doe}
\coverlettersenderaddress{456 Oak Ave}
\coverlettersendercity{Berkeley, CA 94701}
\coverlettersenderphone{+1\,555\,012\,3456}
\coverlettersenderemail{jane@example.com}
\begin{document}
% Use the stored metadata to build the letter body
\end{document}
```

10.3 Invoice

The `invoice` doctype produces a professional invoice with a centered title, from/to address blocks, and a line-item table. It uses `DIV=10` for generous margins and `pagestyle=empty`.

```
\documentclass[doctype=invoice]{../../omnilatex}
\invnumber{INV-2025-0042}
\invdate{2025-10-01}
\invdue{2025-10-31}
\invfrom{Jane Doe Consulting}
\invto{Acme Corporation}
\invfromaddr{456 Oak Ave, Berkeley, CA 94701}
\invtoaddr{123 Main St, San Francisco, CA 94102}
\invcurrency{USD}
\invtaxrate{10}
\invpaymentterms{Net 30}
\begin{document}
\maketitle
```

Table 10.2: Cover letter commands.

Command	Description
`\coverletterrecipient{...}`	Recipient name
`\coverletterposition{...}`	Job position title
`\coverlettercompany{...}`	Company name
`\coverletteraddress{...}`	Recipient street address
`\coverlettercity{...}`	Recipient city and postcode
`\coverletterdate{...}`	Letter date (default: `\today`)
`\coverlettersendername{...}`	Sender name
`\coverlettersenderaddress{...}`	Sender street address
`\coverlettersendercity{...}`	Sender city and postcode
`\coverlettersenderphone{...}`	Sender phone
`\coverlettersenderemail{...}`	Sender email

```
\begin{tabular}{@{} p{6cm} r r r @{}}
    \toprule
    \textbf{Description} & \textbf{Qty} & \textbf{Price} & \textbf{Total} \\
    \midrule
    \invoiceitem{Backend Development}{40}{150}
    \invoiceitem{Code Review}{10}{200}
    \bottomrule
\end{tabular}
\end{document}
```

Table 10.3: Invoice commands.

Command	Description
`\invnumber{...}`	Invoice number (default: INV-0001)
`\invdate{...}`	Issue date
`\invdue{...}`	Due date
`\invfrom{...}`	Sender company name
`\invto{...}`	Client name
`\invfromaddr{...}`	Sender address
`\invtoaddr{...}`	Client address
`\invcurrency{...}`	Currency code (default: USD)
`\invtaxrate{...}`	Tax rate percentage (default: 0)
`\invpaymentterms{...}`	Payment terms (default: Net 30)
`\invnotes{...}`	Additional notes

The `\invoiceitem{desc}{qty}{price}` command emits a table row with description, quantity, unit price, and computed total, followed by a horizontal rule.

10.4 Memo

The `memo` doctype formats a business memorandum with 12 pt serif font and single line spacing. The `\maketitle` command renders a MEMORANDUM header with ruled lines and labelled fields.

```
\documentclass[doctype=memo]{../../omnilatex}
\memoto{Engineering Team}
\memofrom{Jane Doe, CTO}
\memosubject{Q4 Development Roadmap}
\memodate{2025-10-15}
\memocc{CFO Office}
\memoref{MEMO-2025-042}
\begin{document}
\maketitle
This memo outlines the development priorities for Q4\ldots
\end{document}
```

Table 10.4: Memo commands.

Command	Description
`\memoto{...}`	Recipient
`\memofrom{...}`	Sender
`\memosubject{...}`	Subject line
`\memodate{...}`	Date (default: `\today`)
`\memocc{...}`	CC list (omitted when empty)
`\memobcc{...}`	BCC list (omitted when empty)
`\memoref{...}`	Reference number (omitted when empty)

The header renders as **TO**, **FROM**, **DATE**, and **RE** fields with a 1.5 cm label width. `REF` and `CC` fields are only displayed when non-empty.

10.5 Recipe

The `recipe` doctype formats cooking recipes with metadata, an ingredients list, step-by-step instructions, and optional chef's notes. It uses `parskip=half` and single line spacing.

```
\documentclass[doctype=recipe]{../../omnilatex}
\title{Classic Margherita Pizza}
\author{Jane Doe}
\recipeMeta{prep}{30 minutes}
\recipeMeta{cook}{12 minutes}
\recipeMeta{servings}{4}
\recipeMeta{difficulty}{Easy}
\recipeMeta{cuisine}{Italian}
\recipeMeta{category}{Main Course}
\recipeMeta{calories}{280}
\begin{document}
\maketitle
\begin{ingredients}
    \item 500g bread flour
    \item 7g active dry yeast
```

```
    \item 200ml warm water
\end{ingredients}
\begin{instructions}
    \item Combine flour, yeast, and salt in a large bowl.
    \item Add warm water and mix until a dough forms.
\end{instructions}
\recipeNote{For a crispier crust, preheat the oven to 250\textdegree C.}
\end{document}
```

10.5.1 Metadata Commands

Table 10.5: Recipe metadata commands.

Command	Description
`\recipeprep{...}`	Preparation time (default: –)
`\recipecook{...}`	Cooking time (default: –)
`\recipeservings{...}`	Number of servings (default: –)
`\recipedifficulty{...}`	Difficulty level (default: –)
`\recipecuisine{...}`	Cuisine type (default: –)
`\recipecategory{...}`	Meal category (default: –)
`\recipecalories{...}`	Calories per serving (default: –)

All metadata fields can also be set via the helper `\recipeMeta{key}{value}`, which expands to `\recipe`*`key`*`{value}`. The metadata is rendered in a tcolorbox with a blue frame below the title.

10.5.2 Environments and Helpers

- `ingredients` – A bulleted list with compact spacing (`leftmargin=1.5em`, `itemsep=2pt`).
- `instructions` – A numbered list with slightly wider spacing (`leftmargin=2em`, `itemsep=4pt`).
- `\recipeNote{...}` – A tcolorbox with an orange frame, titled “Chef’s Note”, for tips and variations.

11 Presentations & Posters

This chapter covers two doctypes for visual communication: `presentation` and `poster`. Both use sans-serif fonts and full colour mode, but differ significantly in page size and layout strategy.

11.1 Presentations

The `presentation` doctype creates slide decks using KOMA-Script rather than `beamer`. It loads `lib/layout/omnilatex-presentation.sty`, which provides frame environments, block environments, progress bars, and section dividers built with TikZ and tcolorbox.

11.1.1 Page Dimensions

The doctype sets a 16:10 aspect ratio with `paperwidth=254mm` and `paperheight=190.5mm`, applied at `\AtBeginDocument` via `\recalctypearea`. The font size is 14 pt with single line spacing.

11.1.2 Frame Environments

Two frame environments are available:

- `presentationframe` – The primary frame. Each instance triggers a page break, increments the frame counter, renders the header and footer, and draws the progress bar before the next page break.
- `slideframe` – A backward-compatible alias that wraps `presentationframe` with identical behaviour.

```
\documentclass[doctype=presentation]{../../omnilatex}
\title{Introduction to OmniLaTeX}
\author{Jane Doe}
\date{2025-11-01}
\begin{document}
\maketitle
\begin{presentationframe}
    \slidetitle{Overview}
    OmniLaTeX is a universal document template.
\end{presentationframe}
\end{document}
```

11.1.3 Section Dividers

The `\presentationSection{title}` command creates a full-width coloured section divider. It fills the upper half of the page with the university primary colour, centres the section title in large white bold text, and includes the frame counter and progress bar.

11.1.4 Slide Titles

The `\slidetitle{...}` command renders a centred, coloured title with a decorative rule underneath:

```
\slidetitle{Key Results}
The experiment confirmed the hypothesis with $p < 0.01$.
```

The title is set in `\LARGE\bfseries` in the university primary colour, followed by a 0.6\textwidth rule at 40% opacity.

11.1.5 Multi-Column Layouts

Two-column slides use the `presentationcolumns` environment with `\presentationcolumn{width}` commands:

```
\begin{presentationframe}
    \slidetitle{Comparison}
    \begin{presentationcolumns}
        \presentationcolumn{0.48}
            Left column content.
        \end{column}
        \presentationcolumn{0.48}
            Right column content.
    \end{column}
    \end{presentationcolumns}
\end{presentationframe}
```

Note that `presentationcolumn` opens a `column` environment but does not close it; you must end each column with `\end{column}` explicitly.

11.1.6 Block Environments

Five tcolorbox-based block types are provided, each with a coloured left border and optional title:

Table 11.1: Presentation block environments.

Environment	Border colour	Background
`presentationblock`	University primary	White
`alertblock`	Red	Red!5
`exampleblock`	Green	Green!5
`noteblock`	Blue	Blue!5
`definitionblock`	Orange	Orange!5

All blocks accept an optional first argument for additional tcolorbox options and a mandatory second argument for the title:

```
\begin{alertblock}{Warning}
    This approach is deprecated.
\end{alertblock}
\begin{definitionblock}{Definition}
    A \emph{group} is a set with an associative binary operation.
\end{definitionblock}
```

11.1.7 Progress Bar

A thin progress bar is drawn at the bottom of each slide using TikZ. It fills proportionally based on the ratio of the current frame counter to the total frame count. The total is computed by calling `\presentationcountframes` after all frames are defined.

11.1.8 Navigation and Progress Toggles

Table 11.2: Presentation toggle commands.

Command	Default	Effect
`\presentationnavsymbolson`	Off	Show navigation symbols
`\presentationnavsymbolsoff`	Off	Hide navigation symbols
`\presentationprogresson`	On	Show progress bar
`\presentationprogressoff`	On	Hide progress bar

Navigation symbols are off by default for clean slides. The progress bar is on by default.

11.1.9 Frame Counter

Two counters, `framecounter` and `totalframes`, track slide progression. The footer displays "n / N" in the bottom-right corner. Call `\presentationcountframes` at the end of the document body to set the total.

11.2 Posters

The `poster` doctype creates A1 landscape conference posters (841 mm × 594 mm) with a 20 pt sans-serif font, `DIV=20` layout, and `paper=a1` base size. Content is arranged using the `multicol` package.

11.2.1 Poster Blocks

Two tcolorbox environments provide titled and untitled content containers:

- `posterblock[title]` – A titled block with white background, dark frame (`black!70`), 1.5 pt box rule, 3 mm rounded corners, and 8 mm inner padding. The title is rendered in `\bfseries\Large` with white text on the frame.
- `posterblocknotitle` – The same styling without a title bar, useful for embedding figures or TikZ diagrams inside a titled block.

11.2.2 Layout Helpers

- `\posterwidth` – A length set to `0.9\textwidth` for consistent block widths.
- `\columnbreak` – Standard `multicol` column break for multi-column poster layouts.

11.2.3 Example: OmniLaTeX Poster

The example at `examples/poster/main.tex` demonstrates a three-column poster with TikZ graphics:

```
\documentclass[doctype=poster, institution=none]{../../omnilatex}
\title{OmniLaTeX: A Universal Document Template}
\author{Jane Doe, John Smith}
\date{International Conference on Document Engineering, 2026}
\begin{document}
\maketitle
\begin{multicols}{3}
```

```
    \begin{posterblock}[Introduction]
        OmniLaTeX is a modular, universal \LaTeX{} template\ldots
    \end{posterblock}
    \begin{posterblock}[Results]
        \begin{posterblocknotitle}
            \centering
            % pgfplots accuracy chart
            \begin{tikzpicture}
                \begin{axis}[xlabel={Epoch}, ylabel={Accuracy (\%)}]
                    \addplot[blue, thick, domain=1:50, samples=30]
                        {95 - 60*exp(-0.08*x)};
                \end{axis}
            \end{tikzpicture}
        \end{posterblocknotitle}
    \end{posterblock}
    \columnbreak
    \begin{posterblock}[Architecture Overview]
        \begin{posterblocknotitle}
            \centering
            % TikZ system architecture diagram
            \begin{tikzpicture}[
                box/.style={draw, thick, rounded corners,
                    minimum height=8mm, fill=blue!10},
                arr/.style={->, thick, >=stealth},
            ]
                \node[box] (user) {User};
                \node[box, below=of user] (api) {API Layer};
                \node[box, below left=of api] (db) {Database};
                \node[box, below right=of api] (cache) {Cache};
                \draw[arr] (user) -- (api);
                \draw[arr] (api) -- (db);
                \draw[arr] (api) -- (cache);
            \end{tikzpicture}
        \end{posterblocknotitle}
    \end{posterblock}
\end{multicols}
\end{document}
```

The title is rendered in 60 pt bold at the top of the poster, followed by the author list in `\LARGE` and the date in `\Large`. No headers or footers are drawn (`pagestyle=empty`).

Part III

Typography & Content

12 Fonts

OmniLaTeX provides a comprehensive font system built on LuaLaTeX's native OpenType support. All fonts are configured through a layered architecture that separates font selection, feature configuration, and fallback logic.

12.1 Font System Architecture

OmniLaTeX targets LuaHBTeX 1.21.0+, which provides native OpenType and Unicode support without the shims required by XeLaTeX. The font stack is composed of three core packages:

`fontspec` Provides `\setmainfont`, `\setsansfont`, `\setmonofont`, and raw OpenType feature control.

`unicode-math` Replaces the legacy `amsmath` math font interface with OpenType math fonts. Loads `\symup`, `\symbf`, `\symit`, and the full SYMB alphabet.

`lualatex-math` Fixes a handful of LuaTeX math-mode bugs (e.g. `\sqrt` spacing, `\overline` thickness) that affect LibreOffice and other engines.

Font definitions live in the `omnilatex-fonts` module and are loaded before the user preamble, so every document inherits the defaults automatically.

12.2 Default Font Stack

Table 12.1: Default font families shipped with OmniLaTeX..

Family	Default Font	Module
Roman (serif)	Libertinus Serif	`omnilatex-fonts`
Sans-serif	Libertinus Sans	`omnilatex-fonts`
Math	Libertinus Math	`omnilatex-fonts`
Monospaced	Monaspace Neon	`omnilatex-fonts`
Accessibility	Atkinson Hyperlegible Next	`omnilatex-fonts`

Libertinus is the spiritual successor to Linux Libertine and provides a complete serif, sans, and math family with consistent metrics. Monaspace Neon offers five weight-variant glyphs in a single monospaced face, making it ideal for code listings. Atkinson Hyperlegible Next is loaded as an accessibility fallback for readers with low vision.

12.3 Setting Fonts

OmniLaTeX exposes four high-level commands that wrap the underlying `fontspec` API:
Minimal example that switches to Source Serif 4 and Source Code Pro:

Table 12.2: Font-setting commands..

Command	Underlying call
`\setMainFont{...}`	`\setmainfont`
`\setSansFont{...}`	`\setsansfont`
`\setMonoFont[...]{...}`	`\setmonofont`
`\setMathFont{...}`	`\setmathfont`

```
\usepackage{omnilatex-fonts}
\setMainFont{Source Serif 4}
\setSansFont{Source Sans 3}
\setMonoFont[Scale=0.88]{Source Code Pro}
\setMathFont{STIX Two Math}
```

12.4 Font Feature Tables

OpenType fonts expose named features (e.g. `liga`, `kern`, `onum`, `ss01`) that control ligatures, kerning, glyph substitution, and stylistic alternates. OmniLaTeX enables sensible defaults for every family:

Table 12.3: Default OpenType features by font family..

Feature	Roman	Sans	Mono	Math
`kern`	on	on	on	–
`liga`	on	on	on	–
`onum`	on	on	on	–
`pnum`	on	on	on	on
`ss01`	on	off	off	–

To query available features for any installed font, run:

```
lualatex --luaonly -e "
  local f = require'fontloader.font'
  -- inspect font's 'features' table
"
```

Or use the `otfinfo` command-line tool bundled with TEX Live:

```
otfinfo --features LibertinusSerif-Regular.otf
```

To override features per family, pass the `Features` key:

```
\setMainFont{Libertinus Serif}[
  RawFeature={+ss01,+salt=3},
  Ligatures=TeX,
]
```

12.5 Icon Fonts

OmniLaTeX loads `fontawesome5`, which provides over 1 500 scalable icons as font glyphs. The primary interface is `\faIcon{name}`:

```
\faIcon{graduation-cap}  \faIcon{github}  \faIcon{python}
```

Table 12.4: Commonly used Font Awesome icons..

Command	Icon	Use case
`\faIcon{graduation-cap}`		Education
`\faIcon{github}`		Version control
`\faIcon{python}`		Programming
`\faIcon{envelope}`		Contact
`\faIcon{home}`		Navigation
`\faIcon{book}`		References
`\faIcon{exclamation-triangle}`		Alerts
`\faIcon{lightbulb}`		Tips

Icons inherit the surrounding text colour and scale, so they integrate seamlessly into headings, footers, and inline text.

12.6 CJK Fonts

Chinese, Japanese, and Korean (CJK) typesetting is handled by `luatexja-fontspec`, a bridge between `luatexja` and `fontspec`. OmniLaTeX provides three convenience commands:

```
\setCJKMainFont{Noto Serif CJK SC}
\setCJKSansFont{Noto Sans CJK SC}
\setCJKMonoFont{Noto Sans Mono CJK SC}
```

The `Noto CJK` superfamily covers four regional variants:

Table 12.5: Noto CJK regional variants..

Tag	Script / Region	Example font
SC	Simplified Chinese	Noto Serif CJK SC
TC	Traditional Chinese	Noto Serif CJK TC
JP	Japanese	Noto Serif CJK JP
KR	Korean	Noto Serif CJK KR

See Chapter **??** for full CJK typesetting details including punctuation handling, ruby annotations, and inter-paragraph spacing.

12.7 RTL Fonts

Right-to-left (RTL) scripts such as Arabic, Hebrew, and Persian require bidirectional text shaping. OmniLaTeX uses `bidi` (LuaLaTeX) with font fallback chains:

```
\setArabicFont{Noto Naskh Arabic}[Script=Arabic]
\setHebrewFont{Noto Serif Hebrew}[Script=Hebrew]
```

The `bidi` package automatically mirrors punctuation and adjusts mixed LTR/RTL paragraph direction. Fallback chains ensure that glyphs missing from the primary font are drawn from a secondary source. See Chapter **??** for advanced RTL configuration including mixed-script documents and table direction.

12.8 Unit Number Font

Scientific documents that use `siunitx` benefit from a dedicated number font that guarantees consistent digit width and alignment. OmniLaTeX defines `\unitnumberfont` for this purpose:

```
\sisetup{
  number-font = \unitnumberfont,
}
```

This selects the monospaced face (Monaspace Neon by default) so that tabular number columns align on the decimal point without requiring `S` column type tricks. The command can be redefined to use any font with tabular figures:

```
\renewcommand{\unitnumberfont}{\fontspec{Fira Code}}
```

13 Text Formatting

OmniLaTeX ships sensible defaults for text formatting so that most documents look polished out of the box. This chapter covers the micro-typographic engine, dash handling, quotation marks, and spacing controls.

13.1 Microtype

The `microtype` package improves typographic quality through three mechanisms that are all enabled by default in OmniLaTeX:

Protrusion Characters such as commas, periods, hyphens, and quotation marks are allowed to extend slightly into the right margin, eliminating visible raggedness caused by punctuation at line ends.

Expansion Inter-character spacing is microscopically adjusted (typically $\pm 1 - 2$ emu) to achieve even line lengths without hyphenation.

Tracking Global letter-spacing is modified slightly at line beginnings and endings to balance paragraph shape.

No configuration is required. To disable microtype globally (not recommended), pass `microtype=false` to the document class.

13.2 Extended Dashes

The `extdash` package provides shorthand macros that produce correctly kerned en-dashes and em-dashes without relying on TEX's ligature-based `--` / `---` mechanism. This is especially useful in documents that use hyphens inside compound words alongside ranges.

Table 13.1: extdash shortcuts..

Input	Output	Meaning
`fast\-/paced`	fast-paced	Em-dash joining hyphenated words
`1\--10`	1–10	En-dash range
`\---`	–	Em-dash
`\-`	-	Discretionary hyphen (no break)

Key advantage: `fast-paced` produces *fast—paced* (an em-dash) whereas the ligature approach `fast--paced` would incorrectly typeset as fast–paced.

13.3 Quotations

The `csquotes` package provides language-aware quotation marks via `\enquote{text}`. OmniLaTeX loads `csquotes` by default and integrates it with the bibliography:

```
\enquote{Hello world}
% → "Hello world"  (English)
% → ‚Hello world'  (British)
% → »Hello world«  (German)
```

Nested quotes are handled automatically:

```
\enquote{She said, \enquote{Hello world}.}
% → "She said, 'Hello world'."
```

To use `\enquote` with `biblatex`, set:

```
\SetCiteCommand{\parencite}
```

This ensures that `\textcite` and `\parencite` produce properly quoted author names in the running text.

13.4 Ragged Typesetting

The `ragged2e` package improves upon plain `\raggedright` by still hyphenating words when necessary, producing a cleaner right margin than LaTeX's built-in ragged mode. OmniLaTeX loads `ragged2e` and exposes two commands:

Table 13.2: Ragged typesetting commands..

Command	Behaviour
`\RaggedRight`	Left-aligns with hyphenation; ideal for narrow columns, posters, and slide content.
`\RaggedLeft`	Right-aligns with hyphenation; useful for captions or marginal notes.

Use `\RaggedRight` instead of `\raggedright` inside `tabular`, `minipage`, and `tcolorbox` environments to avoid overly loose inter-word spacing.

13.5 Blind Text

The `blindtext` package generates filler paragraphs and complete dummy documents for layout testing:

```
\Blindtext[3][1]      % 3 paragraphs, 1-level sectioning
\Blinddocument        % Full document with chapters and sections
```

OmniLaTeX sets the blind-text language to match `\mainlang`, so `\Blindtext` produces English filler text by default and will switch to German, French, etc. when the document language is changed.

13.6 Line Spacing

OmniLaTeX uses `setspaceenhanced` for fine-grained line-spacing control. The default baseline skip factor is:

```
\linespread{1.05}
```

This produces slightly tighter lines than LaTeX's default 1.2 factor while remaining readable for body text. For wider spacing:

```
\onehalfspacing     % 1.5× line spacing
\doublespacing      % 2.0× line spacing
```

All spacing commands affect the current scope, so they can be used inside groups to localise changes:

```
{\onehalfspacing
  This paragraph has wider spacing.
}
```

13.7 Paragraph Spacing

KOMA-Script provides the `parskip` option to control vertical spacing between paragraphs instead of TeX's default indentation. OmniLaTeX configures this via `\documentparspacing`:

```
\documentparspacing{half}   % half-line skip, no indent (default for manual)
\documentparspacing{full}   % full-line skip, no indent
\documentparspacing{indent} % traditional indent, no skip
```

The `half` setting is the default for the `manual` document type and produces comfortable visual separation between paragraphs without wasting vertical space. The `indent` setting is used for the `thesis` and `article` types.

14 Mathematics

OmniLaTeX combines `amsmath` and `unicode-math` to provide a modern math typesetting system with OpenType math fonts, auto-scaling delimiters, and convenience macros for statistics, physics, and engineering notation.

14.1 Equation Environments

All standard `amsmath` environments are available. The most commonly used are summarised below with examples.

Single equation:

```
\begin{equation}
  E = mc^2
  \label{eq:einstein}
\end{equation}
```

Aligned multi-line:

```
\begin{align}
  \nabla \cdot  \mathbf{E} &= \frac{\rho}{\varepsilon_0}  \label{eq:gauss} \\
  \nabla \times \mathbf{B} &= \mu_0 \mathbf{J}
    + \mu_0 \varepsilon_0 \frac{\partial \mathbf{E}}{\partial t} \label{eq:ampere}
\end{align}
```

Gathered (no alignment):

```
\begin{gather}
  a^2 + b^2 = c^2 \\
  e^{i\pi} + 1 = 0
\end{gather}
```

Multi-line with single number:

```
\begin{multline}
  \int_{-\infty}^{\infty} e^{-x^2} \, dx
    = \sqrt{\pi} \\
  = \sum_{n=0}^{\infty} \frac{(-1)^n}{n!}
    \int_{-\infty}^{\infty} x^{2n} e^{-x^2} \, dx
\end{multline}
```

Split inside equation:

```
\begin{equation}
  \begin{split}
    f(x) &= a_0 + a_1 x + a_2 x^2 \\
         &= \sum_{k=0}^{2} a_k x^k
  \end{split}
\end{equation}
```

14.2 Auto-Scaling Delimiters

Manually writing `\left(` and `\right)` for every expression is tedious. The `omnilatex-math` module provides three convenience macros that automatically size delimiters based on their content:

Table 14.1: Auto-scaling delimiter commands..

Command	Delimiters	Example input
`\parens{x+y}`	( )	`\parens{\frac{a}{b}}`
`\brackets{x}`	[ ]	`\brackets{\Sigma}`
`\braces{x}`	{ }	`\braces{\Omega}`

These commands expand to `\left` / `\right` pairs internally, so they handle nested fractions, summations, and matrices without manual height specification:

```
\parens{\frac{1}{1 + \frac{1}{n}}}
% → ( 1 / (1 + 1/n) )  with correctly sized parentheses
```

14.3 Equation Punctuation

Placing punctuation after display equations is a common source of spacing errors. OmniLaTeX defines three macros that handle proper spacing between the equation and the following punctuation:

Table 14.2: Equation punctuation commands..

Command	Produces	Usage
`\eqend`	.	Period after a concluding equation
`\eqcomma`	,	Comma before a clause continues
`\coloneq`	≔	Colon for a definition equation

Example:

```
\begin{equation}
  \mathcal{L}\{f(t)\} = \int_0^\infty e^{-st} f(t) \, dt \eqend
\end{equation}
The Laplace transform converts a time-domain signal\ldots
```

14.4 Statistical Macros

The `omnilatex-math` module provides shorthand macros for common statistical notation:
These macros use `\DeclarePairedDelimiter` internally, so they accept optional size arguments: `\mean*[\Big]{x}` produces a larger bar.

14.5 Number Formatting

Circled numbers are useful for step-by-step procedures and enumerated equations:

Table 14.3: Statistical convenience macros..

Command	Renders as	Meaning
`\mean{x}`	$\bar{x}$	Arithmetic mean (bar)
`\logmean{x}`	$\tilde{x}$	Logarithmic mean
`\abs{x}`	$\lvert x \rvert$	Absolute value
`\norm{x}`	$\lVert x \rVert$	Norm (double bars)
`\inner{a,b}`	$\langle a, b \rangle$	Inner product (angle brackets)
`\variance{x}`	$\mathrm{Var}[x]$	Variance (σ^2)

```
\circlednum{1}  \circlednum{2}  \circlednum{3}
% → ① ② ③
```

Symbol placeholders for figures and tables under development:

```
\symbolplaceholder{figure}
% → [FIGURE]
```

Custom operators can be declared with `\DeclareMathOperator`:

```
\DeclareMathOperator{\Tr}{Tr}
\DeclareMathOperator*{\argmin}{arg\,min}

$\Tr(A) = \sum_i \lambda_i$
$\argmin_{x} f(x)$
```

14.6 Physical Constants

OmniLaTeX defines shortcuts for frequently used constants and symbols in engineering and physics:

Table 14.4: Physical constant macros..

Command	Renders as	Description
`\grad`	grad	Gradient operator (∇)
`\const`	const.	Denotes a constant quantity

Quoted text inside math mode is provided by `\qq{text}`:

```
$T_{\qq{eff}} = 5778\,\mathrm{K}$
% → T_eff with "eff" in upright text
```

This avoids the common pattern of switching to `\mathrm` or `\textnormal` inline and keeps the source readable.

14.7 Display Equations

The `empheq` package allows annotated display equations with side-by-side text or boxed emphasis. OmniLaTeX preconfigures `empheq` to work with the Libertinus Math font metrics:
Boxed equation:

```
\begin{empheq}[box=\fbox]{equation}
  i\hbar \frac{\partial}{\partial t} \Psi
    = \hat{H} \Psi
\end{empheq}
```

Equation with right-side annotation:

```
\begin{empheq}[right=\eqref{eq:schrodinger}\;\text{(Schr\"odinger)}]{align}
  i\hbar \frac{\partial}{\partial t} \Psi
    &= \hat{H} \Psi \label{eq:schrodinger}
\end{empheq}
```

Aligned equations with shared numbering:

```
\begin{align}
  \frac{\partial u}{\partial t}
    &= \alpha \nabla^2 u \label{eq:heat} \\
  \frac{\partial^2 u}{\partial t^2}
    &= c^2 \nabla^2 u   \label{eq:wave}
\end{align}
```

Cross-referencing equations works exactly like LaTeX figures and tables: `Equation \ref{eq:heat}` produces *Equation 1*.

15 Physical Mathematics

OmniLaTeX provides a comprehensive physical typesetting system built on `siunitx` v3 for units, `chemmacros` for chemistry, and `xfrac`/`nicefrac` for fractions. All configuration is handled by `omnilatex-math.sty`, so you can use these commands directly without additional setup.

15.1 siunitx Setup

`siunitx` is preconfigured in `omnilatex-math.sty` with locale-aware settings. The base configuration applies to all languages, and per-language overrides are activated automatically based on the document language:

```
\sisetup{%
    mode = match,
    per-mode = symbol,
    range-units = single,
    uncertainty-mode = separate,
    propagate-math-font = true,
    ...
}
```

When the document language is German, the locale is set to `DE` (uses a comma as decimal separator). For English documents the locale defaults to `US` (period as decimal separator). This switching happens via `\gappto` and requires no manual intervention.

15.2 Core Commands

The three primary `siunitx` commands cover most use cases:

Table 15.1: Core `siunitx` commands..

Command	Description
`\num{value}`	Formats a plain number respecting locale (e.g. `1,000.5` in US, `1.000,5` in DE).
`\unit{...}`	Typesets a unit with correct spacing and symbols, e.g. `\unit{\metre\per\second}` → m/s.
`\qty{value}{unit}`	Combines number and unit in one call, e.g. `\qty{5.2}{\kilo\gram}` → 5.2 kg.

Additional useful commands include `\qtylist` for comma-separated quantities (e.g. `\qtylist{1;2;3}{\metre}`) and `\numlist` for plain number lists.

15.3 Quantity Ranges

Ranges are typeset with `\qtyrange`, which by default places the unit at the end once (`range-units=single`):

```
\qtyrange{20}{30}{\celsius}
% → 20 °C to 30 °C  (en-dash separator)
```

The range phrase can be customised:

```
\qtyrange[range-phrase={/}]{20}{30}{\celsius}
% → 20/30 °C
```

OmniLaTeX's `\temperaturepair` macro (see Section ??) is a specialised wrapper that uses the `/` separator for temperature ranges.

15.4 Custom SI Units

OmniLaTeX declares several additional units beyond the `siunitx` defaults. These are defined in `omnilatex-math.sty` and are available immediately:

Table 15.2: Custom SI units defined by OmniLaTeX..

Command	Unit	Description
`\volpercent`	vol%	Volume percent
`\watthour`	Wh	Watt-hour
`\annum`	a	Annum (year)
`\atmosphere`	atm	Standard atmosphere
`\partspermillion`	ppm	Parts per million
`\bar`	bar	Bar (pressure)
`\relhumidity`	%rh	Relative humidity (language-dependent)

The relative humidity unit adapts to the document language: it renders as `%r.F.` in German and `%RH` in English.
The Euro sign is also available as a unit via the `eurosym` package–use `\euro` directly inside `\unit{}` or `\qty{}{}`.

15.5 Custom SI Qualifiers

Unit qualifiers appear as subscripts and are used to disambiguate quantities that share the same unit. OmniLaTeX integrates qualifiers with the glossary system so that subscript labels remain consistent throughout the document:
Usage example:

```
\unit{\kilogram\per\cubic\metre\dryair}
% → kg/m³_dry air
```

The qualifier text is sourced from the glossary entries `sub.dry~air`, `sub.water`, and `sub.thermal`, ensuring consistent subscript formatting.

Table 15.3: Custom SI qualifiers defined by OmniLaTeX..

Command	Qualifier	Description
`\dryair`	dry air	Dry air component
`\water`	water	Water component
`\thermal`	thermal	Thermal property

15.6 Chemistry

Chemical typesetting is provided by the `chemmacros` package, configured to use the Libertinus font family for formulae.
Compounds are typeset with `\chcpd`:

```
\chcpd{H2O}    % → H₂O (compound style)
\ch{H2O}       % → H₂O (formula style)
\ch{CO2 + H2O -> H2CO3}
```

Numbered reactions use the `reaction` environment:

```
\begin{reaction}
  2 H2 + O2 -> 2 H2O
\end{reaction}
% → R [1]:  2 H₂ + O₂ → 2 H₂O
```

Reactions are tagged with a bold **R** prefix and square brackets, sharing the equation counter. In chaptered documents the counter is prefixed with the chapter number.
Chemical amounts combine quantity, unit, and compound:

```
\chemamount{1.2}{\partspermillion}{CO2}
% → 1.2 ppm CO₂
```

This is a convenience macro defined in `omnilatex-math.sty` that ensures correct spacing between the numerical value, unit, and compound name.
Multiple reactions can be grouped:

```
\begin{reactions}
  A -> B & C -> D \\
  B + D -> E
\end{reactions}
```

A gathered variant without alignment is also available: `reactionsgather`.

15.7 Fractions

Two packages provide alternative fraction styles:
xfrac – stacked (slanted) fractions for inline use:

```
\sfrac{1}{2}  % → ¹⁄₂  (diagonal fraction)
```

nicefrac – inline fractions with a slash:

```
\nicefrac{1}{3}  % → 1/3  (compact inline fraction)
```

Both packages are loaded by `omnilatex-math.sty`. Use `\sfrac` when you want a true diagonal fraction glyph, and `\nicefrac` when a simple slash is preferable for very compact inline settings.

16 Derivatives & Vector Calculus

OmniLaTeX defines a complete set of derivative, vector, and physical-flow macros in `omnilatex-math.sty`. These replace the functionality of the `physics` package (which has known maintenance issues) with well-tested, self-contained implementations.

16.1 Ordinary Derivatives

All derivative macros use an upright d for total derivatives and ∂ for partial derivatives. The upright `d` follows ISO 80000-2 conventions.

Table 16.1: Derivative macros – operator form..

LaTeX Source	Renders as	Syntax
`\deriv{f}`	$\mathrm{d}f$	First derivative of `f`
`\deriv[2]{f}`	$\mathrm{d}^2 f$	n-th derivative (optional degree)
`\deriv*{f}`	∂f	Partial derivative of `f`
`\deriv*[2]{f}`	$\partial^2 f$	n-th partial derivative

Table 16.2: Derivative macros – fraction form..

LaTeX Source	Renders as	Syntax
`\fracderiv{f}{x}`	$\frac{\mathrm{d}f}{\mathrm{d}x}$	$\frac{\mathrm{d}f}{\mathrm{d}x}$
`\fracderiv[2]{f}{x}`	$\frac{\mathrm{d}^2 f}{\mathrm{d}x^2}$	n-th total derivative
`\fracderiv*{f}{x}`	$\frac{\partial f}{\partial x}$	$\frac{\partial f}{\partial x}$
`\fracderiv*[2]{f}{x}`	$\frac{\partial^2 f}{\partial x^2}$	n-th partial derivative
`\timederiv{f}`	$\frac{\mathrm{d}f}{\mathrm{d}t}$	Derivative w.r.t. time
`\timederiv[2]{f}`	$\frac{\mathrm{d}^2 f}{\mathrm{d}t^2}$	n-th time derivative
`\timederiv*{f}`	$\frac{\partial f}{\partial t}$	Partial time derivative
`\posderiv{f}`	$\frac{\mathrm{d}f}{\mathrm{d}x}$	Derivative w.r.t. position
`\posderiv[2]{f}`	$\frac{\mathrm{d}^2 f}{\mathrm{d}x^2}$	n-th positional derivative
`\posderiv*{f}`	$\frac{\partial f}{\partial x}$	Partial positional derivative

The time and position symbols in `\timederiv` and `\posderiv` are sourced from the glossary entries `sym.time` and `sym.first_cart_coord`, ensuring consistent notation across the document.

16.2 Vector Macros

Table 16.3: Vector macros..

Command	Renders as	Description
`\vect{v}`	$\mathbf{v}$	Bold italic vector ($\mathbf{v}$)

The `\vect` command uses `\symbf` from `unicode-math` to produce a bold italic symbol, which is the convention in physics and engineering for vectors.

16.3 Physical Flow Macros

The following macros are defined for common engineering notations:

Table 16.4: Physical flow macros..

Command	Renders as	Description
`\flow{\dot{m}}`	$\dot{m}$	Mass flow rate (dotted symbol)
`\difference{\Delta T}`	$\Delta\Delta T$	Temperature / general difference
`\heatexentry{T}`	??	Heat exchanger entry (T')
`\heatexexit{T}`	??	Heat exchanger exit (T'')

`\flow` places a dot above its argument for mass flow rate notation. `\heatexentry` and `\heatexexit` append one or two prime marks, respectively, following the common thermodynamic convention for heat exchanger inlet/outlet states.

16.4 Thermodynamic Notation

The `\temperaturepair` macro formats temperature ranges with a forward-slash separator:

```
\temperaturepair{T_min}{T_max}
% → T_min/T_max °C  (e.g., 45/55 °C)
```

Internally this wraps `\qtyrange` with `range-phrase={/)}` and `range-units=single`, so the degree-Celsius symbol appears only once at the end.

16.5 Convention Reference

The following table lists all math macros provided by `omnilatex-math.sty`, organised by category:

Table 16.5: Complete math macro reference..

Category	Command	Description
Operators	`\grad`	Gradient (grad, upright)
	`\const`	Constant notation
	`\qq{text}`	Quad-spaced text in math
	`\nablaoperator[n]{expr}`	Nabla with optional degree
Vectors	`\vect{v}`	Bold italic vector
Derivatives	`\deriv[n]{f}`	Operator form (total)
	`\deriv*[n]{f}`	Operator form (partial)
	`\fracderiv[n]{f}{x}`	Fraction form (total)
	`\fracderiv*[n]{f}{x}`	Fraction form (partial)
	`\timederiv[n]{f}`	Fraction form w.r.t. time
	`\timederiv*[n]{f}`	Partial time derivative
	`\posderiv[n]{f}`	Fraction form w.r.t. position
	`\posderiv*[n]{f}`	Partial positional derivative
Physical	`\flow{x}`	Dotted flow symbol
	`\difference{x}`	Delta difference
	`\heatexentry{x}`	Heat exchanger entry (prime)
	`\heatexexit{x}`	Heat exchanger exit (double prime)
	`\temperaturepair{a}{b}`	Temperature range
Statistics	`\mean{x}`	Arithmetic mean (overbar)
	`\logmean{x}`	Logarithmic mean (tilde)
	`\variance{x}`	Variance (σ^2)
Formatting	`\abs{x}`	Absolute value
	`\norm{x}`	Norm (double bars)
	`\inner{a}{b}`	Inner product (angle brackets)
	`\circlednum{n}`	Circled number ①
Punctuation	`\eqend`	Thin space + period
	`\eqcomma`	Thin space + comma

17 CJK Mathematics

OmniLaTeX supports CJK (Chinese, Japanese, Korean) characters in mathematical contexts through `luatexja`. This chapter covers math-specific CJK features; for full language support including document-level configuration, see Chapter 43.

17.1 CJK Math Fonts

When using LuaLaTeX, `luatexja` provides native support for CJK characters inside math mode. CJK glyphs are automatically rendered alongside Latin math symbols without additional configuration.

```
\[
  E = mc^2 \quad \text{質量} = mc^2
\]
```

`luatexja` maps CJK characters to appropriate math fonts based on the document's font configuration. The main text CJK font is used by default for CJK characters in math mode, ensuring visual consistency between inline text and displayed equations.
For upright CJK text within math mode, use the standard `\text{}` command:

```
$T_{\text{質量}} = 0.85$
% → T_質量 = 0.85
```

17.2 Ruby Annotations

Ruby annotations (furigana in Japanese, pinyin in Chinese) place small reading aids above CJK characters. OmniLaTeX provides these via the `luatexja-ruby` package.

```
\ruby{漢字}{かんじ}        % Japanese furigana
\ruby{漢字}{hànzì}          % Chinese pinyin
```

Shorthand aliases are available for convenience:

Table 17.1: Ruby annotation commands..

Command	Alias	Description
`\ruby{漢字}{かんじ}`	–	Full ruby annotation
`\furigana{漢字}{かんじ}`	`\ruby`	Japanese furigana alias
`\pinyin{漢字}{hànzì}`	`\ruby`	Chinese pinyin alias

The annotation size can be adjusted globally:

```
\setRubyScale{0.5}   % Scale ruby text to 50% of base size
```

Ruby annotations work in both text and math mode, making them useful for annotating technical terms that contain CJK characters in equations.

17.3 Vertical Writing

luatexja supports vertical (top-to-bottom, right-to-left) text layout through the vertical environment:

```
\begin{vertical}
  ▒▒▒▒▒▒▒▒▒▒▒▒
  \[ E = mc^2 \]
\end{vertical}
```

For inline vertical text spans, use:

```
\verticaltext{▒▒▒▒▒▒▒▒}
```

Math environments within vertical mode are rotated so that equations remain readable. Displayed equations appear rotated 90 degrees counter-clockwise relative to the vertical text flow, following Japanese typesetting conventions (JIS X 4051).
Note that vertical writing mode interacts with page layout settings. Margins, floats, and captions are automatically adjusted by luatexja when the vertical environment is active.

17.4 CJK-Latin Spacing

Proper inter-character spacing between CJK and Latin scripts is handled by luatexja's spacing engine. A small amount of space is automatically inserted at CJK–Latin boundaries to prevent characters from appearing cramped.

```
\cjklatinspacing  % Enable CJK-Latin inter-character spacing
```

Fine-grained control is available through:

```
\setCJKLatinSpacing{0.25em}  % Set explicit CJK-Latin gap
```

The default spacing is determined by the font metrics and generally does not require manual adjustment. However, in math-heavy documents with mixed CJK and Latin symbols, explicit spacing control can improve readability when the default gap is too small or too large for the chosen font combination.
For complete CJK language setup including font selection, input methods, and document-level options, refer to Chapter 43.

Part IV

Language & Internationalization

18 Internationalization System

OmniLaTeX provides a layered internationalization (i18n) system that handles language rules, string localization, and bidirectional text. This chapter covers the translation infrastructure; see Chapters 41 and 42 for multilingual documents and RTL scripts respectively.

18.1 polyglossia Setup

OmniLaTeX loads `polyglossia` through `omnilatex-i18n.sty`, which serves as the LuaLaTeX replacement for `babel`. The primary document language is set via the class option `language=<lang>` and propagated to `polyglossia` using a double-`\expandafter` pattern that ensures the macro is fully expanded before `\setdefaultlanguage` receives it.
In addition to the primary language, 27 secondary languages are registered via `\setotherlanguages`, making them available for inline use with `\begin{<lang>}...\end{<lang>}` or `\foreignlanguage{<lang>}{...}`:

```
\setotherlanguages{
  german, ngerman, english, french, spanish, italian, portuguese,
  russian, dutch, polish, czech, greek, turkish, simplifiedchinese,
  traditionalchinese, japanese, korean, arabic, hebrew, vietnamese,
  hindi, swedish, finnish, danish, norsk
}
```

This design allows a German thesis to include English passages with a simple `\begin{english}...\end{english}` switch, or an English document to embed Arabic text inline.

18.2 Translation System

Beyond hyphenation and caption rules, OmniLaTeX uses the `translations` package for localized strings that `polyglossia` does not cover (e.g. custom headings, institutional labels). Translations are declared with `\DeclareTranslation` and retrieved with `\GetTranslation`:

```
\DeclareTranslation{german}{Gloss}{Glossar}
\GetTranslation{Gloss}   % → "Glossary" or "Glossar" depending on locale
```

Users can override any translation without modifying the package:

```
\RenewTranslation{german}{author}{Autorin}
```

This is particularly useful for gender-neutral or gender-specific academic roles. The document preamble in `config/document-settings.sty` demonstrates overriding `author`, `FirstExaminer`, `SecondExaminer`, and `Supervisor` for German documents. OmniLaTeX also uses `\GetTranslationFor{<lang>}{<key>}` to inspect translations for a specific language, enabling conditional logic such as selecting the correct German definite article (`der`/`die`) based on the gendered form of `author`.

18.3 Language Fallback Chain

When a translation key is not defined for the current document language, the `translations` package falls back to English. This behaviour is formally verified in `LanguageFallback.lean`, which defines the fallback function as:

```
def fallback : PrimaryLanguage → PrimaryLanguage
  | .english => .english
  | _        => .english
```

Two properties are machine-checked:

- `fallback_terminates`: for every language l, fallback(l) = english.
- `fallback_stable`: fallback(fallback(l)) = fallback(l), guaranteeing that repeated fallbacks do not oscillate.

English also serves as the fixed point: fallback(english) = english. This ensures that all 18 primary languages resolve to a defined string even if a particular key is missing.

18.4 Dynamic Language Switching

Any registered secondary language can be activated inline using polyglossia's environment form:

```
\begin{english}
  This paragraph uses English hyphenation and caption rules.
\end{english}
```

This is essential for bilingual documents where, for example, a German thesis must include an English abstract for international readers, or where legal disclaimers must appear in a specific language. The language switch affects hyphenation patterns, caption strings, and `\GetTranslation` resolution for the duration of the environment.

18.5 Translation Key Categories

The 47 translation keys are organized into functional categories. Completeness is verified in `I18nCompleteness.lean`:

Document metadata (7) `Title`, `Author`, `CompiledOn`, `CensorNotice`, `PlaceDate`, `Topic`, `Task`, `For`.

Academic roles (5) `FirstExaminer`, `SecondExaminer`, `Supervisor`, `Examiner`, `AuthorshipDeclTitle`, `AuthorshipDeclText`, `authorArticle`.

Glossary groups (9) `Gloss`, `Greek`, `Roman`, `Other`, `Terms`, `Names`, `Acronyms`, `Subscripts`, `Superscripts`, `SubSuperTitle`.

Content lists (8) `ListOfExamples`, `ListOfListings`, `ListOfIllustrations`, `IllustrativeElements`, `Listing`, `Listings`, `Example`, `Examples`, `FurtherReadingTitle`, `FurtherReadingText`.

Scientific notation (4) `Constant`, `Unit`, `PhysicalQuantity`, `AdaptedFrom`.

Reactions (2) `Reaction`, `Reactions`.

Table formatting (2) `Contfoot`, `Conthead`.

Layout (4) `BlankPage`, `First`, `Second`, `LatexClass`, `Generator`.

18.6 Complete Key Table

The following table lists representative keys from each category. The full list of all 47 keys with translations in all 18 languages is documented in Chapter 44.

Table 18.1: Selected translation keys and their English values..

Category	Key	English value
Metadata	`CompiledOn`	Compiled on
Metadata	`PlaceDate`	Place & Date
Academic	`FirstExaminer`	Examiner
Academic	`Supervisor`	Supervisor
Academic	`AuthorshipDeclTitle`	Declaration of Authorship
Glossary	`Gloss`	Glossary
Glossary	`Acronyms`	Acronyms
Glossary	`Subscripts`	Subscripts
Lists	`ListOfExamples`	List of Examples
Lists	`FurtherReadingTitle`	Further Reading
Science	`Constant`	const
Science	`PhysicalQuantity`	Physical Quantity
Science	`AdaptedFrom`	Adapted from
Reaction	`Reaction`	Reaction
Table	`Contfoot`	Continued on next page
Table	`Conthead`	(Continued)
Layout	`BlankPage`	Rest of this page intentionally left blank.

19 Multilingual Documents

OmniLaTeX's translation infrastructure enables documents that mix multiple languages within a single compilation. This chapter provides practical guidance for authoring multilingual content.

19.1 Multi-Language Example

The `examples/multi-language/` directory demonstrates a document that combines English and German content. The document is compiled with English as the primary language and uses inline language switching for German passages:

```
\documentclass[language=english]{omnilatex}
\begin{document}

\begin{english}
  This is the English abstract.
\end{english}

\begin{german}
  Dies ist die deutsche Zusammenfassung.
\end{german}

\end{document}
```

Within each language environment, `\GetTranslation` resolves to the appropriate localized string, and polyglossia applies the correct hyphenation patterns. Captions for figures and tables also switch automatically when inside a language environment.
The example includes a full bibliography and glossary setup that demonstrates how these subsystems interact with language switching. See `examples/multi-language/README.md` for build instructions.

19.2 Bibliography in Multiple Languages

OmniLaTeX uses `biblatex` with the `ext-authoryear` style, which supports multilingual citations out of the box. Bibliography headings are localized through the translation system:

```
\GetTranslation{FurtherReadingTitle}  % → "Further Reading" / "Weitere Literatur"
```

When a bibliography entry contains titles in multiple languages, `biblatex`'s `langid` field can be used to tag individual entries:

```
@book{mueller2020,
  author    = {Müller, Hans},
  title     = {Deutsche Fachsprache},
  langid    = {german},
}
```

The citation style automatically formats the entry according to its language tag, including correct quotation marks and date formatting.

19.3 Glossary in Multiple Languages

The `glossaries-extra` package used by OmniLaTeX supports multi-language labels through the translation system. Glossary group titles are resolved via `\GetTranslation`:

```
\glsxtrsetgrouptitle{greek}{\GetTranslation{Greek}}
\glsxtrsetgrouptitle{roman}{\GetTranslation{Roman}}
```

This means glossary headings automatically appear as “Griech.” in German, “Grec” in French, or “Greek” in English, depending on the active language context.
`\printunsrtglossary` can be called inside a language environment to produce a glossary section with headings in the corresponding language. For documents that maintain separate glossaries per language, call `\printunsrtglossary` multiple times within different language environments.

19.4 Practical Tips

- **Consistent language use.** Define a clear boundary between languages in your document. Using language environments around entire sections or chapters is cleaner than switching mid-paragraph.
- **Translation completeness.** OmniLaTeX ships translations for 47 keys across 18 languages (verified in `I18nCompleteness.lean`). If you add custom keys, declare translations for all required languages to avoid silent English fallbacks.
- **Test all variants.** Compile the document with each target language as the primary language (`language=german`, `language=french`, etc.) to verify that captions, glossary headings, and translated strings render correctly.
- **Institution-specific keys.** Institution modules may add their own translation keys (e.g. `TUHH`, `LinkTUHH`). Ensure these are also translated if you need multilingual output.

20 RTL Scripts

OmniLaTeX supports right-to-left (RTL) scripts–Arabic, Hebrew, and Persian–through the `omnilatex-rtl.sty` module. This chapter covers RTL font configuration, bidi layout, and the module architecture.

20.1 Arabic

Arabic documents are activated by setting `language=arabic` in the class options. OmniLaTeX auto-detects the Arabic language and loads the RTL module, which configures bidi layout, script fonts, and RTL-aware math mode.
The default Arabic font is **Amiri** with **Amiri Bold** for bold weight. To override the font, use `\setArabicFont` in the preamble:

```
\setArabicFont{Noto Naskh Arabic}
\setArabicBoldFont{Noto Naskh Arabic Bold}
```

If the requested font is not found, OmniLaTeX falls back to **Noto Naskh Arabic** (bundled with TEX Live) and emits a `ClassWarning`.
Arabic-Indic digits (٠١٢٣٤٥٦٧٨٩) can be enabled in place of Western Arabic digits:

```
\arabicindictrue    % Use ٠١٢٣٤٥٦٧٨٩
\arabicindicfalse   % Use 0123456789 (default)
```

The bidi layout mirrors margins so the binding edge is on the right side for RTL documents. Lists, TOC dot leaders, and float placement are all automatically adjusted by the `bidi` package.
A complete Arabic example is available in `examples/rtl-arabic/`.

20.2 Hebrew

Hebrew support is activated with `language=hebrew`. The default font is **David CLM** with **David CLM Bold** for bold weight:

```
\setHebrewFont{David CLM}
\setHebrewBoldFont{David CLM Bold}
```

If David CLM is not installed, OmniLaTeX falls back to **Frank Ruehl CLM**. Hebrew setup follows the same RTL layout rules as Arabic: mirrored margins, bidi paragraph direction, and RTL-aware list markers.
A complete Hebrew example is available in `examples/rtl-hebrew/`.

20.3 Persian

Persian (Farsi) support is activated with `language=persian`. Persian uses the same Arabic script fonts but is tracked as a separate script by the RTL module via `\omnilatex@rtlscript`.
Persian reuses the Arabic font configuration (`\setArabicFont`) since the Arabic script covers Persian glyphs. The RTL layout, math mode direction, and list alignment are applied identically to Arabic.

20.4 RTL Title Page

KOMA-Script's `\usekomafont{title}` defaults to `\sffamily`, which overrides fontspec's automatic script-based font detection. OmniLaTeX works around this by overriding the title page fonts at `\AtBeginDocument`:

```
\setkomafont{title}{\arabicfont\bfseries\Huge}     % Arabic/Persian
\setkomafont{title}{\hebrewfont\bfseries\Huge}       % Hebrew
```

The override applies to all title page elements: `title`, `author`, `subtitle`, `date`, and `publishers`. This ensures the title page renders in the correct script font regardless of KOMA-Script's font family selection.

20.5 Inline Direction Control

For mixed-direction documents, OmniLaTeX provides two convenience commands that wrap the `bidi` package's direction primitives:

Table 20.1: Inline direction control commands..

Command	Underlying	Use case
`\LTRinline{text}`	`\textLR`	LTR text in RTL document
`\RTLinline{text}`	`\textRL`	RTL text in LTR document

These are thin wrappers and can be used in any text context:

```
\RTLinline{□□□□□}  % Inline Arabic in an English document
\LTRinline{Hello}  % Inline English in an Arabic document
```

20.6 RTL Module Architecture

`omnilatex-rtl.sty` is structured as a pipeline of setup stages that execute when the module is loaded:

1. **Core bidi engine.** The `bidi` package provides fundamental RTL typesetting. It redefines LaTeX internals for direction-aware paragraph layout, lists, TOC leaders, and float placement.
2. **Font configuration.** Default fonts are stored in internal macros (`\omnilatex@arabicfont@default`, `\omnilatex@hebrewfont@default`). User-facing setter commands (`\setArabicFont`, `\setHebrewFont`) update these macros and, if the font family has already been initialized, recreate the `\newfontfamily` binding.
3. **Font application.** `\omnilatex@apply@arabicfonts` and `\omnilatex@apply@hebrewfonts` use `\IfFontExistsTF` to test whether the requested font is available. If not, they fall back to a TeX Live-bundled font and emit a warning.
4. **Script detection.** At load time, the module reads `\omnilatex@language` (set by `omnilatex.cls`) and dispatches to the appropriate setup: `\omnilatex@setup@arabic`, `\omnilatex@setup@hebrew`, or `\omnilatex@setup@persian`. Each setup applies layout mirroring, font bindings, math direction enforcement, and list alignment.
5. **Math mode direction.** Inline math is forced to left-to-right via `\everymath{\displaystyle\textdirection=0}`, ensuring equations remain readable in RTL text.

6. **Title page override.** At `\AtBeginDocument`, KOMA title fonts are replaced with script-specific font families for Arabic, Hebrew, and Persian.

The class file (`omnilatex.cls`) guards RTL module loading: it only loads `omnilatex-rtl.sty` when the document language is `arabic`, `hebrew`, or `persian`. For all other languages, the RTL module is not loaded, avoiding unnecessary bidi overhead.

21 CJK Scripts

OmniLaTeX provides first-class support for Chinese, Japanese, and Korean (CJK) scripts through luatexja and the Noto CJK font family. When you select a CJK language (e.g. `language=chinese-simplified`), OmniLaTeX automatically configures the appropriate fonts and script settings so you can mix CJK and Latin text seamlessly.

21.1 Chinese (Simplified)

Simplified Chinese uses the Noto Sans CJK SC and Noto Serif CJK SC font families, loaded through luatexja's font mechanism.

```
\setCJKMainFont{Noto Serif CJK SC}
\setCJKSansFont{Noto Sans CJK SC}
\setCJKMonoFont{Noto Sans Mono CJK SC}
```

OmniLaTeX calls `\setCJKScript{sc}` internally whenever `language=chinese-simplified` is detected, which tells luatexja to activate the Simplified Chinese OpenType script features (GB-standard glyphs, punctuation rules, and proportional spacing).
A complete example lives in `examples/cjk-chinese/`, which demonstrates:

- Mixed Latin–Chinese paragraphs with automatic inter-script spacing.
- Serif and sans-serif Chinese body text.
- Chinese-style punctuation (full-width commas, periods, enumeration commas "□").
- Headings, lists, and captions rendered in Chinese.

To compile a Simplified Chinese document:

```
\documentclass{omnilatex}
\usepackage[language=chinese-simplified]{omnilatex-lang}
\begin{document}
  \chapter{□□□□}
  □□□□□□□□□□□□
\end{document}
```

21.2 Chinese (Traditional)

Traditional Chinese switches to the TC (Traditional Chinese) variant of the Noto CJK family:

```
\setCJKMainFont{Noto Serif CJK TC}
\setCJKSansFont{Noto Sans CJK TC}
```

The script tag is `\setCJKScript{tc}`, activated by `language=chinese-traditional`. This selects the correct glyph set (e.g. □ vs. □, □ vs. □) and adjusts punctuation rules for the Traditional Chinese typographic convention used in Taiwan, Hong Kong, and Macau.

```
\documentclass{omnilatex}
\usepackage[language=chinese-traditional]{omnilatex-lang}
\begin{document}
  \chapter{??????}
  ??????????
\end{document}
```

21.3 Japanese

Japanese documents use the Noto CJK JP fonts together with luatexja:

```
\setCJKMainFont{Noto Serif CJK JP}
\setCJKSansFont{Noto Sans CJK JP}
\setCJKMonoFont{Noto Sans Mono CJK JP}
```

The script tag is `\setCJKScript{jp}`, activated by `language=japanese`. In addition to standard CJK support, OmniLaTeX enables *furigana* (reading aids) for Japanese text via luatexja-ruby.
A complete example lives in `examples/cjk-japanese/`, demonstrating furigana placement, mixed-script paragraphs, and Japanese-style punctuation and line-breaking.

```
\documentclass{omnilatex}
\usepackage[language=japanese]{omnilatex-lang}
\begin{document}
  \chapter{???????}
  ??????????
\end{document}
```

21.4 Korean

Korean documents use the Noto CJK KR font family:

```
\setCJKMainFont{Noto Serif CJK KR}
\setCJKSansFont{Noto Sans CJK KR}
```

The script tag is `\setCJKScript{kr}`, activated by `language=korean`. Hangul syllables are rendered with proper morphological spacing, and Korean-style punctuation (middle dots “·”) is supported.
A complete example lives in `examples/cjk-korean/`.

```
\documentclass{omnilatex}
\usepackage[language=korean]{omnilatex-lang}
\begin{document}
  \chapter{??? ??}
  ??? ??? ??????.
\end{document}
```

21.5 Auto-Detection

When you pass a CJK language option to omnilatex-lang, OmniLaTeX performs the following steps automatically:

1. Detects the language family (SC, TC, JP, KR) from the option name.
2. Calls `\setCJKScript{<tag>}` to activate the correct OpenType script.
3. Loads the matching Noto CJK font family for main, sans, and mono.
4. Sets luatexja parameters for inter-character spacing and line-breaking rules appropriate to the script.
5. Configures translation keys (chapter, figure, table, etc.) in the target language.

You can override any of these defaults by setting CJK fonts or scripts manually in your preamble *after* loading omnilatex-lang.

21.6 Ruby Annotations

Ruby annotations (small phonetic glosses above or beside characters) are supported via luatexja-ruby:

- `\ruby{▯▯}{▯▯▯}` – Japanese furigana (above).
- `\furigana{▯▯}{▯▯▯}` – Alias with identical behaviour.
- `\pinyin{▯▯}{hànzì}` – Chinese pinyin (above).

Control the ruby annotation size with `\setRubyScale{0.5}` (default `0.5`), which scales the annotation font relative to the base font size.

```
\ruby{▯▯}{▯▯▯}   % furigana
\pinyin{▯▯}{hànzì}  % pinyin
\setRubyScale{0.45}    % smaller annotations
```

21.7 Vertical Writing Mode

OmniLaTeX supports vertical (top-to-bottom, right-to-left) writing via the `vertical` environment provided by luatexja:

```
\begin{vertical}
  ▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯
\end{vertical}
```

For inline vertical text, use `\verticaltext{▯▯▯}`. Page layout in vertical mode is adjusted automatically–page numbers and running headers rotate to match the text direction.

21.8 CJK–Latin Spacing

When mixing CJK and Latin scripts, proper inter-character spacing prevents the text from looking cramped. OmniLaTeX configures luatexja to insert a thin space (typically $\frac{1}{4}$ em) between CJK and Latin characters by default.
You can fine-tune this behaviour:

```
\cjklatinspacing          % enable (default for CJK languages)
\nocjklatinspacing        % disable
\setCJKLatinSpacing{0.25} % set gap to 0.25 em
```

The spacing value is a fraction of one em in the current font size.

22 Translation Key Reference

OmniLaTeX uses translator to manage localised strings for all 18 supported languages. Every caption, heading, and label is controlled by a *translation key*. This chapter lists all 47 keys and explains how to add new ones.

22.1 Complete Key Table

The table below shows every translation key, its English value, and its purpose. Keys are grouped by category.

22.1.1 Document Metadata

Key	English Value	Purpose
Title	Title	Main document title label
Author	Author	Author name label
Date	Date	Date label
Abstract	Abstract	Abstract heading
Keywords	Keywords	Keywords heading

22.1.2 Academic

Key	English Value	Purpose
BachelorThesis	Bachelor's Thesis	Degree label for bachelor
MasterThesis	Master's Thesis	Degree label for master
DoctoralThesis	Doctoral Thesis	Degree label for doctoral
Dissertation	Dissertation	Generic dissertation label
Supervisor	Supervisor	Thesis supervisor label
Examiner	Examiner	Thesis examiner label
Advisor	Advisor	Alternate advisor label
Chair	Chair	Committee chair label
Reviewer	Reviewer	Reviewer label
Department	Department	Department label
Institution	Institution	Institution label
Faculty	Faculty	Faculty label

22.1.3 Glossary

Key	English Value	Purpose
Glossary	Glossary	Glossary heading
Acronyms	Acronyms	List of acronyms heading
Nomenclature	Nomenclature	Nomenclature heading
Notation	Notation	Notation heading
Symbol	Symbol	Symbol column header
Description	Description	Description column header
Unit	Unit	Unit column header

22.1.4 Lists

Key	English Value	Purpose
TableOfContents	Table of Contents	TOC heading
ListofFigures	List of Figures	LoF heading
ListofTables	List of Tables	LoT heading
ListofExamples	List of Examples	LoE heading
Bibliography	Bibliography	Bibliography heading
References	References	References heading (alt)
Index	Index	Index heading
Appendix	Appendix	Appendix heading

22.1.5 Content

Key	English Value	Purpose
Chapter	Chapter	Chapter heading prefix
Part	Part	Part heading prefix
Section	Section	Section heading prefix
Figure	Figure	Figure caption prefix
Table	Table	Table caption prefix
Example	Example	Example caption prefix
Equation	Equation	Equation label
Theorem	Theorem	Theorem heading
Proof	Proof	Proof heading
Definition	Definition	Definition heading

22.1.6 Reaction

Key	English Value	Purpose
Reaction	Reaction	Reaction caption prefix
Mechanism	Mechanism	Mechanism caption prefix
Scheme	Scheme	Scheme caption prefix

22.2 Adding New Keys

To introduce a new translation key:

1. Declare the key with an English default:

   ```
   \DeclareTranslation{english}{MyNewKey}{My Value}
   ```

2. Add the same key to all 18 supported languages. Each language file in `tex/latex/omnilatex-lang/` needs an entry:

   ```
   \DeclareTranslation{german}{MyNewKey}{Mein Wert}
   \DeclareTranslation{french}{MyNewKey}{Ma Valeur}
   \DeclareTranslation{japanese}{MyNewKey}{???}
   % ... and so on for all languages
   ```

3. Use the key in your templates or documents with `\translate{MyNewKey}`.

All translation declarations must appear in the preamble. If a key is missing for a given language, translator falls back to English automatically.

Part V

Layout & Structure

23 Page Layout

OmniLaTeX uses KOMA-Script's native page geometry system rather than the geometry package. This ensures full compatibility with KOMA's `\areaset`, running headers, and margin-par layout.

23.1 KOMA Page Geometry

KOMA-Script controls page layout through two class options:

`DIV` – Divisor that determines the text area proportions. Higher values yield larger margins. Common choices: `DIV=9` (narrow margins), `DIV=12` (default), `DIV=15` (generous margins).

`BCOR` – Binding correction (Bindekorrektur). Compensates for the inner margin lost to binding. E.g. `BCOR=5mm` shifts the text block 5 mm toward the outer edge on the spine side.

```
\documentclass[DIV=12, BCOR=5mm]{omnilatex}
```

OmniLaTeX defaults to `DIV=12, BCOR=0mm`. The `\areaset` command (KOMA-native) is used internally because it correctly handles the interaction between text width, text height, and the type area within KOMA's header/footer system. Do *not* load the geometry package alongside OmniLaTeX–it will conflict with KOMA's layout calculations.

23.2 Custom Margins

For situations that require precise control over all four margins, OmniLaTeX provides:

```
\setCustomMargins{25mm}{20mm}{30mm}{25mm}
%                 left  right  top   bottom
```

This command converts explicit margin values into equivalent KOMA `DIV`/`BCOR` parameters so that the layout remains consistent with the rest of the KOMA machinery. Call it in the preamble after `\begin{document}` or in a file loaded via `\BeforeBeginDocument`.

23.3 Two-Side Layout

The `twoside` class option enables double-sided typesetting:

```
\documentclass[twoside]{omnilatex}
```

In two-side mode:

- Odd (recto) pages have a wider right margin; even (verso) pages have a wider left margin (mirrored).
- Running headers and footers differ between odd and even pages.
- `BCOR` adds extra space on the spine side of every page.

One-sided layout (the default) gives uniform margins on all pages.

23.4 Headers & Footers

OmniLaTeX uses scrlayer-scrpage for running headers and footers:

```
\usepackage[automark]{scrlayer-scrpage}
```

The `automark` option automatically places the current chapter and section in the header. By default:

- **Odd pages:** chapter title (right) and section title (left).
- **Even pages:** section title (left) and chapter title (right).
- **Page number:** centred in the footer.

A thin rule below the header is enabled with `\headerrule`. Customise the rule thickness:

```
\setheadrule{0.4pt}
```

23.5 Landscape Pages

For wide tables or figures that do not fit in portrait orientation, OmniLaTeX provides a `landscape` environment:

```
\begin{landscape}
  \begin{table}
    \caption{Wide data table}
    % ... wide tabular ...
  \end{table}
\end{landscape}
```

The environment switches to landscape orientation, re-centres the header and footer, and increments page numbers normally. For a single-page landscape section, `\pdflandscape` can be used as a command.

23.6 A5 Format

OmniLaTeX supports A5 output via the `a5` class option:

```
\documentclass[a5paper]{omnilatex}
```

All margins, font sizes, and float placements scale appropriately. A5 is useful for pocket-sized handouts or supplementary booklets.

23.7 Page Numbering

LaTeX and KOMA-Script provide three standard numbering schemes:

`\frontmatter` – Roman numerals (i, ii, iii, …). Used for the title page, abstract, table of contents, and preface.

`\mainmatter` – Arabic numerals (1, 2, 3, …). Used for the body of the document. Resets to 1.

`\backmatter` – Roman numerals again. Used for the bibliography, appendices, and index.

```
\frontmatter
\tableofcontents
\mainmatter
\chapter{Introduction}
\backmatter
\bibliography{refs}
```

23.8 Intentionally Blank Pages

In two-side mode, chapters open on recto (odd) pages. KOMA-Script inserts a blank verso page when needed. OmniLaTeX marks these pages with the `blankpage` page style, which suppresses headers and footers but prints a small note at the bottom:

```
\renewcommand*{\blankpage}{%
  \thispagestyle{empty}%
  \null\vfill
  \centering\itshape This page is intentionally left blank.
  \vfill}
```

To disable the note, set `\blankpagestyle{empty}` in the preamble.

24 Sectioning

KOMA-Script extends the standard LaTeX sectioning commands with additional features such as flexible numbering, paragraph hooks, and style customisation. OmniLaTeX further tailours the sectioning defaults for a clean, modern look.

24.1 KOMA Section Hierarchy

The full hierarchy, from coarsest to finest:

```
\part          % I, II, III, ...
\chapter       % 1, 2, 3, ...      (book/report only)
\section       % 1.1, 1.2, ...
\subsection    % 1.1.1, 1.1.2, ...
\subsubsection % 1.1.1.1, ...
\paragraph     % 1.1.1.1.1, ...
\subparagraph  % 1.1.1.1.1.1, ...
```

Each level is numbered by default. The depth at which numbering and TOC entries appear is controlled by two counters:

```
\setcounter{secnumdepth}{3}  % number up to \subsubsection
\setcounter{tocdepth}{2}     % show up to \subsection in TOC
```

OmniLaTeX defaults to `secnumdepth=3` and `tocdepth=2`.

24.2 Chapter Prefix

OmniLaTeX scales the chapter number to 4.5× its normal size using KOMA's font interface:

```
\setkomafont{chapter}{\fontsize{45}{54}\selectfont\bfseries}
```

This creates a prominent visual anchor for each chapter opening. The chapter title itself is set in the default sans-serif font at normal size, creating a strong typographic contrast between the large number and the readable heading text.
To customise the chapter number size:

```
\setkomafont{chapter}{\fontsize{60}{72}\selectfont\bfseries}
```

24.3 Table of Contents

Generate a TOC with `\tableofcontents`. OmniLaTeX styles the TOC with a clean sans-serif layout and dotted leaders:

```
\frontmatter
\tableofcontents
\mainmatter
```

Control which sectioning levels appear:

```
\setcounter{tocdepth}{1}  % chapters only
\setcounter{tocdepth}{3}  % down to \subsubsection
```

Individual entries can be hidden with `\addtocontents{toc}{\vspace{1em}}` or by using the `tocdepth` counter locally within a chapter.

24.4 Lists of Floats

KOMA-Script provides built-in lists for the standard float types:

```
\listoffigures   % List of Figures
\listoftables    % List of Tables
```

OmniLaTeX also registers `example` as a float type, enabling:

```
\listofexamples  % List of Examples
```

For custom float types, declare a new list with `\DeclareNewTOC`:

```
\DeclareNewTOC[
  type=algorithm,
  name=List of Algorithms,
  listname={Algorithm}
]{loa}
```

This creates a `\listofalgorithms` command and a `loa` file for the auxiliary data.

24.5 PDF Bookmarks

PDF bookmarks (the navigable outline in PDF viewers) are generated automatically from sectioning commands. Add manual bookmarks with:

```
\pdfbookmark[0]{Title Page}{titlepage}
\pdfbookmark[1]{Appendix A}{appa}
```

The optional argument sets the bookmark level (0 = top-level, matching `\part`; 1 = chapter level). OmniLaTeX enables bold chapter bookmarks by default via `\boldPDFchapterstrue`. To disable:

```
\boldPDFchaptersfalse
```

24.6 Unnumbered Chapters

For chapters that should not appear in the numbering sequence:

```
\addchap{Acknowledgements}   % KOMA: unnumbered, appears in TOC
\chapter*{Colophon}          % LaTeX: unnumbered, absent from TOC
```

`\addchap` – KOMA-Script command. The chapter is unnumbered but still listed in the TOC and generates a PDF bookmark.

`\chapter*` – Standard LaTeX starred command. The chapter is unnumbered and does *not* appear in the TOC or PDF bookmarks.

OmniLaTeX recommends `\addchap` for front-matter chapters (abstract, acknowledgements, nomenclature) that should be listed but not numbered.

25 Title Pages

OmniLaTeX ships with four built-in title page styles and provides a framework for institutions to define their own branded title pages.

25.1 Built-in Title Styles

Select a style via the `titlestyle` class option:

`titlestyle=book` – A vertical rule separates the left author block from the right title block. The author name is set in small caps. Suitable for monographs and textbooks.

`titlestyle=thesis` – Similar vertical-rule layout but includes an examiners table below the author block, listing supervisor(s), examiner(s), degree, department, and date. Suitable for academic theses.

`titlestyle=simple` – The default KOMA-Script title page: centred title, author, and date with no decorative elements.

`titlestyle=TUHH` – A TUHH-branded title page with the university logo, vertical rule, document-type badge, and examiners table. Requires `institution=tuhh`.

```
\documentclass[titlestyle=thesis]{omnilatex}
```

The default is `titlestyle=simple`.

25.2 Title Page Metadata

All styles consume the same set of metadata commands:

```
\title{An OmniLaTeX Document}
\subtitle{A Subtitle Goes Here}
\author{Jane Doe}
\date{May 2026}
\publishers{OmniLaTeX Press}
```

For thesis-specific metadata, the following commands are recognised by the `thesis` and institution-specific styles:

```
\degree{Master of Science}
\department{Computer Science}
\institution{University of Example}
\supervisor{Prof.\ Alice Smith}
\examiner{Prof.\ Bob Jones}
\submissiondate{May 2026}
```

25.3 Custom Title Pages

Institutions can register their own title page styles using the `\DefineXxxTitleStyle` pattern. The TUHH style demonstrates the full mechanism:

1. **Logo placement** – The institution logo is positioned at the top of the page using `\includegraphics` inside a `tikzpicture` overlay.
2. **Vertical rule** – A coloured vertical rule (`3pt`, institutional colour) divides the left metadata block from the right title block.
3. **Document-type badge** – A small coloured box (e.g. Master's Thesis) identifies the document type.
4. **Examiners table** – A tabular lists supervisor, examiner, chair, and date in a compact two-column layout.

All of these elements are defined within `\DefineTUHHTitleStyle`, which is loaded automatically when `institution=tuhh` is set.

25.4 Thesis Title Page

The `thesis` style provides a complete academic title page. From top to bottom:

1. **Title and subtitle** – Centred or left-aligned depending on the institution override.
2. **Author line** – Full name of the candidate.
3. **Examiners table** – Two-column layout:

Supervisor:	Prof. Alice Smith
Examiner:	Prof. Bob Jones
Degree:	Master of Science
Department:	Computer Science
Date:	May 2026

4. **Institution** – University name and optional logo.

To use:

```
\documentclass[titlestyle=thesis]{omnilatex}
\title{My Thesis Title}
\author{Jane Doe}
\degree{Master of Science}
\department{Computer Science}
\institution{University of Example}
\supervisor{Prof.\ Alice Smith}
\examiner{Prof.\ Bob Jones}
\date{May 2026}
\begin{document}
  \maketitle
\end{document}
```

25.5 Creating Custom Styles

To create a new institution-specific title page:

1. Create a file `omnilatex-title-`*`slug`*`.sty` in the `tex/latex/omnilatex/` directory.
2. Define the style command:

```
\newcommand{\DefineMyUniTitleStyle}{%
  \renewcommand{\maketitle}{%
    \begin{titlepage}
      % your custom layout here
    \end{titlepage}%
  }%
}
```

3. Register the style with OmniLaTeX's option parser so that `institution=myuni` triggers loading of the file and calls `\DefineMyUniTitleStyle`.
4. Place any required logos or assets in `assets/institutions/myuni/`.
5. Add translations for any new metadata keys (see Chapter ??).

The style command should be idempotent (safe to call multiple times) and should not hard-code font sizes–use `\large`, `\Large`, and `\LARGE` relative size commands so that the layout scales correctly with different base font sizes.

26 Institutions

26.1 Overview

OmniLaTeX ships with 16 institution configurations located in `config/institutions/`. Each institution package provides:

- Logo definitions via `\includesvg` or `\includegraphics`
- Primary and secondary color definitions through `\setkomacolor`
- Optional custom title pages built with TikZ overlays
- Translation-aware branding strings via `\DeclareTranslation`

To activate an institution, load its package in your preamble or pass the institution name through the OmniLaTeX configuration mechanism. The default institution is `tuhh`, which serves as the reference implementation.

26.2 Institution Reference Table

Table 26.1 / Institution configurations shipped with OmniLaTeX

Institution	Primary Color (RGB)	Has Logo	Custom Title
tuhh	–	YES	YES
cambridge	(163,193,173)	commented	commented
cmu	(196,18,48)	commented	commented
columbia	(185,217,235)	commented	commented
epfl	(255,0,0)	commented	commented
eth	(31,64,122)	commented	commented
harvard	(165,28,48)	commented	commented
imperial	(0,62,116)	commented	commented
mit	(163,31,52)	commented	commented
oxford	(0,33,71)	commented	commented
princeton	(231,117,0)	commented	commented
stanford	(140,21,21)	commented	commented
tudelft	(0,166,214)	commented	commented
tum	(0,101,189)	commented	commented
yale	(0,53,107)	commented	commented

Continued on next page

Table 26.1 / Institution configurations shipped with OmniLaTeX (Continued)

Institution	Primary Color (RGB)	Has Logo	Custom Title
generic	–	commented	commented

Institutions marked *commented* ship with placeholder definitions that can be activated by uncommenting the relevant sections and supplying the appropriate logo files.

26.3 TUHH in Detail

The `tuhh` institution is the most complete configuration and serves as the template for all others. Key features include:

26.3.1 Logo Definitions

The TUHH logo is defined as an SVG and embedded via `\includesvg`:

```
\DeclareTranslation{english}{LogoInstitution}{%
    \includesvg[height=1.2cm]{tuhh-logo}%
}
```

Bilingual support is achieved through separate translations for German and English:

```
\DeclareTranslation{ngerman}{LogoInstitution}{%
    \includesvg[height=1.2cm]{tuhh-logo-de}%
}
```

26.3.2 Custom Title Style

The `\DefineTUHHTitleStyle` command creates a TikZ overlay title page with the institution logo, a colored banner, and the document metadata:

```
\DefineTUHHTitleStyle{%
    \begin{tikzpicture}[remember picture, overlay]
        % Background fill
        \fill[tuhhblue] ...
        % Logo placement
        \node[anchor=north east] ...
        % Title text
        \node[anchor=west] ...
    \end{tikzpicture}%
}
```

26.3.3 TUHHuman TikZ Pic

The TUHH configuration also defines a `TUHHuman` TikZ `pic` for decorative use on title pages and section dividers. It is invoked with:

```
\pic at (0,0) {TUHHuman};
```

26.4 Creating Your Own Institution

Follow these steps to add a new institution configuration:

1. **Create the directory:**

   ```
   mkdir config/institutions/myuni/
   ```

2. **Create the package file** `myuni.sty` with a proper package declaration:

   ```
   \ProvidesPackage{myuni}[2026/01/01 My University Config]
   ```

3. **Define the logo** using `\DeclareTranslation` so that it is language-aware:

   ```
   \DeclareTranslation{english}{LogoMyuni}{%
       \includesvg[height=1.2cm]{myuni-logo}%
   }
   ```

4. **Set institution colors** via KOMA-Script color commands:

   ```
   \setkomacolor{title}{RGB}{0,50,100}
   \setkomacolor{sectioning}{RGB}{0,50,100}
   ```

5. **Optionally define a custom title page** using TikZ overlays, following the TUHH pattern shown above.

26.5 Logo Management

OmniLaTeX uses `\includesvg` from the `svg` package to embed vector graphics. The workflow requires:

- **Inkscape** must be installed and available on `PATH`. The `svg` package invokes Inkscape to convert SVG files to PDF on the fly.
- **`--shell-escape`** must be passed to the LaTeX engine to allow external command execution.
- **`\svgpath`** configures search paths for SVG files, analogous to `\graphicspath`:

  ```
  \svgpath{{images/logos/}{assets/}}
  ```

- Converted PDFs are cached alongside the source SVG so subsequent compilations are faster.

For debugging SVG inclusion, consult the generated `.svg-inkscape` log files in the output directory.

Part VI

Figures, Tables & Floats

27 Floats

27.1 Float Placement

KOMA-Script extends the standard LaTeX float placement with additional specifiers. OmniLaTeX loads the `floatrow` package to provide automatic caption positioning and improved float layouts.

Table 27.1 / Float placement specifiers

Specifier	Meaning
h	Place here if possible
t	Place at the top of a page
b	Place at the bottom of a page
p	Place on a dedicated float page
H	Place exactly here (requires `float` package)
!	Override internal LaTeX placement constraints
+	Allow floats to appear in the current region (KOMA extension)

The default placement in OmniLaTeX is `htbp`. Override it globally or per-float:

```
\begin{figure}[!htb]
    ...
\end{figure}
```

27.2 Continued Floats

For figures that span multiple pages, use `\ContinuedFloat` from the `caption` package. The caption counter is not incremented and a *(continued)* marker is appended:

```
\begin{figure}[htbp]
    \caption{First part of a multi-page figure}
    \label{fig:long-first}
    \includegraphics[width=\textwidth]{part1}
\end{figure}

\begin{figure}[htbp]
    \ContinuedFloat
    \caption{Second part of a multi-page figure}
    \label{fig:long-second}
    \includegraphics[width=\textwidth]{part2}
\end{figure}
```

27.3 Float Footnotes

Standard LaTeX footnotes inside floats are fragile and may not survive page breaks. OmniLaTeX provides a robust mechanism:

```
\omnlFloatFootmark{fn1}
...
\omnlFloatFoottext{fn1}{This footnote survives page breaks.}
```

The footnote mark is rendered inline at the point of invocation, while the footnote text is placed below the caption. Multiple keys can be used within a single float.

27.4 Side Captions

Place captions beside figures using a `minipage` layout:

```
\begin{figure}[htbp]
    \begin{minipage}[c]{0.65\textwidth}
        \includegraphics[width=\textwidth]{photo}
    \end{minipage}%
    \hfill
    \begin{minipage}[c]{0.30\textwidth}
        \captionof{figure}{Description alongside the image.}
        \label{fig:side-caption}
    \end{minipage}
\end{figure}
```

27.5 Key-Value Float Interface

OmniLaTeX provides high-level commands for figures and tables using key-value syntax:

```
\omnlFigure[
    caption        = {An example figure},
    label          = {fig:kv-example},
    placement      = htbp,
    caption-width  = 0.8\textwidth,
    caption-position = bottom,
]{%
    \includegraphics[width=\textwidth]{example-image}
}
```

Table 27.2 / `\omnlFigure` options

Key	Description
caption	Caption text
label	Label for cross-references

Continued on next page

Table 27.2 / `\omnlFigure` options (Continued)

Key	Description
placement	Float placement specifiers (default: `htbp`)
caption-width	Width of the caption box
caption-position	`top` or `bottom` (default: `bottom`)
source	Source attribution text

The `\omnlTable` command accepts the same keys and wraps content in a `table` environment.

27.6 Float Grid

For multi-column float layouts, use `omnlFloatRow`:

```
\omnlFloatRow[n-cols=3]{%
    \includegraphics[width=\linewidth]{img1}%
    \includegraphics[width=\linewidth]{img2}%
    \includegraphics[width=\linewidth]{img3}%
}
```

This creates an evenly spaced row of images. The optional `n-cols` parameter defaults to 2.

27.7 Figure Environment

The standard `figure` environment is fully supported with OmniLaTeX enhancements:

```
\begin{figure}[htbp]
    \centering
    \includegraphics[width=0.9\textwidth]{diagram}
    \caption{A standard figure with OmniLaTeX styling.}
    \label{fig:standard}
\end{figure}
```

OmniLaTeX automatically applies the configured caption font and separator style to all float captions.

27.8 Table Environment

Similarly, the `table` environment benefits from OmniLaTeX defaults:

```
\begin{table}[htbp]
    \centering
    \begin{tabular}{lcc}
        \toprule
        Parameter & Value & Unit \\
        \midrule
        Mass      & 1.23  & kg   \\
        Length    & 4.56  & m    \\
        \bottomrule
```

```
    \end{tabular}
    \caption{A standard table with booktabs styling.}
    \label{tab:standard}
\end{table}
```

28 Figures

28.1 Including Images

The primary command for raster and vector image inclusion is `\includegraphics` from the `graphicx` package:

```
\includegraphics[width=\textwidth]{images/photo}
\includegraphics[height=5cm, keepaspectratio]{diagram}
\includegraphics[scale=0.5]{chart}
```

Table 28.1 / Common `\includegraphics` options

Option	Effect
width	Scale to given width
height	Scale to given height
scale	Scale factor relative to natural size
keepaspectratio	Maintain aspect ratio when both width and height are set
angle	Rotate the image by the given angle in degrees
trim	Crop the image: `left bottom right top`
clip	Enable cropping set by `trim`
draft	Show a bounding box instead of the image
page	Select a page from a multi-page PDF

OmniLaTeX configures `\graphicspath` to search common asset directories. Place images in `images/`, `figures/`, or `assets/` without specifying the subdirectory in the path argument.

28.2 SVG Graphics

OmniLaTeX supports SVG graphics through the `svg` package, which converts SVG files to PDF using Inkscape at compile time.

```
\includesvg[width=\textwidth]{logo}
```

- **Requirements:** Inkscape must be installed and the `--shell-escape` flag must be passed to the LaTeX engine.
- **Search paths:** Configure additional SVG search directories with `\svgpath`:

  ```
  \svgpath{{images/svg/}{assets/vector/}}
  ```

- **Debugging:** Use `\debugtikzsvg` to enable verbose output during SVG conversion, showing Inkscape commands and intermediate file paths.

Converted PDFs are cached next to the source SVG file. Delete the cached `.pdf` to force re-conversion after editing the SVG.

28.3 Text Over Images

When placing text over images with varying backgrounds, use contour commands for legibility:

```
% White contour for dark backgrounds
\ctrw{Readable on dark images}

% Black contour for light backgrounds
\ctrb{Readable on light images}
```

These commands wrap the `contour` package to draw a thin outline around each glyph, ensuring the text remains legible regardless of the underlying image content.

28.4 TikZ Basics

OmniLaTeX loads common TikZ libraries through `omnilatex-tikz-core.sty`. The following libraries are available by default:

```
\usetikzlibrary{
    positioning,
    arrows.meta,
    calc,
    shapes.geometric,
    decorations.pathreplacing,
    backgrounds,
    fit,
}

\begin{tikzpicture}
    \node[draw, circle] (a) {A};
    \node[draw, circle, right=2cm of a] (b) {B};
    \draw[-{Stealth}] (a) -- (b);
\end{tikzpicture}
```

Coordinate systems, basic shapes, nodes, and edges all follow standard TikZ conventions. Refer to the TikZ manual for the full set of available options.

28.5 Subfigures

Use the `subcaption` package for side-by-side figures with individual labels:

```
\begin{figure}[htbp]
    \centering
    \begin{subfigure}[b]{0.45\textwidth}
        \centering
        \includegraphics[width=\textwidth]{img-left}
```

```
        \caption{Left subfigure}
        \label{fig:sub-left}
    \end{subfigure}
    \hfill
    \begin{subfigure}[b]{0.45\textwidth}
        \centering
        \includegraphics[width=\textwidth]{img-right}
        \caption{Right subfigure}
        \label{fig:sub-right}
    \end{subfigure}
    \caption{Two subfigures side by side.}
    \label{fig:subfigures}
\end{figure}
```

Subfigure labels are formatted as *(a)*, *(b)*, etc. Cross-reference individual subfigures with `\cref{fig:sub-left}` and the parent figure with `\cref{fig:subfigures}`.

28.6 Figure References

Use the `cleveref` package for automatic naming of figure references:

```
As shown in \cref{fig:example}, ...
See also \cref{fig:sub-left,fig:sub-right}.
```

OmniLaTeX configures the `cleveref` capitalization names so that `\Cref` produces *Figure* at the start of a sentence. Customize the naming with `\crefname`:

```
\crefname{figure}{fig.}{Figs.}
\Crefname{figure}{Figure}{Figures}
```

29 Tables

29.1 booktabs

The `booktabs` package provides professional-quality horizontal rules. OmniLaTeX loads it by default. The booktabs philosophy discourages vertical lines and encourages generous spacing.

```
\begin{tabular}{lcc}
    \toprule
    Parameter   & Value  & Unit \\
    \midrule
    Temperature & 298.15 & K    \\
    Pressure    & 101.3  & kPa  \\
    \addlinespace
    Density     & 1.225  & kg/m$^{3}$ \\
    \bottomrule
\end{tabular}
```

Table 29.1 / booktabs rule commands

Command	Purpose
`\toprule`	Heavy rule at the top of the table
`\midrule`	Medium rule below the header
`\bottomrule`	Heavy rule at the bottom of the table
`\addlinespace`	Insert extra vertical space between rows

Avoid `\hline` and vertical lines ({) in booktabs tables.

29.2 siunitx S-Columns

The `siunitx` package provides the `S` column type for automatic alignment of numbers by their decimal point:

```
\begin{tabular}{l S[table-format=3.2] S[table-format=3.2]}
    \toprule
    {Material} & {Length (mm)} & {Mass (g)} \\
    \midrule
    Steel      & 125.40        & 492.10     \\
    Aluminum   &  98.70        &  33.55     \\
    Copper     & 100.00        & 893.30     \\
    \bottomrule
\end{tabular}
```

Enclose column headers in braces (`{...}`) to prevent siunitx from parsing them as numeric values. The `table-format` key specifies the expected number of digits before and after the decimal separator.

29.3 tabularray

OmniLaTeX recommends the `tabularray` package for complex tables. It provides a modern key-value interface, long table support, and built-in row coloring.

```
\begin{longtblr}[
    caption = {Experimental results},
    label   = {tab:results},
    remark  = {Note},
]{colspec = {l S[table-format=2.2] S[table-format=2.2]},
    rowhead = 1,
    row{odd} = {gray9},
}
    \toprule
    Sample & {Voltage (V)} & {Current (A)} \\
    \midrule
    A      & 3.14          & 0.42          \\
    B      & 5.00          & 1.07          \\
    C      & 2.71          & 0.88          \\
    \bottomrule
\end{longtblr}
```

Variable-width columns use `l`, `c`, `r`, or explicit widths: `X[2,c]` for a centered column twice the default width.

29.4 Multi-Column Cells

Span cells across multiple columns with `\multicolumn`:

```
\begin{tabular}{lccc}
    \toprule
    Experiment & \multicolumn{2}{c}{Results} & Status \\
    \cmidrule(lr){2-3}
    Run 1      & 0.95 & 0.97 & Pass \\
    Run 2      & 0.82 & 0.79 & Fail \\
    \bottomrule
\end{tabular}
```

Use `\cmidrule(lr){2-3}` for partial horizontal rules that span only the merged columns. The `(lr)` trims shorten the rule at both ends for a cleaner appearance.

29.5 Rotated Cells

Rotate text within table cells using `\multirotatecell`:

```
\multirotatecell{3}{Rotated text spanning three rows}
```

This is particularly useful for column headers with long labels in wide tables. The first argument specifies the number of rows the cell should span.

29.6 Table Footnotes

Add footnotes to tabularray tables with `\TblrNote` and `\TblrRemark`:

```
\begin{longtblr}[remark = {Notes}]{colspec = {l S}, remark = {Note}}
    \toprule
    Entry & {Value} \\
    \midrule
    Alpha & 1.23\TblrNote{a} \\
    Beta  & 4.56\TblrNote{b} \\
    Gamma & 7.89 \\
    \bottomrule
\end{longtblr}
\TblrNote{a}{Measured at 25\,°C.}
\TblrNote{b}{Average of three trials.}
\TblrRemark{c}{Data collected in 2025.}
```

Notes are collected below the table and referenced by key.

29.7 Table Styles

OmniLaTeX defines several tabularray templates for consistent table styling:

Table 29.2 / OmniLaTeX tabularray templates

Template	Purpose
contfoot-text	Table with continued footer text across pages
conthead-text	Table with continued header text across pages
caption-sep	Enforced vertical separation between caption and table body

Apply a template with the `theme` key:

```
\begin{longtblr}[theme = caption-sep, caption = {...}]
    ...
\end{longtblr}
```

These templates are defined in the OmniLaTeX core style files and can be extended or overridden in your own preamble.

30 PGFPlots

30.1 Overview

PGFPlots is loaded by OmniLaTeX through `omnilatex-tikz-core.sty` with `compat=1.18`. This provides a consistent coordinate system, automatic axis scaling, and a rich set of plot types. All axes default to SI unit formatting via `siunitx`.

```
\usepackage{pgfplots}
\pgfplotsset{compat=1.18}
```

OmniLaTeX sets up default plot styles, color cycles, and axis label formatting so that most plots work out of the box without additional configuration.

30.2 Line & Scatter Plots

The `regularplot` style is the default for line and scatter plots:

```
\begin{tikzpicture}
    \begin{axis}[
        regularplot,
        xlabel={Time (s)},
        ylabel={Temperature (K)},
    ]
        \addplot coordinates {
            (0, 293) (1, 295) (2, 300)
            (3, 308) (4, 318) (5, 325)
        };
    \end{axis}
\end{tikzpicture}
```

OmniLaTeX provides several custom plot styles:

Table 30.1 / OmniLaTeX plot styles

Style	Description
regularplot	Default style with grid, marks, and axis labels
plainplot	Minimal style: no grid, no axis box
tuftelike	Tufte-inspired: sparse grid, no axis box, data-ink maximised
arrowplot	Adds arrow markers at data points

Apply a style by passing it as an option to the `axis` environment:

```
\begin{axis}[tuftelike, xlabel={...}, ylabel={...}]
    \addplot coordinates { ... };
\end{axis}
```

30.3 Bar Charts

Bar charts use the `ybar` plot type. Stacked bars add the `stack plots=y` option:

```
\begin{tikzpicture}
    \begin{axis}[
        ybar,
        xlabel={Category},
        ylabel={Value},
        symbolic x coords={A, B, C, D},
        xtick=data,
        bar width=20pt,
    ]
        \addplot coordinates {(A,42) (B,58) (C,35) (D,71)};
    \end{axis}
\end{tikzpicture}
```

For stacked bars:

```
\begin{axis}[
    ybar stacked,
    symbolic x coords={Q1, Q2, Q3, Q4},
    xtick=data,
]
    \addplot coordinates {(Q1,10) (Q2,15) (Q3,12) (Q4,20)};
    \addplot coordinates {(Q1,8)  (Q2,9)  (Q3,14) (Q4,11)};
    \legend{Series A, Series B}
\end{axis}
```

30.4 Contour Plots

Contour plots require `gnuplot` as a backend. Ensure gnuplot is installed and pass `--shell-escape` to the LaTeX engine:

```
\begin{tikzpicture}
    \begin{axis}[
        view={0}{90},
        colorbar,
        title={Contour of $f(x,y) = x^2 - y^2$},
    ]
        \addplot3[
            contour gnuplot={levels={-4,-2,0,2,4}},
            domain=-2:2, domain y=-2:2,
            samples=30,
        ] {x^2 - y^2};
```

```
    \end{axis}
\end{tikzpicture}
```

Custom functions replace the expression in braces. The `levels` key controls which contour lines are drawn. Layer management allows combining contour fills with surface plots by adjusting the `z buffer` order.

30.5 Date Plots

The `dateplot` library provides axis formatting for ISO 8601 dates:

```
\usepackage{pgfplots}
\usepgfplotslibrary{dateplot}

\begin{tikzpicture}
    \begin{axis}[
        date coordinates in=x,
        xticklabel={\day/\month/\year},
        xlabel={Date},
        ylabel={Revenue (\$)},
    ]
        \addplot coordinates {
            (2025-01-01, 1200)
            (2025-02-01, 1450)
            (2025-03-01, 1380)
            (2025-04-01, 1600)
        };
    \end{axis}
\end{tikzpicture}
```

Date strings must follow the `YYYY-MM-DD` format. Use `xticklabel` to customise the display format.

30.6 CSV Data Import

PGFPlots reads CSV files with `\pgfplotstableread`:

```
\pgfplotstableread{data/measurements.csv}\datatable
\begin{tikzpicture}
    \begin{axis}[
        regularplot,
        xlabel={Time (s)},
        ylabel={Displacement (mm)},
    ]
        \addplot table [x=time, y=displacement] {\datatable};
    \end{axis}
\end{tikzpicture}
```

Error bars are added with the `error bars/.cd` keys:

```
\addplot[
    table[x=time, y=displacement, y error=uncertainty],
    error bars/.cd, y dir=both, y explicit,
] {\datatable};
```

Filter rows with `\pgfplotsinvokeforeach` or the `skip coords between index` key. Column selection uses `x=colname` and `y=colname` to pick arbitrary columns from the data table.

30.7 Custom Styles

OmniLaTeX defines several PGFPlots styles and color cycles:

Table 30.2 / OmniLaTeX PGFPlots styles

Style / Cycle	Description
regularplot	Default: grid lines, axis box, mark size 2pt
plainplot	No grid, no box, minimal frame
tuftelike	Tufte aesthetic: thin grid, no box, data-ink focus
arrowplot	Arrow markers at each data point
RdYlBu3	Diverging colour cycle: red–yellow–blue
mod mark list	Alternating mark styles for multi-series plots
viridis	Perceptually uniform colour cycle

Select a colour cycle with the `cycle list name` key:

```
\begin{axis}[
    regularplot,
    cycle list name=viridis,
]
    \addplot coordinates { ... };
    \addplot coordinates { ... };
\end{axis}
```

30.8 Dual Axes

Add a secondary y-axis using `axis y line*=right` on a second `axis` environment:

```
\begin{tikzpicture}
    \begin{axis}[
        regularplot,
        axis y line*=left,
        xlabel={Time (s)},
        ylabel={Temperature (K)},
    ]
        \addplot coordinates { (0,293) (5,325) };
    \end{axis}
    \begin{axis}[
```

```
        regularplot,
        axis y line*=right,
        axis x line=none,
        ylabel={Pressure (kPa)},
        ylabel style={at={(1,0.5)}},
    ]
        \addplot coordinates { (0,101) (5,115) };
    \end{axis}
\end{tikzpicture}
```

The second axis hides its x-axis with `axis x line=none` and positions the y-label on the right with `ylabel style`.

30.9 Groupplots

The `groupplots` library arranges multiple plots in a grid with shared axes:

```
\usepgfplotslibrary{groupplots}

\begin{tikzpicture}
    \begin{groupplot}[
        group style={
            group size=2 by 1,
            horizontal sep=2cm,
        },
        regularplot,
        width=0.45\textwidth,
    ]
        \nextgroupplot[title={Plot A}]
        \addplot coordinates { (0,1) (1,3) (2,2) };

        \nextgroupplot[title={Plot B}]
        \addplot coordinates { (0,4) (1,2) (2,5) };
    \end{groupplot}
\end{tikzpicture}
```

`group size=columns by rows` sets the grid layout. `horizontal sep` and `vertical sep` control spacing between plots. Shared axis labels are set at the group level; per-plot overrides use `\nextgroupplot[...]`.

31 TikZ Engineering Diagrams

31.1 Flowcharts

OmniLaTeX provides node styles for standard flowchart shapes via `omnilatex-tikz-core.sty`:

Table 31.1 / Flowchart node styles

Style	Shape
startstop	Rounded rectangle (terminal)
io	Parallelogram (input/output)
process	Rectangle (process step)
decision	Diamond (conditional branch)

```
\begin{tikzpicture}[
    node distance=1.5cm,
    startstop/.style={...},
    process/.style={...},
    decision/.style={...},
]
    \node (start) [startstop] {Start};
    \node (proc1) [process, below=of start] {Read input};
    \node (dec1)  [decision, below=of proc1] {Valid?};
    \node (proc2) [process, below=of dec1]   {Process data};
    \node (stop)  [startstop, below=of proc2]{Stop};

    \draw[-{Stealth}] (start) -- (proc1);
    \draw[-{Stealth}] (proc1) -- (dec1);
    \draw[-{Stealth}] (dec1)  -- node[right] {Yes} (proc2);
    \draw[-{Stealth}] (dec1.west) -- ++(-2,0)
                      node[above] {No} |- (proc1);
    \draw[-{Stealth}] (proc2) -- (stop);
\end{tikzpicture}
```

Arrow styles use the `arrows.meta` library. Common tips include `Stealth`, `Latex`, `>`, and `|`.

31.2 Circuit Diagrams

The `circuits.ee.IEC` TikZ library provides standard electrical symbols:

```
\usetikzlibrary{circuits.ee.IEC}

\begin{tikzpicture}[circuit ee IEC]
```

```
    \draw (0,0) to[resistor] (3,0)
                to[battery]   (3,-2)
                to[contact]   (0,-2) -- cycle;
\end{tikzpicture}
```

Table 31.2 / IEC circuit symbols

Symbol	Component
resistor	Resistor
capacitor	Capacitor
inductor	Inductor
battery	Voltage source / battery
contact	Switch / contact
diode	Diode

Component values are specified with the `info` key: `to[resistor, info=$R_1{=}100\Omega$]`.

31.3 3D Drawings

The `tdplot` library provides three-dimensional coordinate frames:

```
\usepackage{tikz-3dplot}
\tdplotsetmaincoords{70}{110}

\begin{tikzpicture}[tdplot_main_coords]
    \draw[->] (0,0,0) -- (4,0,0) node[right] {$x$};
    \draw[->] (0,0,0) -- (0,4,0) node[above] {$y$};
    \draw[->] (0,0,0) -- (0,0,4) node[below left] {$z$};

    \draw[fill=blue!20] (0,0,0) -- (3,0,0) -- (3,2,0)
                        -- (0,2,0) -- cycle;
\end{tikzpicture}
```

Canvas planes (`xy`, `yz`, `xz`) help project 2D annotations onto 3D drawings. Flow arrows use the `-Stealth` arrow tip with the `line width` key for visibility in perspective views.

31.4 Custom Thermodynamic Shapes

OmniLaTeX defines custom TikZ shapes for thermodynamic and process engineering diagrams:

Table 31.3 / Custom thermodynamic shapes

Shape	Anchors	Description
valve	in, out	Control valve

Continued on next page

Table 31.3 / Custom thermodynamic shapes (Continued)

Shape	Anchors	Description
pump	in, out	Circulation pump
compressor	in, out	Compressor
heat exchanger	in, out	Heat exchanger
simple heat exchanger	in, out	Simple HX
radiator	top, right, exit, lower entry, upper entry, ventilation	Radiator with multiple connections
sensor		Sensor
levelindicator		Level indicator
boiler		Boiler
equalizing tank		Equalizing tank

Use these shapes in node definitions:

```
\node[valve] (v1) at (0,0) {};
\node[pump, right=2cm of v1] (p1) {};
\draw[-{Stealth}] (v1.out) -- (p1.in);
```

Shapes with `in`/`out` anchors connect naturally to pipe segments. The `radiator` shape provides multiple connection points (`top`, `right`, `exit`, `lower entry`, `upper entry`, `ventilation`) for complex piping layouts.

31.5 Pipe Networks

Pipe networks use dedicated node styles for fittings and junctions:

```
\begin{tikzpicture}
    \node[pipe] (p1) at (0,0) {};
    \node[threeway valve] (tv1) at (3,0) {};
    \node[control valve]  (cv1) at (6,0) {};
    \node[fourway valve]  (fv1) at (3,-3) {};

    \draw[-{Stealth}] (p1) -- (tv1);
    \draw[-{Stealth}] (tv1) -- (cv1);
    \draw[-{Stealth}] (tv1) -- (fv1);
\end{tikzpicture}
```

Table 31.4 / Pipe network node styles

Style	Description
pipe	Straight pipe segment
threeway valve	Three-way branching valve

Continued on next page

Table 31.4 / Pipe network node styles (Continued)

Style	Description
control valve	Two-way control valve
fourway valve	Four-way cross valve

Pipes connect via the `in`/`out` anchors on valve nodes. The `pipe` node acts as a directional segment for flow visualisation.

31.6 Control System Diagrams

Simulink-style block diagrams use rectangular nodes with named ports:

```
\begin{tikzpicture}[
    block/.style={draw, rectangle, minimum height=2.5em,
                  minimum width=4em, anchor=west},
    sum/.style={draw, circle, node distance=1.5cm},
]
    \node[sum]   (sum)  {$\Sigma$};
    \node[block, right=of sum]  (plant)  {Plant};
    \node[block, right=of plant](ctrl)   {Controller};
    \node[block, below=of ctrl] (sat)    {Saturation};

    \draw[-{Stealth}] (sum)  -- (plant);
    \draw[-{Stealth}] (plant) -- (ctrl);
    \draw[-{Stealth}] (ctrl)  -- (sat);
    \draw[-{Stealth}] (ctrl.east) -- ++(1,0) |- (sum.south)
        node[pos=0.95, left] {$-$};
\end{tikzpicture}
```

Feedback loops are drawn by routing the output signal back to the summing junction with `-- ++(dx,0) |-` paths. The saturation block models actuator limits with `min`/`max` parameters.

31.7 Annotation Arrows

OmniLaTeX provides utility commands for engineering annotations:

Table 31.5 / Annotation arrow commands

Command	Description
`\annotationarrow`	Arrow with a text label at the tip
`\wall`	Hatched wall for boundary conditions
`\flowarrow`	Directional flow arrow on a pipe or duct
`\arrowlabel`	Text label positioned along an arrow

```
\flowarrow{(0,0)}{(4,0)}{Flow direction}
\wall{(6,-1)}{(6,1)}
\annotationarrow{(2,2)}{(5,3)}{Note: pressure drop}
```

These commands simplify common annotation patterns in process and mechanical engineering diagrams, keeping the TikZ source concise and readable.

Part VII

References & Glossaries

32 Bibliography

32.1 biblatex + biber Setup

OmniLaTeX configures `biblatex` with `biber` as the backend. The default style is `ext-authoryear` with several OmniLaTeX-specific customisations:

```
\usepackage[
    backend      = biber,
    style        = ext-authoryear,
    sorting      = nyt,
    maxbibnames  = 99,
    maxcitenames = 2,
    giveninits   = true,
    uniquename   = init,
    uniquelist   = false,
    dashed       = false,
    isbn         = false,
    url          = false,
    doi          = false,
    eprint       = false,
    backref      = true,
    backrefstyle = none,
]{biblatex}
```

Key OmniLaTeX customisations:

- **introcite=label** – prints a numeric label at the first citation in each chapter.
- **Family-given name format** – author names are rendered as Family Given.
- **Semicolon delimiter** – multiple authors in a single citation are separated by semicolons.
- **Small caps family names** – family names appear in small capitals for visual distinction.

32.2 Adding Bibliography

Declare bibliography resource files with `\addbibresource`:

```
\addbibresource{references.bib}
\addbibresource{supplementary.bib}
```

Multiple `.bib` files can be loaded. All entries from all files are available for citation. The `.bib` files are parsed by `biber` at compile time.
Print the bibliography at the end of the document:

```
\printbibliography
```

32.3 Citation Commands

OmniLaTeX supports the full set of `biblatex` citation commands:

Table 32.1 / Citation commands

Command	Output
`\{}cite{key}`	Numeric label: [1]
`\{}textcite{key}`	Author (year) inline
`\{}parencite{key}`	Parenthesised: (Author, year)
`\{}footcite{key}`	Footnote citation
`\{}fullcite{key}`	Full bibliographic entry inline
`\{}autocite{key}`	Context-sensitive (parencite or footcite)
`\{}citeauthor{key}`	Author name only
`\{}citeyear{key}`	Year only
`\{}parencites{key1}{text1}`	Multiple parenthesised citations
`\{}textcites{key1}{text1}`	Multiple textual citations
`\{}cites{key1}{text1}`	Multiple citations with notes
`\{}footcites{key1}{text1}`	Multiple footnote citations

32.4 Citation Groups

Group multiple citations with per-citation notes using the starred variants:

```
\parencites{key1}[see][p.~42]{key2}[cf.]{key3}

\cites{key1}{see also \textit{key2}}{key3}

\textcites{key1}{as discussed in key2}{key3}
```

Each key may be followed by optional pre- and post-note arguments in square brackets: `\parencites[pre1][post1]{key1}[pre2][post2]{key2}`.

32.5 Per-Chapter Bibliography

Two mechanisms exist for splitting the bibliography by chapter:

`refsection` – Creates independent bibliography sections. Each `refsection` environment resets the citation counter and produces its own bibliography. Use `\printbibliography[heading=subbibliography]` inside each section.

```
\begin{refsection}
    \chapter{Thermodynamics}
    % ... citations ...
    \printbibliography[heading=subbibliography]
\end{refsection}
```

refsegment – Segments share a single bibliography but group entries by segment. Use `\printbibliography[segment=0]|` to list only the current segment's entries.

`refsegment` is simpler to set up; `refsection` provides stronger isolation between chapters.

32.6 Back References

The `backref` option adds page references to each bibliography entry showing where it was cited:

```
\usepackage[backref=true, backrefstyle=none]{biblatex}
```

Table 32.2 / Back reference styles

Style	Description
none	Cited on pages 1, 3, 7
chapter	Cited on chapters 1, 2
section	Cited on sections 1.1, 2.3

32.7 Further Reading Section

A "Further Reading" section lists uncited references using `category` filtering:

```
\addbibresource{cited.bib}
\addbibresource{uncited.bib}

% In cited.bib:  @book{key1, ...}
% In uncited.bib: @book{key2, category={notcited}, ...}

\printbibliography[title={References}, notcategory=notcited]
\printbibliography[title={Further Reading}, category=notcited]
```

Alternatively, use `\nocite{*}` to include all entries, then split by category. Entries without a `category` field default to `cited`.

33 Citation Styles

33.1 Style Switching

OmniLaTeX provides the `\citationstyle` command to switch bibliography styles from the document preamble:

```
\documentclass{omnilatex}
\citationstyle{ieee}

\begin{document}
...
\end{document}
```

The command accepts one of the predefined style names (see ??). It must be called in the preamble, before `\begin{document}`. Internally, `\citationstyle` reconfigures `biblatex` options and reloads the appropriate style file.

33.2 Available Styles

OmniLaTeX ships nine predefined citation styles:

Table 33.1 / Available citation styles

Style	Use Case	Full Name
ieee	Engineering	IEEE
acm	Computer Science	ACM
apa	Social Sciences	APA 7th
chicago	Humanities	Chicago
nature	Science journals	Nature
science	General science	Science
harvard	UK academia	Harvard
vancouver	Medical	Vancouver
mla	Liberal arts	MLA

Each style maps to a `biblatex` style package:

Table 33.2 / Style to biblatex mapping

\{}citationstyle	biblatex style
ieee	style=ieee
acm	style=acm
apa	style=apa
chicago	style=chicago-authordate
nature	style=nature
science	style=science
harvard	style=ext-authoryear
vancouver	style=vancouver
mla	style=mla

33.3 Style Comparison

The same three-entry bibliography renders differently depending on the chosen style. Below is a description of how each major style formats citations and the bibliography list.
Consider these entries:

```
@article{smith2023,
    author  = {Smith, John and Doe, Jane},
    title   = {A Study of Thermal Systems},
    journal = {Journal of Thermodynamics},
    year    = {2023},
    volume  = {15},
    pages   = {1--10},
}
@book{lee2021,
    author    = {Lee, Alice},
    title     = {Fundamentals of Heat Transfer},
    publisher = {Academic Press},
    year      = {2021},
}
@inproceedings{wang2022,
    author    = {Wang, Bob and Chen, Carol},
    title     = {Efficiency in Cogeneration Plants},
    booktitle = {Proceedings of ECOS 2022},
    year      = {2022},
    pages     = {112--120},
}
```

IEEE: Numeric citations [1], [2]. Bibliography lists entries in order of first citation. Author names in initials-first format.
APA: Parenthetical (Smith & Doe, 2023) or narrative Smith and Doe (2023). Bibliography uses hanging indent with full first names.
Chicago (author-date): Similar to APA but with different punctuation and ordering rules. Uses em-dash for multiple authors.

Harvard: (Smith and Doe, 2023). Similar to author-date but with specific capitalisation and abbreviation rules common in UK academia.
Vancouver: Numeric superscript[1] or bracketed [1]. Bibliography lists all authors up to six, then *et al.*
Nature: Numeric superscript citations. Bibliography lists authors in smaller font with abbreviated journal names.
MLA: (Smith and Doe 223). Page numbers preferred. Bibliography uses *Works Cited* heading with author-page format.
ACM: Numeric [1] with optional author-name citation. Bibliography follows ACM reference format with DOI links.
Science: Numeric superscript[1]. Bibliography is numbered and uses abbreviated journal titles with year in parentheses.

33.4 Default Style

When no `\citationstyle` command is issued, OmniLaTeX defaults to `ext-authoryear`. This style extends the standard `authoryear` with additional features:

- **Extended name handling** – `giveninits=true` for consistent initial-based name formatting.
- **Dashed entries** – replaced by title in repeated-author runs (`dashed=false` in OmniLaTeX).
- **introcite** – numeric label at first citation per chapter.
- **backref** – page references in the bibliography.

To override OmniLaTeX's default while keeping the extended features, set the style to `harvard`, which maps to `ext-authoryear` with UK academic conventions.

34 Glossaries

OmniLaTeX uses the `glossaries-extra` package to provide glossary, acronym, symbol, and index functionality. All glossary infrastructure is loaded by `omnilatex-glossary.sty`; you do *not* need to load `glossaries` or `glossaries-extra` yourself.

34.1 glossaries-extra Setup

The package is initialised with the `nomain` option so that `omnilatex-glossary.sty` retains full control over which glossary types are created. The following glossary types are registered:

- `acronyms` – abbreviations and acronyms
- `symbols` – mathematical and physical symbols
- `index` – back-of-book index entries
- `numbers` – physical constants and named numbers

Each type receives an appropriate sorting and labelling strategy via `glossaries-extra`'s `sort` and `label` keys.

34.2 Custom Glossary Fields

OmniLaTeX extends the default entry keys with several custom fields that are recognised by the symbol and constant glossary styles.

Table 34.1: Custom glossary fields added by OmniLaTeX.

Field	Type	Purpose
unit	text	SI unit associated with a symbol
specific	text	Specific context or domain qualifier
firstname	text	Person's first name (for named entries)
value	text	Numerical value of a constant
international-symbol	text	International symbol variant
alternative-symbol	text	Alternative or legacy symbol

These fields are accessed inside glossary styles with `\glsuseri`, `\glsuserii`, etc.

34.3 Shortcut Commands

OmniLaTeX provides shorthand macros that create and reference glossary entries in a single call. Each command targets a specific glossary type.

Table 34.2: Glossary shortcut commands.

Command	Glossary Type	Example
`\sym{Key}`	symbol	`\sym{rho}`
`\sub{Key}`	subscript	`\sub{max}`
`\name{Key}`	name	`\name{Einstein}`
`\cons{Key}`	constant	`\cons{g}`
`\idx{Key}`	index	`\idx{entropy}`
`\??Key}`	abbreviation	`\??CPU}`
`\num{Key}`	number	`\num{pi}`

The first invocation of each command creates the entry (with sensible defaults); subsequent calls expand to the formatted representation.

34.4 Symbol Glossary

The `symbols` glossary uses the `symbunitlong` style, which displays each symbol together with its unit in a long-table layout. Entries that carry a `specific` field are annotated with a footnote-style marker so the reader can distinguish homographic symbols (e.g. c for speed of light vs. c for specific heat capacity).

34.5 Constants Glossary

The `numbers` glossary employs the `numberlong` style, which prints each constant's name, symbol, and numerical value side by side. The `value` field is rendered in a fixed-width column so that mantissas align vertically.

34.6 Printing

Glossaries are printed with `\printunsrtglossary`. Two calls are typically placed in the back matter:

```
\printunsrtglossary[type=symbols, style=symbunitlong,
                    title={Nomenclature}]
\printunsrtglossary[type=acronyms,
                    style=\OmniLaTeXAcronymStyle]
```

`\OmniLaTeXAcronymStyle` is a custom style defined in `omnilatex-glossary.sty` that formats acronyms with their long form on first use and the short form thereafter.

34.7 Index

The back-of-book index is generated with

```
\printunsrtindex[style=bookindex]
```

Index entries are created via `\idx{key}`, which delegates to `\gls` with the `index` glossary type.

35 Cross References

OmniLaTeX configures `cleveref` and `hyperref` to provide automatic, contextual cross-references throughout the document. Every reference command produces a clickable hyperlink in the output PDF, and `cleveref` automatically determines the correct label prefix (“Figure”, “Table”, “Chapter”, …) from the counter associated with each label.

35.1 Basic Reference Commands

LaTeX provides three core reference commands. Each resolves to the number assigned to the labelled object:

Table 35.1: Core reference commands.

Command	Output	Use case
`\ref{label}`	Number only (e.g. 3)	Generic fallback
`\eqref{label}`	Parenthesised (e.g. (3.7))	Equations
`\pageref{label}`	Page number	Locating floats
`\autopageref{label}`	“on page 42”	Inline prose

The `\autopageref` command (provided by `varioref`, which OmniLaTeX loads automatically) formats page references as “on page 42” or “on the preceding page” depending on proximity, producing natural prose.

35.2 cleveref

The `cleveref` package is loaded automatically. Use `\cref{label}` for lower-case and `\Cref{label}` for capitalised references:

```
As shown in \cref{fig:x}, the results align with
\Cref{tab:results} and the derivation in \cref{eq:mass}.
```

```
\documentclass[doctype=article]{../../omnilatex}
\begin{document}
\section{Introduction}
\label{sec:intro}

See \cref{sec:intro} for the background.
\end{document}
```

The reference string is determined by the counter associated with the label, so no manual type name is needed. Pluralisation is also automatic: `\cref{fig:x,fig:y}` produces “figures 1 and 2”.

35.3 Custom crefnames

Beyond the standard counters, OmniLaTeX registers custom `\crefname` definitions for environment types that ship with the template, such as `example` and `code`/`listing`:

```
\crefname{code}{listing}{listings}
\crefname{example}{example}{examples}
```

This means `\cref{lst:hello}` resolves to "listing 1" without any additional configuration. To add custom names for your own environments, use `\crefname` in the preamble:

```
\crefname{algorithm}{algorithm}{algorithms}
\Crefname{algorithm}{Algorithm}{Algorithms}
```

35.4 Label Placement Rules

Labels must be placed *after* the counter incrementing command. The following table shows the correct placement for each object type:

Table 35.2: Label placement for common objects.

Object	Label position
Chapter / Section	Immediately after `\chapter` or `\section`
Figure / Table	Inside the float, after `\caption`
Equation	Inside `equation` / `align`, any position
Theorem	After `\begin{theorem}`
Enumerated item	After `\item`

Incorrect placement produces wrong numbers. For example, placing a label before `\caption` inside a figure environment causes the label to resolve to the previous figure's number:

```
% WRONG -- label resolves to previous figure number
\begin{figure}
  \label{fig:wrong}
  \caption{Description}
\end{figure}

% CORRECT -- label follows caption
\begin{figure}
  \caption{Description}
  \label{fig:right}
\end{figure}
```

35.5 Cross-Referencing Floats

Figures and tables are numbered per chapter automatically. When a float spans across pages, `\ContinuedFloat` ensures the second caption continues the numbering:

```
\begin{figure}
  \caption{First part of a wide diagram}
  \label{fig:wide}
```

```
  \includegraphics[width=\textwidth]{diagram-part1}
\end{figure}

\begin{figure}
  \ContinuedFloat
  \caption{Continued from previous page}
  \includegraphics[width=\textwidth]{diagram-part2}
\end{figure}
```

References to both parts of the continued float resolve to the same number via `\cref{fig:wide}`.

35.6 Cross-Referencing Equations

By default `cleveref` formats equation references as "eq. (1.1)". OmniLaTeX overrides this with

```
\creflabelformat{equation}{#2#1#3}
```

which removes the redundant "eq." prefix so that `\cref{eq:mass}` produces "(1.1)" – consistent with the convention of most physics and engineering journals. Use `\eqref` when you need an explicit "Equation (1.1)" prefix:

```
\begin{equation}
  E = mc^2
  \label{eq:mass}
\end{equation}

Referencing with \cref{eq:mass} yields just the number in parentheses.
Using \eqref{eq:mass} yields ``Equation (1)'' with the full prefix.
```

To restore the default `cleveref` behaviour, redefine `\crefformat{equation}` after loading the template:

```
\crefformat{equation}{eq.~(#2#1#3)}
\Crefformat{equation}{Equation~(#2#1#3)}
```

35.7 Cross-Referencing Sections and Theorems

Section and theorem references use the chapter.number scheme:

```
\chapter{Methods}
\label{ch:methods}

\section{Experimental Setup}
\label{sec:setup}

\begin{theorem}[Cauchy--Schwarz]
  \label{thm:cs}
  |\langle u, v \rangle|^2
    \leq \langle u, u \rangle \cdot \langle v, v \rangle
\end{theorem}

In \cref{sec:setup} we describe the apparatus.
\Cref{thm:cs} establishes the inequality bound.
```

35.8 hyperref and nameref

`hyperref` is configured with PDF metadata (title, author, subject) taken from the preamble keys. All reference commands produce clickable links that navigate to the target in the PDF viewer.
The `nameref` package (loaded by `hyperref`) provides `\nameref{label}`, which resolves to the *title* of the labelled section rather than its number:

```
\nameref{sec:setup}
% → "Experimental Setup" (the section title, not "2.1")
```

Chapter-level bookmarks in the PDF outline can be typeset in bold when the `boldPDFchapters` template option is enabled:

```
\documentclass[doctype=book, boldPDFchapters]{../../omnilatex}
```

Internally this uses:

```
\ifboldPDFchapters
    \pdfstringdefaliasprefix{<b>}{</b>}
\fi
```

35.9 Common Pitfalls

Duplicate labels Using the same label name twice causes `cleveref` and `hyperref` to emit warnings and produce incorrect references. Use a consistent naming scheme: `fig:`, `tab:`, `eq:`, `sec:`, `ch:`.

Forward references On the first compilation pass, forward references appear as "??". Run `latexmk` again (the default `dev` mode performs enough passes to resolve all references). In `ultra` mode, forward references may remain unresolved.

Labels outside floats A `\label` placed outside a `figure` or `table` environment but intended for that float will pick up the enclosing section counter instead. Always place labels inside the float, after `\caption`.

Hyperlinks in captions Avoid using `\label` in captions that contain hyperlinks. The link target may shift after additional passes. Place the label on a separate line after the caption.

Part VIII

Code & Listings

36 Code Listings

OmniLaTeX provides syntax-highlighted code listings via the `minted` package, configured in `omnilatex-listings.sty`.

36.1 minted Setup

The default style is `manni` with a left-line frame, line numbers on every line, and caching enabled for fast recompilation:

```
\usemintedstyle{manni}
\setminted{
    frame      = lines,
    framesep   = 2mm,
    linenos,
    numbersep  = 8pt,
    cache      = true,
}
```

The cache directory is derived from the job name so that parallel builds do not collide.

36.2 Inline Code

Inline code fragments use `\mintinline`:

```
The function \mintinline{python}{print("hello")} writes to stdout.
```

The language is specified as the first mandatory argument.

36.3 Block Listings

Full listings are placed inside the `minted` environment:

```
\begin{minted}{python}
def greet(name):
    print(f"Hello, {name}!")
\end{minted}
```

Line numbers and the frame are applied automatically through the global `\setminted` defaults.

36.4 Long Listings

For listings that span multiple pages, wrap the environment in a `longlisting` float:

```
\begin{longlisting}[H]
    \inputminted{lua}{scripts/init.lua}
    \caption{Initialisation script}
    \label{lst:init}
\end{longlisting}
```

`longlisting` is a float that may break across pages while preserving the listing counter and caption numbering.

36.5 Line Highlighting

Specific lines can be highlighted with the `highlightlines` option:

```
\begin{minted}[highlightlines={3,7-9}]{python}
x = 42          # highlighted
y = x + 1
z = y * 2       # highlighted
\end{minted}
```

OmniLaTeX also defines placeholder commands for annotating template code:

Table 36.1: Placeholder highlight commands.

Command	Colour
`\{text}`	red
`\{text}`	blue
`\{text}`	green
`\{text}`	orange

These are intended for documentation that shows replaceable portions of code.

36.6 Language Support

`minted` leverages Pygments, so any Pygments lexer is available. Languages commonly used with OmniLaTeX include:

- Python
- MATLAB
- Lua
- Modelica
- C / C++
- bash
- Dockerfile
- JSON
- SQL
- YAML

Add the `--shell-escape` flag when invoking the compiler so that Pygments can be called.

36.7 Simulink Icons

`omnilatex-graphics.sty` provides `\mtlbsmlkicon{name}` for embedding Simulink block icons inline:

`\mtlbsmlkicon{Gain}`

The icon is rendered at a fixed size suitable for inline use.

36.8 Accessible Line Numbers

For PDF/UA-1 compliance, line-number digits are hidden from screen readers via

`\emptyaccsupp`

This macro is applied automatically inside every `minted` environment when the accessibility mode is active.

37 Lua Scripting

OmniLaTeX embeds LuaTeX scripting to provide dynamic metadata, conditional compilation, and build-time automation.

37.1 Lua in LaTeX

Lua code can be executed inline with `\directlua` or in a dedicated environment:

```
\directlua{tex.sprint("2 + 2 = ", 2 + 2)}

\begin{lua}
    local function greeting(name)
        return "Hello, " .. name .. "!"
    end
    tex.sprint(greeting("World"))
\end{lua}
```

`\luasetup` is provided for one-time initialisation blocks that run before the document body.

37.2 Git Metadata

`omnilatex-utils.sty` queries the working tree at compile time and exposes the following macros:

Table 37.1: Git metadata macros.

Macro	Expansion
`\GitRefName`	Current branch or tag name
`\GitShortSHA`	Abbreviated commit hash (8 hex digits)
`\GitLongSHA`	Full 40-character SHA
`\BuildDate`	ISO-8601 build timestamp
`\BuildMode`	String: `dev`, `prod`, or `ultra`

These macros expand to empty strings when Git metadata cannot be resolved (e.g. in CI environments without a `.git` directory).

37.3 Conditional Compilation

The build mode is determined by the `BUILD_MODE` environment variable. OmniLaTeX defines three boolean flags:

```
\newif\ifomnilatex@dev
\newif\ifomnilatex@prod
\newif\ifomnilatex@ultra
```

Detection is performed at load time and exposed through user-facing switches:

```
\OmniDev   % active when BUILD_MODE=dev
\OmniProd  % active when BUILD_MODE=prod
\OmniUltra % active when BUILD_MODE=ultra
```

Use these in the document body to include or exclude content:

```
\ifomnilatex@dev
    \typeout{[dev] Draft watermark enabled.}
\fi
```

37.4 Custom Lua Commands

Arbitrary Lua functions can be wrapped as LaTeX commands with the `\directlua\stringdef` pattern:

```
\directlua\stringdef\mycmd#1{%
    tex.sprint(string.upper(#1))%
}
```

`\mycmd{hello}` then expands to `HELLO`.

37.5 Example Walkthrough

A complete demonstration of Lua scripting, including metadata queries, conditional blocks, and custom commands, is available in the `examples/lua-showcase/` directory. Compile that sub-project with

```
latexmk -lualatex examples/lua-showcase/main.tex
```

Part IX

Advanced Features

38 Color Themes

38.1 Overview

The `omnilatex-themes.sty` package provides a flexible theming system built on top of `xcolor` and `tcolorbox`. Seven curated color palettes are included, each defining a complete set of semantic color slots used throughout the template–from headings and links to block environments and footers.
All palettes define the same set of named color slots, so switching themes never breaks document markup.

38.2 Available Palettes

Palette	Description
`default`	Standard blue and gray tones
`midnight`	Dark blue and charcoal
`forest`	Green tones inspired by nature
`rose`	Pink and magenta hues
`monochrome`	Grayscale for a clean, neutral look
`monochrome-dark`	Dark grayscale for dark-background documents
`sepia`	Warm brown tones for a vintage feel

Table 38.1: Available color palettes.

38.3 Loading a Theme

Load a theme with the `\usetheme` command in the preamble:

```
\documentclass{omnilatex}
\usetheme{midnight}
\begin{document}
...
\end{document}
```

Themes may also be switched mid-document:

```
\usetheme{rose}   % Switch to rose palette from here onward
```

This is useful for appendices or supplementary material that should use a different visual identity.

38.4 Dark / Light Mode

Two convenience commands toggle between dark and light variants of the active palette:

```
\darkmode           % Switch to dark variant
\lightmode          % Switch to light variant
```

Conditional theming is available through `\ifthememode`:

```
\ifthememode{dark}{Dark background}{Light background}
```

The macro expands to the first argument when the current mode is dark and to the second argument when it is light.

38.5 Per-Slot Overrides

Override individual color slots without changing the entire palette:

```
\setthemecolor{primary}{NavyBlue}
\setthemecolor{accent}{Orange}
```

Common slots include `primary`, `secondary`, `accent`, `heading`, `link`, `footer`, and `blockbg`. Consult `omnilatex-themes.sty` for the complete list.

38.6 tcolorbox Integration

All themed block environments (see Chapter **??**) automatically inherit the active palette colors. The following boxes are styled by the theme system:

- `presentationblock` – uses the primary slot
- `alertblock` – uses the alert/error slot
- `exampleblock` – uses the success slot
- `noteblock` – uses the secondary slot
- `definitionblock` – uses the heading slot

Switching palettes or overriding slots propagates immediately to all tcolorbox-based environments.

39 Boxes & Environments

39.1 Example Box

The `example` environment provides a numbered, captioned box for worked examples:

```
\begin{example}[A Simple Derivation]
  \caption{Differentiating $x^2$}
  \begin{align*}
    \frac{d}{dx}\,x^2 &= 2x
  \end{align*}
\end{example}
```

Key properties:

- Auto-incrementing counter (Example 1, Example 2, …).
- Entries appear in the List of Examples (LoE) when applicable.
- Breakable across pages via `tcolorbox`'s `breakable` library.
- Supports `\caption` for a descriptive label.

39.2 Key Concept Box

Handout-focused documents can use `keyconceptbox` to highlight essential ideas:

```
\begin{keyconceptbox}
  \keyconcept{The Central Limit Theorem}
  The mean of a sufficiently large sample of independent random variables
  is approximately normally distributed.
\end{keyconceptbox}
```

39.3 Callout Box

White papers and reports benefit from the `\callout` macro:

```
\callout{Important}{This section assumes familiarity with basic
probability theory.}
```

The callout renders as a `tcolorbox` with the active theme's accent color.

39.4 Presentation Blocks

Five themed block environments are available for presentations and slides:

```
\begin{presentationblock}{Main Idea}
  Content goes here.
\end{presentationblock}
```

```
\begin{alertblock}{Warning}
  Proceed with caution.
\end{alertblock}

\begin{exampleblock}{Illustration}
  An example goes here.
\end{exampleblock}

\begin{noteblock}{Remark}
  Additional context.
\end{noteblock}

\begin{definitionblock}{Definition}
  A \emph{widget} is defined as \ldots
\end{definitionblock}
```

Each block is styled by the active color theme (see Chapter ??).

39.5 Git Verification Box

The `\GitVerificationBox` command renders a colophon-style box showing the current Git commit SHA and verification instructions:

```
\GitVerificationBox
```

Output includes the full SHA, short SHA, and a shell command for the reader to verify the build. See Chapter ?? for full details.

39.6 Todo Notes

Conditional TODO notes are available via the `todonotes` option:

```
\todo{Revise this section before final submission.}
```

Notes are only rendered when the `todonotes` class option is enabled. They are stripped automatically in production builds.

39.7 Censor Box

The `\censorbox` command creates a censored region for larger redactions:

```
\censorbox{%
  This entire paragraph contains sensitive information
  that should not appear in the published version.
}
```

Like `\todo`, censor boxes are conditional on the `censoring` class option. See Chapter ?? for details.

40 Censoring

40.1 Censoring Mode

Activate censoring with the `censoring=true` class option:

```
\documentclass[doctype=report, censoring=true]{../../omnilatex}
```

This loads the `censor` package and enables all censoring commands described below. When `censoring=false` (the default), all censoring commands are no-ops and produce no output. This design allows a single source file to produce both redacted and unredacted PDFs simply by toggling the class option.

40.2 \censor{text}

The `\censor` command applies inline black-bar redaction:

```
The secret project codename is \censor{Operation Sunflower}.
```

The redacted text is replaced with a black rectangle matching the text width. Multiple words can be censored in a single call:

```
The meeting took place at \censor{Building 7, Room 302}
on \censor{15 March 2025}.
```

Use `\StopCensoring` to disable censoring from a point onward:

```
\censor{Classified information}
\StopCensoring
This text is no longer censored.
```

Conversely, `\StartCensoring` re-enables censoring after it has been stopped:

```
\StopCensoring
Visible text here.
\StartCensoring
\censor{This will be redacted again.}
```

40.3 \blackout{text}

The `\blackout` command renders a black rectangle without revealing the length of the original text. Unlike `\censor`, which matches the text width, `\blackout` produces a fixed-height bar:

```
Subject: \blackout{entire sentence removed for legal reasons}
```

This is useful when the redacted text length itself could reveal sensitive information (e.g. the number of characters in a password).

40.4 \xblackout{text}

The `\xblackout` command renders each character as a separate black rectangle, similar to fully redacted government documents:

```
Agent \xblackout{Jane Doe} reported to \xblackout{Langley}.
```

Each letter is replaced by an individual black box, making it clear that text has been removed while preserving word spacing.

40.5 Censoring in Math Mode

Censoring can be applied inside mathematical expressions. The `\censor` command works in both inline and display math:

```
The encryption key is $k = \censor{4829A7F3}$.

\begin{equation}
  P(x) = \prod_{i=1}^{n} \censor{x_i + \alpha}
\end{equation}
```

For full-line math redaction, wrap the entire equation in `\censorbox` (see **??**).

40.6 Censoring in Code Listings

Code listings that contain sensitive values can be redacted. Because `minted` environments are processed by Pygments externally, censoring commands must be applied to the rendered output. The recommended approach is to censor the text within the listing source:

```
api_key = "\censor{sk-abc123def456}"
endpoint = "https://api.example.com"
token = get_token(user="\censor{admin}", pass="\censor{s3cret}")
```

When `censoring=true`, the sensitive strings are replaced with black bars. When `censoring=false`, the full values are shown. This approach keeps the code structurally valid for syntax highlighting while redacting the sensitive portions.

40.7 \censorbox{...}

For larger redactions that span multiple lines, use `\censorbox`:

```
\censorbox{%
  The following dataset contains personally identifiable
  information (PII) and must not be published:
  \begin{itemize}
    \item Names, addresses, phone numbers
    \item Medical records
    \item Financial account details
  \end{itemize}
}
```

The box renders as a solid black rectangle covering the entire content area. Use `\censorbox` for paragraphs, tables, or any block-level content that must be completely obscured.

40.8 \todo{text}

The `\todo` command inserts a marginal note for internal review:

```
\todo{Verify these figures with the experimental team.}
\todo[inline]{This paragraph needs revision for clarity.}
```

TODO notes are rendered only when the `todonotes` class option is enabled. In production builds they are automatically suppressed, ensuring no draft annotations leak into the final PDF.

40.9 Custom Censor Marks

The `censor` package supports custom mark characters through the `\censorrule` command. To change the censoring colour or symbol:

```
% Use a dark grey instead of pure black
\renewcommand{\censorrule}{\textcolor{darkgray}{\rule{...}{...}}}

% Use X marks instead of bars
\setcounter{censorcounter}{0}
\censorrule{X}
```

Custom marks must be defined before the first censoring command is encountered in the document.

40.10 Package Options

The `censor` package accepts several options that can be passed through OmniLaTeX's class option interface:

Table 40.1: censor package options.

Option	Effect
`counttotal`	Count total number of censored items
`split`	Allow `\censor` to break across lines
`explode`	Explode censored text into individual characters
`block`	Block censor: solid rectangle
`nuke`	Remove censored text entirely (no mark)

40.11 Compliance Use Cases

Censoring is designed for two primary workflows:

Thesis Submission Advisors and reviewers often insert comments, suggestions, and revisions during the drafting process. Before final submission, enable `censoring=true` to redact all reviewer annotations while preserving the document structure.

Sensitive Data Redaction Research papers handling PII, proprietary data, or classified material can use censoring commands to prepare public-facing versions without maintaining separate source files.

Table 40.2: Censoring commands for compliance frameworks.

Framework	Command	Requirement
GDPR	`\censor`	Mask personal identifiers
HIPAA	`\censorbox`	Redact entire medical records
ITAR	`\xblackout`	Character-level redaction
FOUO	`\blackout`	Mask sensitive paragraphs

For regulatory compliance, the following table maps censoring commands to common requirements: Both workflows benefit from the fact that censoring is a compile-time feature–no manual editing of source content is required. The same source tree produces both the internal review copy and the public release copy.

41 PDF Accessibility

41.1 PDF/UA-1 Compliance

The template activates PDF/UA-1 tagging through `omnilatex-accessibility.sty`, which loads and configures the `tagpdf` package. Validation is performed with:

```
\validatstructure
```

This command checks the tag tree at the end of compilation and reports structural issues such as missing headings or incorrectly nested elements.

41.2 Alt Text for Figures

Every figure must have alternative text for screen readers. Place `\alttext` immediately before the `figure` environment:

```
\alttext{Scatter plot showing correlation between temperature
and ice cream sales from 2019 to 2023}
\begin{figure}
  \centering
  \includegraphics{figures/scatter-plot}
  \caption{Temperature vs.\ Ice Cream Sales}
\end{figure}
```

For TikZ diagrams, use `\tikzalttext` inside the `tikzpicture` environment:

```
\begin{tikzpicture}
  \tikzalttext{Block diagram of the compiler pipeline}
  \node {Lexer} -- node {tokens} {Parser} -- node {AST} {CodeGen};
\end{tikzpicture}
```

41.3 Accessible Links

The `\accessiblelink` macro attaches a screen-reader description to hyperlinks:

```
\accessiblelink{LaTeX Project}{https://www.latex-project.org/}%
  {Visit the official LaTeX Project website}
```

The first argument is the visible link text, the second is the URL, and the third provides context for assistive technology.

41.4 Heading Validation

`\validatstructure` verifies that the document heading hierarchy is correct:

- No skipped levels (e.g. `\section` directly followed by `\subsubsection`).
- Every `\chapter`/`\section` has a corresponding PDF bookmark and tag.
- The logical reading order matches the visual order.

41.5 Contrast Checking

`\checkcontrast` verifies that foreground and background colors meet WCAG 2.1 contrast requirements:

```
\checkcontrast{NavyBlue}{white}
```

A warning is issued at compile time if the contrast ratio falls below 4.5:1 for normal text or 3:1 for large text.

41.6 Reading Order

For complex layouts, explicitly set the PDF reading order:

```
\readingorder{n}   % Linear reading order
```

This ensures that multi-column layouts and sidebars are read in the correct sequence by assistive technology.

41.7 Language Tags

Multi-language documents should annotate text with language tags:

```
\langtag{french}{Ceci est un exemple en fran\c{c}ais.}
```

Screen readers use these tags to switch pronunciation rules appropriately.

41.8 Example Walkthrough

A complete accessibility example is provided in `examples/accessibility-test/`. Build it with:

```
BUILD_MODE=prod python build.py
```

The resulting PDF includes full PDF/UA-1 tags, alt text on all figures, and accessible hyperlink annotations.

42 Advanced Presentations

This chapter expands on the `presentation` doctype overview from Chapter ?? with advanced customization patterns, internal details, and practical techniques.

42.1 Design Philosophy

OmniLaTeX presentations use KOMA-Script (`scrartcl`) rather than the `beamer` class. This gives several advantages:

- Consistent font, colour, and bibliography handling with all other doctypes.
- No separate theme system to learn – institution configs apply directly.
- Full access to the entire OmniLaTeX macro library, including `minted` code listings, glossaries, and `tcolorbox` environments.
- 16:10 aspect ratio (254 mm × 190.5 mm) via direct `\paperwidth` assignment, avoiding geometry conflicts.

The core engine lives in `lib/layout/omnilatex-presentation.sty` (v1.13.0).

42.2 Colour Customization

The header, footer, and progress bar use two colours: `universityprimary` and `universitysecondary`. When no institution config is loaded, the module defines fallback values:

```
% Fallback defaults (RGB 0,63,114 and RGB 0,113,188)
% Override in document-settings.sty:
\definecolor{universityprimary}{RGB}{102,0,153}
\definecolor{universitysecondary}{RGB}{153,51,204}
```

Institution configs set these colours automatically. For example, TUHH defines a dark navy primary and a teal secondary. The presentation module checks with `\@ifundefined{color@universityprimary}` before applying its fallback, so any definition in the preamble or an institution config takes precedence.

42.3 Title Page Internals

The `\maketitle` override fills the upper 35% of the page with `universityprimary`, adds a 2% accent stripe of `universitysecondary`, and centres the title in 28 pt bold white. The author appears below in large white, and the institute and date are placed beneath the coloured region.

```
\title{My Research Talk}
\author{Jane Doe}
% Institute is not a standard KOMA command; define it manually:
\makeatletter
\gdef\@institute{Department of Computer Science, University of Examples}
\makeatother
```

```
\date{June 2026}

\begin{document}
\maketitle
% The title page is followed by a \clearpage
\end{document}
```

The page style is set to `empty` for the title page, so no header or footer decorations appear.

42.4 Frame Environment Details

Each `presentationframe` performs these steps in order:

1. `\clearpage` – flush previous content.
2. `\presentationframemark` – increment `framecounter`.
3. `\presentationHeader` – draw the TikZ header bar with title and author.
4. User content is rendered.
5. `\presentationFooter` – draw the footer rule, short title, and frame counter.
6. `\presentationProgressBar` – draw the 2 pt progress bar.
7. `\clearpage` – finalize the slide.

The environment accepts an optional argument (currently unused but reserved for future per-frame configuration):

```
\begin{presentationframe}
    \slidetitle{Slide Title}
    Content goes here.
\end{presentationframe}
```

42.5 Progress Bar Mechanics

The progress bar requires two-pass compilation to know the total frame count. Place `\presentationcountframes` at the end of the document body:

```
\begin{document}
\maketitle
\begin{presentationframe}
    \slidetitle{First}
    Content.
\end{presentationframe}
\begin{presentationframe}
    \slidetitle{Second}
    Content.
\end{presentationframe}
\presentationcountframes  % <-- snapshots total, resets counter
\end{document}
```

Internally, this command copies `framecounter` into `totalframes` and resets `framecounter` to zero. On the next compilation pass, each frame increments `framecounter` again, and the progress bar fill width is computed as:

```
\paperwidth * \ratio{framecounter}{totalframes}
```

If `\presentationcountframes` is omitted, the progress bar shows 0% fill on every slide (since `totalframes` remains zero).

42.6 Disabling UI Elements

Two toggle pairs control optional slide decorations:

```
% Hide navigation symbols (default: off)
\presentationnavsymbolsoff

% Show navigation symbols
\presentationnavsymbolson

% Hide progress bar (default: on)
\presentationprogressoff

% Show progress bar
\presentationprogresson
```

Place these commands in the preamble after `\begin{document}` or in `config/document-settings.sty`.

42.7 Block Customization via tcolorbox

All five block environments are standard `tcolorbox` definitions with the `enhanced` skin. The optional first argument passes through to tcolorbox, allowing per-block overrides:

```
% Custom width and shadow for one block
\begin{presentationblock}[width=0.6\textwidth, drop shadow]{Tip}
    This block is narrower and has a drop shadow.
\end{presentationblock}

% Override the border colour of a definition block
\begin{definitionblock}[borderline west={3pt}{0pt}{purple}]{Lemma}
    A partial order is reflexive, antisymmetric, and transitive.
\end{definitionblock}
```

The five environments and their default styles:

Table 42.1: Block environment defaults.

Environment	Background	Border	Padding
`presentationblock`	White	Primary (2.5 pt)	6 pt/4 pt
`alertblock`	Red!5	Red!70!black	6 pt/4 pt
`exampleblock`	Green!5	Green!50!black	6 pt/4 pt
`noteblock`	Blue!5	Blue!50!black	6 pt/4 pt
`definitionblock`	Orange!5	Orange!70!black	6 pt/4 pt

42.8 Multi-Column Layouts

The `presentationcolumns` environment wraps a `columns[T]` environment (top-aligned). Use `\presentationcolumn{w}` to open each column, where `w` is a fraction of `textwidth`:

```
\begin{presentationframe}
    \slidetitle{Two-Column Comparison}
    \begin{presentationcolumns}
        \presentationcolumn{0.45}
            \textbf{Advantages:}
            \begin{itemize}
                \item Reproducible
                \item Version controlled
            \end{itemize}
        \end{column}
        \presentationcolumn{0.45}
            \textbf{Disadvantages:}
            \begin{itemize}
                \item Learning curve
                \item Compile time
            \end{itemize}
        \end{column}
    \end{presentationcolumns}
\end{presentationframe}
```

The default column width is 0.48`textwidth`. Each column must be closed explicitly with `\end{column}`. For three-column layouts, adjust widths accordingly:

```
\begin{presentationcolumns}
    \presentationcolumn{0.30}
        Column 1.
    \end{column}
    \presentationcolumn{0.30}
        Column 2.
    \end{column}
    \presentationcolumn{0.30}
        Column 3.
    \end{column}
\end{presentationcolumns}
```

42.9 Section Dividers

The `\presentationSection{title}` command creates a full-width section divider page. It fills the upper 50% with `universityprimary`, centres the title in 32 pt bold white, and includes the footer and progress bar. The frame counter is incremented automatically.

```
\presentationSection{Experimental Setup}
\begin{presentationframe}
    \slidetitle{Equipment}
    We used a calibrated spectrometer\ldots
\end{presentationframe}
```

Section dividers count toward the total frame count, so the progress bar accounts for them correctly.

42.10 TikZ Guard Behaviour

The presentation module auto-loads TikZ if it is not already present:

- If `enabletikz=true` (default), TikZ is loaded before the presentation module, so all overlays render.

- If `enabletikz=false`, the module attempts to load TikZ itself. If `tikz.sty` is unavailable, the `\ifpresentation@tikz` flag is set to `false` and all TikZ overlays (header, footer, progress bar, title page) are silently skipped.

This means presentations always compile even without TikZ, but without any decorative elements.

42.11 Building Presentations

Use the standard build commands:

```
# Via Docker
docker run --rm --entrypoint "" \
    -v "$PWD:/workspace" \
    ghcr.io/wyattau/omnilatex:latest \
    python3 build.py --mode prod --force build-example presentation

# Via Nix
nix develop .# --command python3 build.py --mode prod build-example presentation
```

The output PDF uses the 16:10 aspect ratio. For 4:3 projectors, override the paper dimensions in `config/document-settings.sty`:

```
\AtBeginDocument{%
    \setlength{\paperwidth}{254mm}%
    \setlength{\paperheight}{190.5mm}%   % 16:10 (default)
    % \setlength{\paperheight}{190.5mm}%  % uncomment for 4:3: use 254mm x 190.5mm
}
```

42.12 Common Patterns

42.12.1 Thank-You Slide

A simple closing slide:

```
\begin{presentationframe}
    \vfill
    \begin{center}
        {\Huge\bfseries\color{universityprimary}Thank You!}\par
        \vspace{8mm}
        {\Large Questions?}\par
        \vspace{6mm}
        {\normalsize\texttt{jane@example.com}}
    \end{center}
    \vfill
\end{presentationframe}
```

42.12.2 Listing Slides

Combine `minted` with `presentationblock`:

```
\begin{presentationframe}
    \slidetitle{Example Implementation}
    \begin{presentationblock}[Python]{Fibonacci}
        \mintinline[fontsize=\small]{python}|def fib(n): ...|
    \end{presentationblock}
\end{presentationframe}
```

42.12.3 Figure Slides

Place figures inside frames without float wrappers:

```
\begin{presentationframe}
    \slidetitle{System Architecture}
    \begin{center}
        \includegraphics[width=0.75\textwidth]{figures/architecture}
    \end{center}
\end{presentationframe}
```

Do not use `\begin{figure}` inside frames – floats are suppressed by the empty page style.

43 Advanced Posters

This chapter expands on the `poster` doctype overview from Chapter ?? with detailed customization, layout strategies, and integration with OmniLaTeX's TikZ and plotting libraries.

43.1 Design Philosophy

OmniLaTeX posters use `scrartcl` with an A1 landscape layout (841 mm × 594 mm). Unlike dedicated poster classes such as `beamerposter`, the OmniLaTeX approach:

- Provides the same typography, bibliography, and code listing infrastructure as all other doctypes.
- Uses standard `multicol` for column layout rather than a custom grid system.
- Defines poster-specific tcolorbox environments for titled and untitled content blocks.
- Supports all OmniLaTeX features including glossaries, minted code listings, and institution colour schemes.

The poster profile is defined in `config/document-types/poster.sty`.

43.2 Layout Configuration

The doctype applies these settings automatically:

Table 43.1: Poster doctype defaults.

Setting	Value
Font size	20 pt
Font mode	Sans-serif
Paper size	A1 landscape (841 mm × 594 mm)
KOMA `DIV`	20
Line spacing	Single
Page style	Empty
Section depth	0 (no section numbers)
TOC depth	0
Colour mode	Full colour

Override paper dimensions for different conference requirements:

```
% A0 landscape (1189mm x 841mm)
\AtBeginDocument{%
    \setlength{\paperwidth}{1189mm}%
```

```
    \setlength{\paperheight}{841mm}%
}

% A2 portrait (420mm x 594mm)
\AtBeginDocument{%
    \setlength{\paperwidth}{420mm}%
    \setlength{\paperheight}{594mm}%
}
```

43.3 Title Page

The `\maketitle` override renders the title in 60 pt bold, the author list in `\LARGE`, and the date in `\Large`, all centred. No decorative bars or TikZ overlays are drawn.

```
\documentclass[doctype=poster, institution=none]{../../omnilatex}
\title{OmniLaTeX: A Universal Document Template}
\author{Jane Doe, John Smith}
\date{International Conference on Document Engineering, 2026}
\begin{document}
\maketitle
% Poster content follows
\end{document}
```

The title page adds 8 mm of vertical space after the date block before content begins.

43.4 Poster Block Environments

Two tcolorbox environments are defined:

43.4.1 posterblock

```
\begin{posterblock}[Section Title]
    Content goes here.  Supports all standard \LaTeX{} commands,
    including itemize, enumerate, math, and figures.
\end{posterblock}
```

Default styling:

- White background
- Frame colour: `black!70`
- Title text: white on `black!70` background
- Title font: `\bfseries\Large`
- Rounded corners: 3 mm
- Box rule: 1.5 pt
- Inner padding: 8 mm all sides

43.4.2 posterblocknotitle

```
\begin{posterblocknotitle}
    \centering
    % Embedded figure or TikZ diagram
    \includegraphics[width=\textwidth]{results-chart}
\end{posterblocknotitle}
```

Same styling as `posterblock` but without the title bar. Useful for nesting figures inside a titled block:

```
\begin{posterblock}[Results]
    Our model achieves 95\% accuracy after 30 epochs.

    \begin{posterblocknotitle}
        \centering
        \begin{tikzpicture}
            \begin{axis}[
                width=0.8\textwidth,
                height=50mm,
                xlabel={Epoch},
                ylabel={Accuracy (\%)},
            ]
                \addplot[blue, thick, domain=1:50, samples=30]
                    {95 - 60*exp(-0.08*x)};
            \end{axis}
        \end{tikzpicture}
    \end{posterblocknotitle}
\end{posterblock}
```

43.5 Customising Block Styles

Since both environments are standard tcolorbox definitions, you can override them in `config/document-settings.sty`:

```
% Redefine posterblock with institution colours
\renewtcolorbox{posterblock}[1][]{%
    enhanced,
    colback=white,
    colframe=universityprimary,
    coltitle=white,
    fonttitle=\bfseries\Large,
    title=#1,
    arc=3mm,
    boxrule=1.5pt,
    top=8mm, bottom=8mm, left=8mm, right=8mm,
}
```

This replaces the default `black!70` frame with the institution's primary colour.

43.6 Layout Helpers

43.6.1 posterwidth

The `\posterwidth` command provides a consistent width for blocks:

```
\begin{posterblock}[Introduction]
    % This block automatically uses 0.9\textwidth internally
    % via the \posterwidth length if you reference it:
    Content with \texttt{\textbackslash posterwidth} = 0.9\textbackslash textwidth.
\end{posterblock}
```

The length is defined as `\providecommand`, so it can be redefined:

```
\renewcommand{\posterwidth}{0.95\textwidth}
```

43.6.2 Column Breaks

Use `\columnbreak` inside `multicols` to force content to the next column:

```
\begin{multicols}{3}
    \begin{posterblock}[Introduction]
        First block in column 1.
    \end{posterblock}
    \begin{posterblock}[Methods]
        Second block in column 1.
    \end{posterblock}

    \columnbreak  % Force to column 2

    \begin{posterblock}[Results]
        First block in column 2.
    \end{posterblock}

    \columnbreak  % Force to column 3

    \begin{posterblock}[Conclusion]
        First block in column 3.
    \end{posterblock}
\end{multicols}
```

43.7 Multi-Column Layout Strategy

The `multicol` package handles column balancing automatically, but posters typically need manual control. Use these strategies:

```
\begin{multicols}{3}

    % === Column 1 ===
    \begin{posterblock}[Introduction]
        Background and motivation\ldots
    \end{posterblock}
    \begin{posterblock}[Objectives]
        Research questions\ldots
    \end{posterblock}

    \columnbreak

    % === Column 2 ===
    \begin{posterblock}[Methods]
        Experimental setup\ldots
    \end{posterblock}
    \begin{posterblock}[Results]
        Key findings with embedded chart\ldots
        \begin{posterblocknotitle}
            \centering
            % pgfplots or TikZ figure
        \end{posterblocknotitle}
    \end{posterblock}

    \columnbreak

    % === Column 3 ===
    \begin{posterblock}[Discussion]
        Interpretation\ldots
    \end{posterblock}
```

```
    \begin{posterblock}[References]
        \small
        \begin{enumerate}
            \item Author, \textit{Title}, Journal, Year.
        \end{enumerate}
    \end{posterblock}

\end{multicols}
```

For two-column posters, use `\begin{multicols}{2}` and adjust block widths if needed.

43.8 Embedding TikZ Diagrams

Posters often include architecture diagrams or flowcharts. Use `posterblocknotitle` as a container:

```
\begin{posterblock}[System Architecture]
    \begin{posterblocknotitle}
        \centering
        \begin{tikzpicture}[
            box/.style={draw, thick, rounded corners,
                minimum height=10mm, minimum width=22mm,
                fill=universityprimary!10, font=\small},
            arr/.style={->, thick, >=stealth},
            node distance=10mm,
        ]
            \node[box] (input) {Input Data};
            \node[box, below=of input] (proc) {Processing};
            \node[box, below left=of proc] (model) {Model};
            \node[box, below right=of proc] (eval) {Evaluation};
            \node[box, below=of proc] (output) {Output};

            \draw[arr] (input) -- (proc);
            \draw[arr] (proc) -- (model);
            \draw[arr] (proc) -- (eval);
            \draw[arr] (model) -- (output);
            \draw[arr] (eval) -- (output);
        \end{tikzpicture}
    \end{posterblocknotitle}
\end{posterblock}
```

Use `universityprimary!10` for fill colours to maintain visual consistency with the header.

43.9 Embedding pgfplots Charts

Combine pgfplots with `posterblocknotitle` for data visualisation:

```
\begin{posterblock}[Experimental Results]
    Accuracy improves with training epochs.

    \begin{posterblocknotitle}
        \centering
        \begin{tikzpicture}
            \begin{axis}[
                width=0.85\textwidth,
                height=55mm,
                xlabel={Epoch},
                ylabel={Accuracy (\%)},
                legend style={font=\footnotesize,
```

```
                    at={(0.03,0.97)}, anchor=north west},
                grid=major,
                thick,
            ]
                \addplot[blue, thick, smooth, domain=1:50, samples=30]
                    {95 - 60*exp(-0.08*x)};
                \addlegendentry{Model A}

                \addplot[red, thick, dashed, smooth, domain=1:50, samples=30]
                    {93 - 55*exp(-0.06*x)};
                \addlegendentry{Model B}
            \end{axis}
        \end{tikzpicture}
    \end{posterblocknotitle}
\end{posterblock}
```

Set `width` relative to `\textwidth` to scale with column width. Heights between 45 mm and 60 mm work well for A1 posters.

43.10 Code Listings on Posters

The `minted` package works inside poster blocks:

```
\begin{posterblock}[Algorithm]
    \begin{posterblocknotitle}
        \mintinline[fontsize=\footnotesize]{python}|def train(model, data, epochs=50): ...|
    \end{posterblocknotitle}
\end{posterblock}
```

Use `fontsize=\footnotesize` or `fontsize=\small` to fit code within poster blocks.

43.11 Building Posters

```
# Via Docker
docker run --rm --entrypoint "" \
    -v "$PWD:/workspace" \
    ghcr.io/wyattau/omnilatex:latest \
    python3 build.py --mode prod --force build-example poster

# Via Nix
nix develop .# --command python3 build.py --mode prod build-example poster
```

Poster compilation is slower than other doctypes due to the large page size and TikZ/pgfplots rendering. Allow 30–60 seconds for complex posters.

43.12 Institution Integration

Load an institution config to apply its colour scheme:

```
\documentclass[
    doctype=poster,
    institution=tuhh,
    enabletikz=true,
    enableengineering=true,
]{../../omnilatex}
```

The institution's `universityprimary` and `universitysecondary` colours become available for use in poster blocks and TikZ diagrams. You can reference them directly:

```
\begin{posterblock}[Introduction]
    % Use institution colour for emphasis
    \textcolor{universityprimary}{\textbf{Key insight:}}
    Our approach reduces compile time by 40\%.
\end{posterblock}
```

43.13 Common Patterns

43.13.1 QR Code for Digital Supplement

Include a QR code linking to a repository or supplementary material:

```
\usepackage{qrcode}

\begin{posterblock}[Online Resources]
    \begin{posterblocknotitle}
        \centering
        \qrcode[height=30mm]{https://github.com/WyattAu/OmniLaTeX-template}\\[4mm]
        \small\texttt{github.com/WyattAu/OmniLaTeX-template}
    \end{posterblocknotitle}
\end{posterblock}
```

43.13.2 Acknowledgements Section

Use a narrow block at the bottom of the final column:

```
\begin{posterblock}[Acknowledgements]
    \small
    This work was supported by the National Science Foundation
    (Grant \#12345).  We thank our colleagues for valuable feedback.
\end{posterblock}
```

43.13.3 Numbered References

For conferences requiring numbered references:

```
\begin{posterblock}[References]
    \small
    \begin{enumerate}
        \item A.\ Author, B.\ Author.
              \textit{Title of Paper}.
              Journal Name, 2025.
        \item C.\ Author.
              \textit{Another Title}.
              Proceedings of Conference, 2026.
    \end{enumerate}
\end{posterblock}
```

For biblatex integration, use the standard `\printbibliography` command with a `refsection` if you need poster-specific references.

Part X

Build & Automation

44 Build Modes

OmniLaTeX supports three build modes that trade completeness for speed. The active mode is determined by the `BUILD_MODE` environment variable and is communicated to the LaTeX source via macros defined in `.latexmkrc`.

44.1 dev Mode

Development mode (`BUILD_MODE=dev`) is the default. It runs `latexmk` with up to six passes, bibliography processing, and `shell-escape` enabled:

```
BUILD_MODE=dev python build.py
```

This mode is optimised for iteration speed while still producing a correct PDF with resolved cross-references and citations. Biber runs without `--validate-datamodel` to avoid slow validation on every iteration. Glossary compilation via `bib2gls` is active.

44.2 prod Mode

Production mode (`BUILD_MODE=prod`) performs full validation:

```
BUILD_MODE=prod python build.py
```

All packages are active, including accessibility checks (`\validatstructure`), glossary compilation, and minted code highlighting. Biber runs with `--validate-datamodel` to catch malformed bibliography entries. This is the mode to use for final output and archival builds.

44.3 ultra Mode

Ultra mode (`BUILD_MODE=ultra`) prioritises speed over completeness:

```
BUILD_MODE=ultra python build.py
```

Only a single `latexmk` pass is executed. Bibliography processing is disabled entirely (`$bibtex_use = 0`) and Biber is replaced with `true`. Glossary compilation is skipped. Cross-references will show "??" placeholders. Use this mode for rapid content drafting where exact output is not yet needed.

44.4 Mode Detection

Document source can detect the current build mode via three macros:

```
\ifdefined\OmniDev
  % Development build -- include draft notes
  \todo{Check this section before submission.}
\fi

```

```
\ifdefined\OmniProd
  % Production build -- run all validation
  \validatstructure
\fi

\ifdefined\OmniUltra
  % Ultra-fast build -- skip heavy packages
\fi
```

The `BUILD_MODE` environment variable is also available in shell commands invoked via `shell-escape`.

44.5 Mode Comparison

Table 44.1: Build mode feature comparison.

Feature	dev	prod	ultra
`max_repeat`	6	7	1
Biber runs	Yes (no validation)	Yes (`--validate-datamodel`)	No (`true`)
`bib2gls`	Yes	Yes	No
Shell escape	Yes	Yes	Yes
Minted highlighting	Yes	Yes	Yes
Accessibility checks	No	Yes	No
Forward refs resolved	Yes	Yes	No

44.6 Setting Build Mode

Two methods are available:

1. **Environment variable:**

```
BUILD_MODE=prod python build.py
```

2. **Command-line flag:**

```
python build.py --mode prod
```

The command-line flag takes precedence over the environment variable. If neither is set, `dev` is used.

44.7 .latexmkrc Detection

The mode is detected in `.latexmkrc` by reading the `BUILD_MODE` environment variable:

```
my $build_mode = $ENV{'BUILD_MODE'} || 'dev';

if ($build_mode eq 'prod') {
    $max_repeat = 7;
} elsif ($build_mode eq 'ultra') {
    $max_repeat = 1;
    $bibtex_use = 0;
    $do_gls = 0;
```

```
} else {
    $max_repeat = 6;
}
```

The `$max_repeat` variable controls how many times `latexmk` reruns `lualatex` before declaring the build complete. Complex documents with many cross-references may need all seven passes in production mode.

44.8 When to Use Each Mode

Table 44.2: Recommended mode by workflow.

Mode	When to use
`ultra`	Rapid drafting, typing content, checking layout roughly. Accepts broken refs and missing bib.
`dev`	Normal editing. Resolves all refs and bib. Default for day-to-day work.
`prod`	Final builds, submission, archival. Full validation with accessibility checks.

44.9 Performance Comparison

Typical build times for a 100-page document with bibliography and glossary:

Table 44.3: Approximate build times (100-page document).

Mode	Cold build	Incremental
`ultra`	~5 s	~3 s
`dev`	~30 s	~8 s
`prod`	~45 s	~12 s

Incremental times assume a single paragraph change. Actual times vary with hardware, TeX Live installation size, and document complexity.

44.10 Custom Configuration

To add a custom build mode, extend `.latexmkrc`:

```
if ($build_mode eq 'thesis') {
    $max_repeat = 8;
    $biber = "biber --validate-datamodel --debug %O %S";
    $do_gls = 1;
}
```

Define a corresponding macro in the document preamble:

```
\ifdefined\OmniThesis
  \usepackage{todonotes}
  \setlength{\marginparwidth}{2cm}
\fi
```

45 Git Integration

45.1 Git Metadata Commands

The template provides four macros for embedding repository metadata in the document, all defined in `omnilatex-utils.sty`:

\GitRefName The current branch or tag name.

```
Compiled from branch: \GitRefName
```

\GitShortSHA The abbreviated commit hash (8 characters).

```
Commit: \GitShortSHA
```

\GitLongSHA The full 40-character commit hash.

```
Full SHA: \GitLongSHA
```

\BuildDate The date and time of compilation.

```
Built on \BuildDate.
```

All Git macros require `shell-escape` to be enabled, which is the default in all build modes. The macros invoke `git` at compile time, so the working directory must be inside a Git repository.

45.2 Verification Box

`\GitVerificationBox` renders a formatted box suitable for colophon pages:

```
\chapter*{Colophon}
\GitVerificationBox
```

The box displays:

- The full and short commit SHA.
- The branch or tag name.
- A verification command the reader can run to confirm the build.

45.3 Build Date

`\BuildDate` integrates with `datetime2` (`\DTMnow`) to produce a locale-aware timestamp:

```
This document was compiled on \BuildDate.
```

The format respects the document's language and date settings configured via the `mainlang` class option.

45.4 .gitignore for LaTeX Projects

A well-configured `.gitignore` keeps the repository clean by excluding generated files. OmniLaTeX repositories should include the following patterns:

```
# LaTeX auxiliary files
*.aux
*.lof
*.lot
*.loe
*.lol
*.lor
*.toc
*.out
*.synctex.gz
*.fdb_latexmk
*.fls
*.idx
*.ilg
*.ind
*.ist
*.acn
*.acr
*.alg
*.glg
*.glo
*.gls
*.glsdefs
*.slg
*.slo
*.sls

# Bibliography and glossary intermediates
*.bbl
*.bcf
*.blg
*.run.xml
*-*.glstex
*.glstex

# Build output
*.pdf
build/

# Minted cache
_minted-*/
__pycache__/
```

The pattern `*-*.glstex` is required because `glossaries-extra` generates numbered variants (`main-1.glstex`, `main-2.glstex`, …) when multiple glossary types are present.

45.5 Binary PDF Handling

PDFs generated by the build should generally be excluded from version control. However, some workflows require committing specific release PDFs. Use `.gitattributes` to control diff behaviour:

```
# Binary diff (default)
*.pdf binary
```

```
# Force text diff on .tex files regardless of line endings
*.tex text diff=latex
```

Git's built-in binary diff marks PDFs as changed without showing content differences. To compare PDFs visually, use an external diff driver such as `diff-pdf`:

```
[diff "pdf"]
  textconv = diff-pdf
  binary = true
```

45.6 .latexmkrc in Version Control

The root `.latexmkrc` should be committed to the repository so that all collaborators and CI pipelines share identical build settings:

```
# Verify .latexmkrc is tracked
git ls-files .latexmkrc
```

When `build.py` creates example builds, it symlinks the root `.latexmkrc` into each example directory. This ensures that standalone `latexmk` invocations use the same configuration:

```
# build.py creates this symlink automatically
ln -s ../../.latexmkrc examples/my-example/.latexmkrc
```

45.7 Pre-Commit Hooks

Pre-commit hooks catch errors before they reach the repository. Two common hooks for LaTeX projects are `chktex` (syntax checking) and `latexmk -c` (auxiliary file cleanup):

```
#!/usr/bin/env bash
# .git/hooks/pre-commit

echo "Running chktex..."
find . -name '*.tex' -not -path './build/*' \
  -exec chktex -q -n 1 -n 44 {} + || true

echo "Cleaning auxiliary files..."
latexmk -c -silent 2>/dev/null || true
```

For a more robust setup, use the `pre-commit` framework with community-maintained hooks:

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v5.0.0
    hooks:
      - id: trailing-whitespace
        files: \.tex$
      - id: end-of-file-fixer
        files: \.tex$
  - repo: https://github.com/cmhughes/latexindent.pl
    rev: v3.24.4
    hooks:
      - id: latexindent
```

Table 45.1: Recommended branch strategy.

Branch	Purpose	Build mode
`main`	Stable release	`prod`
`develop`	Active work	`dev`
`draft/*`	Section drafts	`ultra`
`release/*`	Release prep	`prod`

45.8 Branch Strategy

A simple branch strategy for LaTeX documents:

45.9 Collaborative Editing

When multiple authors work on the same document, follow these practices:

1. Divide content into separate `.tex` files and assemble them via `\input{}` or `\import{}` in the main file.
2. Use `git merge` rather than rebasing to preserve a linear history for review.
3. Resolve merge conflicts promptly; LaTeX syntax errors from malformed merges are difficult to diagnose.
4. Commit the `flake.lock` file when using Nix to ensure all collaborators share identical tool versions.

45.10 Release Tagging

Tag release versions in Git and let the build system embed the tag name via `\GitRefName`:

```
git tag -a v1.0.0 -m "Thesis final submission"
git push origin v1.0.0
BUILD_MODE=prod python build.py
```

The resulting PDF will display `v1.0.0` as the branch/tag name on the colophon page.

45.11 .gitattributes for Line Endings

LaTeX source files should use LF line endings consistently. Configure `.gitattributes` to enforce this:

```
# Force LF for all text files
* text=auto eol=lf

# Binary formats
*.pdf binary
*.png binary
*.jpg binary
*.svg binary
```

This prevents CRLF/LF mismatches that cause spurious diffs on Windows systems.

46 Docker Workflow

46.1 Docker Image

The official OmniLaTeX Docker image is published to GitHub Container Registry:

```
ghcr.io/wyattau/omnilatex-docker
```

The image bundles TeX Live 2026 with all template dependencies pre-installed. Font caches are pre-warmed, eliminating the cold-start penalty on first compilation. The compressed image is approximately 2 GB.

46.2 Pulling the Image

```
docker pull ghcr.io/wyattau/omnilatex-docker:latest
```

Pin to a specific version for reproducibility:

```
docker pull ghcr.io/wyattau/omnilatex-docker:v2026.1.0
```

46.3 Build Commands

A full production build inside Docker:

```
docker run --rm \
  --entrypoint "" \
  -v "$PWD:/workspace" \
  -w /workspace \
  ghcr.io/wyattau/omnilatex-docker:latest \
  bash -c "BUILD_MODE=prod python build.py"
```

Development build:

```
docker run --rm \
  --entrypoint "" \
  -v "$PWD:/workspace" \
  -w /workspace \
  ghcr.io/wyattau/omnilatex-docker:latest \
  bash -c "BUILD_MODE=dev python build.py"
```

Ultra-fast iteration:

```
docker run --rm \
  --entrypoint "" \
  -v "$PWD:/workspace" \
  -w /workspace \
  ghcr.io/wyattau/omnilatex-docker:latest \
  bash -c "BUILD_MODE=ultra python build.py"
```

46.4 ENTRYPOINT Bypass

The Docker image sets `ENTRYPOINT` to `latexmk "$@"`, which means any arguments passed to `docker run` are forwarded directly to `latexmk`. To run arbitrary shell commands (such as `build.py` or `make`), you *must* use:

```
--entrypoint ""
```

Without this flag, the container attempts to pass your command as arguments to `latexmk`, which will fail.

46.5 Volume Mounting

The standard volume-mount pattern maps the current directory into the container's workspace:

```
-v "$PWD:/workspace" -w /workspace
```

This ensures that output PDFs are written directly to your local filesystem. No copy step is needed.

46.6 Font Cache

The Docker image ships with pre-warmed font caches for all bundled fonts. On a cold host, this saves 30–60 seconds on the first `latexmk` pass compared to a bare TeX Live installation.

46.7 Cross-Platform

The Docker image runs on all major platforms:

- **Linux** – native Docker Engine.
- **macOS** – Docker Desktop (Intel or Apple Silicon).
- **Windows** – WSL2 with Docker Desktop.

No platform-specific configuration is required. The same `docker run` command works identically everywhere.

47 Nix Workflow

47.1 Overview

The OmniLaTeX repository includes a `flake.nix` that declares all TeX Live packages and development dependencies as pinned, reproducible derivations. Nix ensures that every collaborator and CI pipeline uses identical tool versions regardless of host operating system or installed packages.

47.2 flake.nix Structure

The flake declares inputs and outputs using `flake-utils` for cross-platform support:

```
{
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable";
    flake-utils.url = "github:numtide/flake-utils";
  };

  outputs = { nixpkgs, flake-utils, ... }:
    flake-utils.lib.eachDefaultSystem (system:
      let pkgs = nixpkgs.legacyPackages.${system};
      in {
        devShells.default = pkgs.mkShell {
          packages = with pkgs; [
            (texlive.combine {
              inherit (texlive) scheme-full ...;
            })
            python3
            lean4
            lake
          ];
        };
      }
    );
}
```

The `nixos-unstable` channel provides the latest package versions. Inputs are locked by `flake.lock`, which pins every dependency to an exact Git revision.

47.3 Packages Included

The development shell provides the following packages:

47.4 Activating the Dev Shell

Enter the development environment with:

```
nix develop .#
```

Table 47.1: Packages in the Nix dev shell.

Package	Version	Purpose
`texlive`	Full scheme	Complete TEX distribution
`python3`	System Python	`build.py`, Pygments
`lean4`	Latest stable	Formal verification
`lake`	Bundled with Lean 4	Lean build tool
`latexmk`	Bundled with TeX Live	Build orchestrator
`biber`	Bundled with TeX Live	Bibliography processor
`bib2gls`	Bundled with TeX Live	Glossary processor

This provides a shell with TeX Live (full scheme), Python 3, and Lean 4. All tools are available on `PATH` without polluting the host system. On first invocation, Nix downloads and builds all derivations; subsequent activations use the Nix store cache and start instantly.
For a one-off command without entering the shell:

```
nix develop .# --command python build.py --mode prod
```

47.5 Reproducible Builds

`flake.lock` pins every dependency to an exact revision, ensuring bit-for-bit reproducible builds across machines. Commit `flake.lock` alongside your source to guarantee that collaborators and CI use identical package versions:

```
git add flake.lock
git commit -m "Pin Nix dependencies"
```

47.6 Pinning nixpkgs

To pin to a specific nixpkgs release rather than `nixos-unstable`:

```
{
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.11";
  };
}
```

Or pin to an exact commit hash for maximum reproducibility:

```
nixpkgs.url = "github:NixOS/nixpkgs/a1e5e2e3f4d5c6b7a8";
```

After changing the input URL, update the lock file:

```
nix flake lock --update-input nixpkgs
```

47.7 Lean 4 Integration

Proof-heavy documents can use Lean 4 for formal verification. The dev shell includes `lean4` and `lake`, so no separate installation is needed:

```
lake build
```

Lean 4 source files (`.lean`) are kept alongside the `.tex` files. The `minted` package provides syntax highlighting for Lean 4 code blocks:

```
\mintinline{lean4}|theorem add_comm (m n : Nat) : m + n = n + m := by
  induction m with
  | zero => simp
  | succ m ih => simp [Nat.succ_add, ih]|
```

47.8 Multi-Platform Support

The flake supports multiple systems via `flake-utils`:

Table 47.2: Supported platforms.

System	Status
`x86_64-linux`	Primary target, fully tested
`aarch64-darwin`	Apple Silicon (macOS)
`x86_64-darwin`	Intel macOS
`aarch64-linux`	ARM Linux (experimental)

47.9 nix flake check

Validate the flake without building the full TeX distribution:

```
nix flake check
```

This verifies that the flake evaluates correctly and all outputs are consistent. It does not compile any LaTeX documents.

47.10 nix build

To build a specific derivation (e.g. the dev shell closure) without entering it:

```
nix build .#devShells.x86_64-linux.default
```

The resulting store path is a symlink at `./result`.

47.11 CI Integration

GitHub Actions workflows use Nix for reproducible CI:

```
- uses: DeterminateSystems/nix-installer-action@main
- uses: DeterminateSystems/magic-nix-cache-action@main
- run: nix develop .# --command bash -c \
      "BUILD_MODE=prod python build.py"
```

The `magic-nix-cache-action` enables binary cache sharing between CI runs, reducing build times significantly. First-time CI runs download the full TeX Live closure; subsequent runs restore from cache in seconds.

47.12 Multi-User Collaboration

When multiple users share a Nix-based workflow:

1. Commit `flake.lock` to version control.
2. Use `nix develop .` instead of installing packages globally.
3. Document the Nix version requirement in the project README.
4. For CI, use the `DeterminateSystems/nix-installer-action` to guarantee a consistent Nix version.

47.13 Troubleshooting

TeX Live not found Ensure `shell-escape` is enabled in `.latexmkrc`. Nix installs TeX Live in the Nix store, which is on `PATH` inside the dev shell but not outside it.

Pygments missing The `python3` package in the dev shell includes `pip`. Install Pygments with `pip install Pygments` inside the dev shell, or add it to the flake packages list:

```
packages = with pkgs; [
  python3Packages.pygments
  # ... other packages
];
```

Flake evaluation errors Run `nix flake check` to identify syntax or type errors in the flake definition.

Stale lock file If builds fail after a Nix channels update, run `nix flake lock --update-input nixpkgs` to refresh the lock file.

48 CI/CD

48.1 GitHub Actions Overview

OmniLaTeX uses GitHub Actions with nine workflows covering every stage of the development lifecycle. Each workflow is triggered by different events:

Workflow	Trigger	Purpose
`build.yml`	push, PR, manual	Full build, test, deploy
`lean4-ci.yml`	push, PR, manual	Lean 4 proof verification
`ctan-upload.yml`	tag push (`v*`)	CTAN package upload
`ctan-release.yml`	tag push (`v*`)	GitHub Release creation
`ctan.yml`	push, tag, manual	CTAN zip validation
`cross-platform.yml`	push, PR, manual	Linux + Windows Docker
`docker-ci.yml`	push, tag, PR	Docker image build + push
`docker-digest-sync.yml`	workflow_run	Digest sync PR
`integration-matrix.yml`	weekly cron, manual	Doctype × language matrix

Table 48.1: GitHub Actions workflow summary.

48.2 Build Pipeline

The primary workflow (`build.yml`) runs on every push to `main`, `master`, or `develop`, and on pull requests to `main`/`master`. It consists of six jobs:

1. **build** – Pulls the pinned Docker image from `.env.docker`, runs `python3 build.py --mode prod --verbose build-all` to compile all 43 examples, verifies every expected PDF was produced, uploads root and example PDFs as artifacts, and prepares a GitHub Pages bundle.
2. **performance** – Depends on `build`. Rebuilds all examples with `--timings --force`, generates a performance summary with regression detection via `.github/scripts/extract`$_m$`etrics.py`, *and uploads* `build/metrics.json` *as an artifact*.
3. **determinism** – Runs independently. Builds the thesis example twice with `--source-date-epoch 1700000000`, compares page counts and file sizes with a 3% tolerance to detect non-deterministic output.
4. **test** – Depends on `build`. Downloads PDF artifacts and runs integration tests via `pytest` against `tests/test_modules.py` and `tests/test_ctan.py`.
5. **deploy-pages** – Depends on `build`. Deploys the site bundle to GitHub Pages (only on `main` push).
6. **changelog** – Runs only on pull requests. Checks that `CHANGELOG.md` was updated whenever `.sty` or `.cls` files were modified.

48.3 Lean 4 Verification

The `lean4-ci.yml` workflow runs on pushes and pull requests to `main`/`master`. It uses Nix for a reproducible environment:

```
nix develop --command bash -c "cd specs/proofs && lake build"
```

After building, a verification step generates a summary table for all ten proof modules. If any file contains `sorry`, a warning is emitted. The ten modules verified are:

Module	Domain
BuildModes	Build mode properties
BuildSystem	Cache correctness, parallelism bounds
DoctypeClassMapping	Doctype-to-class mapping (60 aliases)
DocumentSettings	KOMA class partition
DoctypeResolution	Alias resolution
FloatInvariant	Float placement bounds
FontHierarchy	Strict total order on 9 font sizes
I18nCompleteness	Translation key parity (846 keys)
LanguageFallback	Fallback chain stability
PageGeometry	Page balance equations

Table 48.2: Lean 4 proof modules verified in CI.

48.4 CTAN Auto-Upload

The `ctan-upload.yml` workflow triggers on version tags (`v*`). It implements a two-job pipeline backed by a five-phase upload script (`scripts/ctan-upload.sh`):

1. **Pre-flight job** – Builds the CTAN zip, validates its contents (must contain `omnilatex.cls`), extracts the version from the tag, generates a changelog, and runs a dry-run upload to verify all endpoints are reachable.
2. **Pre-flight validation** (within the upload script):
   - Validate package name via `/upload/validatePackageName`.
   - Validate version via `/upload/validatePackageVersion`.
   - Validate CTAN path via `/upload/validateCtanPath` (new packages only).
   - Check URL reachability for homepage, repository, bugs, and support links.
3. **CSRF token fetch** – Retrieves a hidden `ticket` field from the CTAN upload page for form-based authentication.
4. **Upload job** – Rebuilds the zip, regenerates the changelog, and submits the package to CTAN via a multipart `POST` request with all required metadata fields.
5. **Response parsing** – Checks for success patterns (“has been uploaded”), failure patterns (“already exists”, “invalid”), and HTTP status codes.

Secrets required: `CTAN_EMAIL` and `CTAN_AUTHOR`.

48.5 CTAN Release

The `ctan-release.yml` workflow also triggers on `v*` tags. It builds and validates the CTAN zip, then creates a GitHub Release with auto-generated release notes via `softprops/action-gh-release@v2`. The zip is attached to the release and uploaded as a 90-day artifact.

48.6 Cross-Platform

The `cross-platform.yml` workflow validates Docker builds on both Linux and Windows:

- **Linux job** – Runs on `ubuntu-latest`, pulls the Docker image, and builds the article example in production mode.
- **Windows job** – Runs on `windows-latest` with `continue-on-error: true` (Docker Desktop may not be available on all Windows runners). Uses `linux/amd64` platform emulation.
- **Report job** – Generates a cross-platform summary table showing pass/fail status for each platform.

48.7 Docker CI/CD

The `docker-ci.yml` workflow builds multi-architecture Docker images (`linux/amd64`, `linux/arm64`) and pushes them to `ghcr.io/wyattau/omnilatex-docker`. Key features:

- Uses BuildKit with `moby/buildkit:v0.29.0` and GHA cache sharing.
- Generates SBOM and provenance metadata for non-PR builds.
- Tags include `latest`, semver, and major.minor patterns.
- 6-hour timeout for multi-arch builds.

The companion `docker-digest-sync.yml` workflow runs automatically after a successful Docker build. It pulls the new `latest` image, extracts its digest, updates `.env.docker`, and creates a pull request with the `automation` label. This keeps all other workflows pointing at the latest validated image without editing workflow files.

48.8 Integration Matrix

The `integration-matrix.yml` workflow runs weekly (Sunday 02:00 UTC) or on manual dispatch. It tests 13 targeted doctype × language combinations:
Each combination generates a minimal document, compiles it with `latexmk -lualatex`, and verifies the PDF was produced. The `fail-fast: false` strategy ensures all combinations run even if some fail.

48.9 Determinism

The `determinism` job in `build.yml` verifies that the build system produces reproducible output. It uses `SOURCE_DATE_EPOCH` to freeze timestamps:

1. Build the thesis example with `--source-date-epoch 1700000000` and save the PDF.
2. Run `python3 build.py clean`.
3. Rebuild with the same epoch.

Doctype	Language
thesis	english
article	english
book	english
presentation	english
cv	english
cover-letter	english
dissertation	english
manual	english
patent	english
thesis	german
article	german
article	french
article	chinese

Table 48.3: Integration matrix combinations.

4. Compare the two PDFs.

Because LuaLaTeX is not fully bit-for-bit deterministic (PDF object ordering, Lua hash table randomisation, font subsetting), the comparison uses page counts and file sizes rather than exact SHA256 hashes. A 3% file size tolerance accounts for non-determinism in font subsetting and PDF object reordering across CI runners.

Part XI

Formal Verification

49 Lean 4 Formal Verification

49.1 Why Formal Verification?

LaTeX templates can harbour subtle bugs that escape manual review: wrong margin calculations, inconsistent doctype-to-class mappings, missing translation keys, or broken fallback chains. These errors surface only when a user hits the specific combination of options that triggers them.
OmniLaTeX addresses this with Lean 4 formal verification. Properties of the template system are expressed as mathematical theorems and proven correct at compile time. If a pull request breaks a mapping or introduces a contradiction, `lake build` fails before the LaTeX code is ever compiled.
The proofs live in `specs/proofs/` and are verified by CI on every push to `main`/`master` (see Chapter **??**).

49.2 Proof Modules

All ten proof modules are imported through the root file `specs/proofs/OmniLaTeXProofs.lean`. Each module covers a distinct aspect of the template system:

49.3 Proof Structure

Lean 4 proofs use a small set of foundational constructs:
Inductive types define the domain. For example, the three build modes are declared as:

```
inductive BuildMode where
  | dev : BuildMode
  | prod : BuildMode
  | ultra : BuildMode
  deriving DecidableEq, Repr
```

Structures bundle related fields:

```
structure BuildConfig where
  mode : BuildMode
  maxPasses : Nat
  runBiber : Bool
  deriving Repr
```

Function definitions provide the mapping logic:

```
def buildConfigFor : BuildMode → BuildConfig
  | .dev => ⟨.dev, 10, true, true, false, true, false⟩
  | .prod => ⟨.prod, 5, true, true, true, true, true⟩
  | .ultra => ⟨.ultra, 1, false, false, false, true, false⟩
```

Theorems state the properties to prove. The proof follows after `:= by`:

```
theorem ultra_no_bib :
    (buildConfigFor .ultra).runBiber = false ∧
    (buildConfigFor .ultra).runBib2gls = false := by
  simp [buildConfigFor]
```

The hierarchy within each module follows the convention `theorem` for main results, with `lemma` and `corollary` used for derived statements.

Module	Theorems	What It Proves
BuildModes	5	Build mode properties: ultra skips bibliography, prod validates datamodel, dev has enough passes, all modes enable shell-escape, strictness ordering.
BuildSystem	9	Cache correctness, 43-example count, parallelism bounds ($0 < \text{workers} \leq \text{examples}$), production is default.
DoctypeClassMapping	7	Deterministic doctype→class mapping, 60 aliases across 3 KOMA classes (34 scrartcl, 6 scrbook, 20 scrreprt), covering theorem, scrartcl is largest.
DocumentSettings	8	KOMA class partition: 15 article-class, 6 report-class, 5 book-class doctypes sum to 26, each non-empty.
DoctypeResolution	4	Alias resolution is deterministic and total over 55 known aliases; profile-to-class consistency.
FloatInvariant	2	Float placement never exceeds its section boundary; bounded deferral invariant.
FontHierarchy	4	Font sizes form a strict total order: irreflexive, asymmetric, transitive, and connex over 9 sizes.
I18nCompleteness	3	Translation key count is constant across all 18 languages; $18 \times 47 = 846$ total keys.
LanguageFallback	7	18 primary + 26 secondary = 44 languages; all fallbacks terminate at English; fallback is idempotent.
PageGeometry	5	Horizontal and vertical page balance equations hold for oneside and twoside layouts; DIV textwidth formula; caption width bounds.

Table 49.1: Lean 4 proof modules and their properties.

49.4 Reading Lean Proofs

Key patterns you will encounter in the proof files:

`def` Function or constant definitions.

`theorem` / `lemma` Statements to be proven.

`by` Opens a tactic-mode proof block.

`simp [name]` Rewrites using the definition of `name`.

`decide` Automatically proves decidable propositions (finite enumeration, equality checks).

`omega` Solves linear arithmetic over integers or naturals.

`cases` Pattern-matches on an inductive type.

`intro` / `exact` Introduces hypotheses and provides exact proof terms.

`have` Introduces a local lemma.

`rw` / `simp_all` Rewrites equations in the goal or all hypotheses.

49.5 Zero sorry Policy

All theorems across all ten modules are fully proven. No `sorry` or `admit` appears anywhere in the proof source. A `sorry` in Lean 4 is a placeholder that makes the type checker accept an unproven theorem – essentially an assertion without evidence.
The CI workflow (`lean4-ci.yml`) enforces this by running `grep -rq 'sorry' specs/proofs/` after every build. If any `sorry` is found, the step summary emits a warning, making incomplete proofs visible in the Actions UI.

49.6 Extending Proofs

To add a new proof module:

1. Create a new `.lean` file in `specs/proofs/OmniLaTeXProofs/`.
2. Define the relevant inductive types, structures, and functions.
3. State and prove theorems using tactics (`simp`, `decide`, `omega`, `cases`).
4. Add an `import` line to `specs/proofs/OmniLaTeXProofs.lean`.
5. Run `lake build` to verify compilation and correctness.
6. Add the module name to the verification loop in `lean4-ci.yml` if needed.

Proofs should avoid `sorry` and should prefer `decide` for finite domains and `omega` for arithmetic. For properties that require non-linear reasoning, consider whether the statement can be reformulated to stay within Lean 4's `Init` library (as done in `PageGeometry.lean`, where Float was replaced with Int for `omega` compatibility).

50 Module Architecture

50.1 Module Dependency Graph

OmniLaTeX loads its 27 library modules in a strict order to satisfy dependencies. The load order forms a directed acyclic graph with no circular dependencies:

1. **Core** – `omnilatex-base.sty` is always loaded first by `omnilatex.cls`. It provides fundamental macros and sets up the package infrastructure.
2. **Utils** – Utility modules loaded early because they are depended on by nearly everything else: `omnilatex-utils`, `omnilatex-colors`, `omnilatex-themes`.
3. **Typography** – Font, list, typesetting, and math modules: `omnilatex-fonts`, `omnilatex-lists`, `omnilatex-typesetting`, `omnilatex-math`.
4. **Layout** – Page geometry, KOMA integration, floats, boxes, document structure, presentation, and accessibility: `omnilatex-page`, `omnilatex-koma`, `omnilatex-floats`, `omnilatex-boxes`, `omnilatex-document`, `omnilatex-presentation`, `omnilatex-accessibility`.
5. **Graphics** – Image support and TikZ integration: `omnilatex-graphics`, `omnilatex-tikz-core`, `omnilatex-tikz-engineering`.
6. **References** – Hyperlinks, bibliography, citations, and glossary: `omnilatex-hyperref`, `omnilatex-biblio`, `omnilatex-citations`, `omnilatex-glossary`.
7. **Code** – Code listing support: `omnilatex-listings`.
8. **Tables** – Table formatting: `omnilatex-tables`.
9. **Language** – Internationalisation, CJK, and RTL: `omnilatex-i18n`, `omnilatex-cjk`, `omnilatex-rtl`.

Optional modules are gated by `enable*` class options (see Chapter **??**). When disabled, the corresponding `\RequirePackage` call is skipped entirely, reducing load time and avoiding unnecessary package conflicts.

50.2 Module Categories

All 27 modules are organised into nine categories by functional domain:

50.3 Design Principles

50.3.1 Single Responsibility

Each module has a clearly defined scope. `omnilatex-fonts` handles only font selection; `omnilatex-floats` handles only float placement; `omnilatex-i18n` handles only language and translation setup. Cross-cutting concerns (e.g. a float that uses a custom font) are handled by the composing document, not by merging module boundaries.

Category	Modules
Core	omnilatex-base
Utils	omnilatex-utils, omnilatex-colors, omnilatex-themes
Typography	omnilatex-fonts, omnilatex-lists, omnilatex-typesetting, omnilatex-math
Layout	omnilatex-page, omnilatex-koma, omnilatex-floats, omnilatex-boxes, omnilatex-document, omnilatex-pr
Graphics	omnilatex-graphics, omnilatex-tikz-core, omnilatex-tikz-engineering
References	omnilatex-hyperref, omnilatex-biblio, omnilatex-citations, omnilatex-glossary
Code	omnilatex-listings
Tables	omnilatex-tables
Language	omnilatex-i18n, omnilatex-cjk, omnilatex-rtl
Total	

Table 50.1: Module categories and their constituent packages.

50.3.2 Optional Loading

Modules that pull in heavy dependencies are gated behind boolean class options. The six optional feature flags are:

Option	Default	Modules Enabled
`enablefonts`	false	omnilatex-fonts (extended)
`enablegraphics`	false	omnilatex-graphics
`enablemath`	true	omnilatex-math
`enabletikz`	true	omnilatex-graphics, omnilatex-tikz-core
`enableengineering`	true	omnilatex-tikz-engineering
`enablecode`	true	omnilatex-listings
`enabletables`	true	omnilatex-tables

Table 50.2: Feature flags controlling optional module loading.

50.3.3 KOMA Integration

Layout modules integrate tightly with KOMA-Script rather than fighting against it. `omnilatex-koma.sty` provides the bridge layer that translates OmniLaTeX's doctype system into KOMA class options (`\LoadClass`), while `omnilatex-page.sty` uses KOMA's **`typearea`** for margin calculations rather than manual `\geometry` calls.

50.3.4 No Circular Dependencies

The load order is strictly hierarchical: Core → Utils → Typography → Layout → Graphics → References → Code → Tables → Language. A module in one tier may depend on modules in earlier tiers but never on modules in the same or later tiers. This is enforced by the sequential `\RequirePackage` calls in `omnilatex.cls`.

50.3.5 File Location

All library modules reside under `lib/` with one subdirectory per category:

```
lib/
  core/          omnilatex-base.sty
  utils/         omnilatex-utils.sty, omnilatex-colors.sty, omnilatex-themes.sty
  typography/    omnilatex-fonts.sty, omnilatex-lists.sty, ...
  layout/        omnilatex-page.sty, omnilatex-koma.sty, ...
  graphics/      omnilatex-graphics.sty, omnilatex-tikz-core.sty, ...
  references/    omnilatex-hyperref.sty, omnilatex-biblio.sty, ...
  code/          omnilatex-listings.sty
  tables/        omnilatex-tables.sty
  language/      omnilatex-i18n.sty, omnilatex-cjk.sty, omnilatex-rtl.sty
```

A Command Reference

This appendix provides a comprehensive quick-reference table of every OmniLaTeX custom command, organised by functional category. Detailed usage and examples for each command are found in the corresponding chapter.

A.1 Document Setup

Table A.1 / Document setup commands

Command	Description
`\documenttype{name}`	Declare document type label in a profile `.sty` file
`\manualbrand{label}`	Insert manual-specific branding element
`\signaturefield{width}`	Underlined signature field for forms

A.2 Typography

Table A.2 / Typography and text formatting commands

Command	Description
`\enquote{text}`	Context-aware quotation marks via **csquotes**
`\qq{text}`	Quick quotation shorthand: `\quad` of text between equation parts
`\name{x}`	Format a proper noun or name (glossary-linked)
`\sym{x}`	Format a mathematical symbol name (glossary-linked)
`\sub{x}`	Format a subscript reference (glossary-linked)
`\cons{x}`	Format a constant or physical constant name
`\censor{text}`	Black-bar redaction preserving layout (requires censoring option)
`\hologo{name}`	Typeset a software logo (e.g. LaTeX, TeX)
`\boldPDFchapterstrue`	Enable bold chapter entries in PDF bookmarks
`\boldPDFchaptersfalse`	Disable bold chapter entries in PDF bookmarks

A.3 Mathematics

Table A.3 / Mathematics commands

Command	Description
`\abs{x}`	Absolute value: $\lvert x\rvert$ (glossary-linked)
`\parens{x}`	Parenthesis delimiters: (x); starred variant auto-scales
`\brackets{x}`	Bracket delimiters: $[x]$; starred variant auto-scales
`\braces{x}`	Brace delimiters: $\{x\}$; starred variant auto-scales
`\mean{x}`	Mean value: $\bar{x}$ (glossary-linked)
`\logmean{x}{y}`	Logarithmic mean: $(x-y)/\ln(x/y)$ (glossary-linked)
`\vect{x}`	Vector notation: $\vec{x}$ (glossary-linked)
`\flow{x}`	Volumetric flow rate: $\dot{V}$ (glossary-linked)
`\difference{x}`	Delta/difference: Δx (glossary-linked)
`\nablaoperator{x}`	Nabla operator applied: ∇x (glossary-linked)
`\grad{x}`	Gradient operator: $\operatorname{grad} x$
`\const{x}`	Mathematical constant (upright)
`\coloneq`	Colon-equals operator: $:=$
`\deriv[n]{f}{x}`	Ordinary derivative: $\mathrm{d}^n f$ (glossary-linked)
`\deriv*[n]{f}{x}`	Partial derivative: $\partial^n f$ (glossary-linked)
`\fracderiv[n]{f}{x}`	Fractional ordinary derivative
`\fracderiv*[n]{f}{x}`	Fractional partial derivative
`\timederiv[n]{f}`	Time derivative: $\partial^n f/\partial t^n$
`\timederiv*[n]{f}`	Partial time derivative (synonym)
`\posderiv[n]{f}`	Positional derivative: $\partial^n f/\partial x^n$
`\posderiv*[n]{f}`	Partial positional derivative (synonym)
`\eqend`	Equation ending with period (aligned environments)
`\eqcomma`	Equation ending with comma (aligned environments)
`\circlednum{n}`	Circled number: ①
`\symbolplaceholder`	Placeholder for a symbol yet to be defined

A.4 Units and Physical Quantities

Table A.4 / Unit and physical quantity commands

Command	Description
`\num{x}`	Format a number with locale-aware grouping (siunitx)
`\temperaturepair{T1}{T2}`	Temperature pair: 45/55 °C format
`\heatexentry`	Mark heat exchanger entry in a stream table (prime)

Continued on next page

Table A.4 / Unit and physical quantity commands (Continued)

Command	Description
\heatexexit	Mark heat exchanger exit in a stream table (double-prime)
\mtlbsmlkicon	Material balance icon for process diagrams

A.5 Layout and Spacing

Table A.5 / Layout and spacing commands

Command	Description
\setCustomMargins{inner}{outer}{top}{bottom}	Override typearea margins with explicit dimensions
\ctrw	Open a tcolorbox (wrapper command)
\ctrb	Close a tcolorbox (wrapper command)

A.6 Floats and Figures

Table A.6 / Float and figure commands

Command	Description
\omnlFigure[options]{caption}{file}	OmniLaTeX figure environment
\omnlTable[options]{caption}{file}	OmniLaTeX table environment
\omnlFloatRow[cols]{content}	Float row for subfigure layout
\omnlFloatFootmark{n}	Float footnote mark in caption
\omnlFloatFoottext{n}{text}	Float footnote text below float
\adaptedfrom{source}	Attribution for adapted figures/tables
\includesvg[file]{name}	Include an SVG graphic via Inkscape
\debugtikzsvg	Debug helper for TikZ SVG inclusion

A.7 References and Citations

Table A.7 / Reference and citation commands

Command	Description
\ctanpackage{name}	Hyperlinked CTAN package reference
\idx{x}	Format an index entry reference (glossary-linked)
\todo{text}	Inline TODO annotation (requires todonotes option)

A.8 Presentation Build Helpers

Table A.8 / Presentation-specific commands

Command	Description
`\presentationframe[options]`	Begin a presentation slide frame
`\endpresentationframe`	End a presentation slide frame
`\slidetitle{text}`	Slide title with rule decoration
`\presentationSection{text}`	Full-page section divider slide
`\presentationHeader`	Insert frame header bar (title + author)
`\presentationFooter`	Insert frame footer (title + page count)
`\presentationProgressBar`	Insert progress bar at page bottom
`\presentationframemark`	Increment frame counter
`\presentationcountframes`	Count total frames (call after last frame)
`\presentationprogresson`	Enable progress bar
`\presentationprogressoff`	Disable progress bar
`\presentationnavsymbolson`	Enable navigation symbols
`\presentationnavsymbolsoff`	Disable navigation symbols

A.9 Chemistry

Table A.9 / Chemistry commands (via **chemmacros**)

Command	Description
`\ch{formula}`	Typeset a chemical formula (e.g. `\ch{H2O}`)
`\chcpd[name]{formula}`	Typeset a chemical compound with optional name
`\chemamount{qty}{unit}{compound}`	Formatted amount: quantity, unit, and compound name

B Option Reference

B.1 Class Options

Table ?? lists all 15 class options accepted by `\documentclass[...]{omnilatex}`.

Table B.1: Complete class option reference.

Option	Type	Default	Description
language	string	english	Document language
doctype	string	thesis	Document profile
institution	string	none	Institutional template
titlestyle	string	book	Title page variant
censoring	bool	false	Enable `\censored`
loadGlossaries	bool	false	Load glossaries-extra
todonotes	bool	false	Load todonotes markup
enablefonts	bool	false	Extended font selection
enablegraphics	bool	false	Standalone graphics module
enablemath	bool	true	unicode-math support
enabletikz	bool	true	TikZ core and graphics
enableengineering	bool	true	Engineering TikZ libraries
enablecode	bool	true	Minted code listings
enabletables	bool	true	Booktabs and table module
a5	void	–	Switch to A5 / 10 pt

B.2 Document Types

All 26 canonical doctypes with their KOMA-Script base class:

B.3 Citation Styles

Nine predefined styles passed to `\citationstyle{...}`:

B.4 Color Themes

Seven palettes loaded via `\usetheme{...}`:

Table B.2: Doctypes and KOMA base class.

Doctype	KOMA Base
article	scrartcl
inlinepaper	scrartcl
journal	scrartcl
cv	scrartcl
cover-letter	scrartcl
letter	scrartcl
memo	scrartcl
invoice	scrartcl
white-paper	scrartcl
homework	scrartcl
exam	scrartcl
lecture-notes	scrartcl
syllabus	scrartcl
handout	scrartcl
poster	scrreprt
presentation	scrreprt
recipe	scrartcl
manual	scrreprt
technicalreport	scrreprt
standard	scrreprt
patent	scrreprt
research-proposal	scrreprt
book	scrbook
thesis	scrbook
dissertation	scrbook
dictionary	scrbook

Style	Domain
ieee	Engineering
acm	Computer science
apa	Social sciences
chicago	Humanities
nature	Science journals
science	General science
harvard	UK academia
vancouver	Medical
mla	Liberal arts

Palette	Description
default	Standard blue and gray
midnight	Dark blue and charcoal
forest	Green tones
rose	Pink and magenta
monochrome	Grayscale
monochrome-dark	Dark grayscale
sepia	Warm brown tones

B.5 Build Modes

Three modes set via `BUILD_MODE` or `python3 build.py --mode <mode>`:

Mode	Description
dev	3 passes, bibliography, shell-escape (default)
prod	3 passes, full validation, glossary, accessibility
ultra	2 passes, no bibliography (fastest)

B.6 Institutions

16 institutional configurations in `config/institutions/`:

Institution	Institution
tuhh	mit
cambridge	oxford
cmu	princeton
columbia	stanford
epfl	tudelft
eth	tum
harvard	yale
imperial	generic

B.7 Languages

18 primary languages supported by the i18n system:

Language	Language
english	italian
german	portuguese
french	dutch
spanish	polish
russian	czech
greek	turkish
vietnamese	hindi
swedish	finnish
danish	norwegian

C Document Type Reference

This appendix provides a consolidated reference of all 26 canonical doctypes, their underlying KOMA-Script class, category, user-facing aliases, and key configuration parameters. See Chapter ?? for the full architectural description.

C.1 Complete Doctype Table

Table C.1 / Complete doctype reference – all 26 canonical docty

Name	KOMA Class	Category	Font Size	Font Mode	Colour Mode	Aliases
article	scrartcl	Academic	11pt	serif	color	paper, papers
inlinepaper	scrartcl	Academic	10pt	serif	color	inlinepapers, inline-res
journal	scrartcl	Academic	11pt	serif	color	journals, magazine, ma
cv	scrartcl	Business	10pt	sans	bw	resume, resumes, curri
cover-letter	scrartcl	Business	11pt	serif	color	coverletter
letter	scrartcl	Business	12pt	serif	color	letters
memo	scrartcl	Business	12pt	serif	color	memos, memorandum,
invoice	scrartcl	Business	–	serif	color	invoices
white-paper	scrartcl	Business	–	serif	color	whitepapers
homework	scrartcl	Education	–	serif	color	homeworks
exam	scrartcl	Education	–	serif	color	exams, examination, ex
lecture-notes	scrartcl	Education	–	serif	color	lecturenotes, lecture-no
syllabus	scrartcl	Education	–	serif	color	syllabi
handout	scrartcl	Education	–	serif	color	handouts
poster	scrartcl	Visual	20pt	sans	color	posters
presentation	scrartcl	Visual	14pt	sans	color	presentations, slides, ta
recipe	scrartcl	Personal	–	serif	color	recipes
manual	scrreprt	Reference	11pt	sans	bw	manuals, guide, guides,
technicalreport	scrreprt	Academic	–	–	–	report, reports, technic
standard	scrreprt	Technical	–	–	–	standards
patent	scrreprt	Technical	–	–	–	patents
research-proposal	scrreprt	Academic	–	–	–	researchproposal, resea
book	scrbook	Publishing	11pt	serif	color	books

Table C.1 / Complete doctype reference – all 26 canonical doctypes (

Name	KOMA Class	Category	Font Size	Font Mode	Colour Mode	Aliases
thesis	scrbook	Academic	12pt	serif	color	theses
dissertation	scrbook	Academic	–	–	–	dissertations
dictionary	scrbook	Reference	10pt	–	–	dictionaries, lexicon, le

Doctypes marked -- inherit the default from their KOMA class options or do not explicitly set a value.

C.2 KOMA Class Options Per Doctype

Each canonical doctype passes a specific set of options to its underlying KOMA-Script base class. The following table lists the key KOMA options that each doctype configures.

Table C.2 / KOMA class options per doctype

Doctype	Base Class	Key KOMA Options
book	scrbook	listof=totoc, bibliography=totoc, chapterprefix=true, open=right, numbers=noenddot, titlepage=true, twoside=true
thesis	scrbook	Same as book; adds DIV=12 layout
dissertation	scrbook	Same as book defaults
dictionary	scrbook	Same as book; adds DIV=13, headinclude=false, footinclude=false
article	scrartcl	bibliography=totoc, numbers=noenddot, parskip=half, titlepage=true
inlinepaper	scrartcl	bibliography=totoc, numbers=noenddot, parskip=half, titlepage=false
journal	scrartcl	bibliography=totoc, numbers=noenddot, parskip=half, titlepage=true
cv	scrartcl	numbers=noenddot, parskip=full, titlepage=false, twoside=false
cover-letter	scrartcl	bibliography=totoc, numbers=noenddot, parskip=half, titlepage=false
letter	scrartcl	bibliography=totoc, numbers=noenddot, parskip=half, titlepage=false
memo	scrartcl	bibliography=totoc, numbers=noenddot, parskip=half, titlepage=false
white-paper	scrartcl	bibliography=totoc, numbers=noenddot, parskip=half, titlepage=true
invoice	scrartcl	bibliography=totoc, numbers=noenddot, parskip=half, titlepage=false

Continued on next page

Table C.2 / KOMA class options per doctype (Continued)

Doctype	Base Class	Key KOMA Options
homework	scrartcl	bibliography=totoc, numbers=noenddot, parskip=half, titlepage=true
exam	scrartcl	bibliography=totoc, numbers=noenddot, parskip=half, titlepage=true
lecture-notes	scrartcl	bibliography=totoc, numbers=noenddot, parskip=half, titlepage=true
syllabus	scrartcl	bibliography=totoc, numbers=noenddot, parskip=half, titlepage=true
handout	scrartcl	bibliography=totoc, numbers=noenddot, parskip=half, twoside=false, twocolumn=true
poster	scrartcl	bibliography=totoc, numbers=noenddot, parskip=half, titlepage=false, twoside=false
presentation	scrartcl	bibliography=totoc, numbers=noenddot, parskip=half, titlepage=false, twoside=false
recipe	scrartcl	bibliography=totoc, numbers=noenddot, parskip=half, titlepage=false, twoside=false
manual	scrreprt	listof=totoc, bibliography=totoc, chapterprefix=false, open=any, numbers=noenddot
technicalreport	scrreprt	listof=totoc, bibliography=totoc, chapterprefix=false, open=any, numbers=noenddot
standard	scrreprt	listof=totoc, bibliography=totoc, chapterprefix=false, open=any, numbers=noenddot, parskip=half
patent	scrreprt	listof=totoc, bibliography=totoc, chapterprefix=false, open=any, numbers=noenddot
research-proposal	scrreprt	listof=totoc, bibliography=totoc, chapterprefix=false, open=any, numbers=noenddot

C.3 Doctype-Specific Metadata Commands

Many doctypes define custom metadata commands for their title pages. The following table summarises the most commonly used commands per doctype category.

Table C.3 / Doctype-specific metadata commands

Doctype	Command	Purpose
thesis	`\thesisinstitution{name}`	University name on title page
thesis	`\thesisdegree{name}`	Degree designation (e.g. MSc, PhD)
thesis	`\thesisadvisor{name}`	Advisor name
thesis	`\defensedate{date}`	Date of thesis defence
dissertation	`\dissertationinstitution{name}`	University name
dissertation	`\firstsupervisor{name}`	Primary supervisor
article	`\subtitle{text}`	Abstract text (reuses subtitle field)
article	`\keywords{text}`	Keyword list
article	`\affiliation{text}`	Author affiliation
journal	`\journalname{name}`	Journal publication name
journal	`\journalvolume{n}`	Volume number
journal	`\highlights{list}`	Comma-separated highlight bullets
technicalreport	`\reportnumber{id}`	Report identifier (e.g. TR-001)
technicalreport	`\confidentiality{text}`	Confidentiality classification
book	`\publisher{name}`	Publisher name
book	`\isbn{id}`	ISBN number
cv	`\cvrole{text}`	Professional role/subtitle
cv	`\cvimage{path}`	Path to circular profile photo
presentation	`\presentationframe`	Begin a new slide frame
presentation	`\slidetitle{text}`	Slide title with rule decoration
memo	`\memoto{name}`	Memo recipient
memo	`\memofrom{name}`	Memo sender
homework	`\hwcourse{name}`	Course name
homework	`\hwnumber{text}`	Assignment number (e.g. Homework 3)
exam	`\examduration{text}`	Exam duration (e.g. 2 hours)
lecture-notes	`\coursename{text}`	Course name for header
lecture-notes	`\lecturenumber{n}`	Lecture number
syllabus	`\coursenumber{id}`	Course code (e.g. CS 101)

C.4 Example `\documentclass` Lines

The following examples show the minimal class invocation for common doctype selections:

```
% Academic documents
\documentclass[doctype=article]{../../omnilatex}
\documentclass[doctype=journal]{../../omnilatex}
\documentclass[doctype=technicalreport]{../../omnilatex}
\documentclass[doctype=thesis, institution=tuhh]{../../omnilatex}
```

```
\documentclass[doctype=dissertation]{../../omnilatex}
\documentclass[doctype=research-proposal]{../../omnilatex}

% Business documents
\documentclass[doctype=cv]{../../omnilatex}
\documentclass[doctype=cover-letter]{../../omnilatex}
\documentclass[doctype=memo]{../../omnilatex}
\documentclass[doctype=invoice]{../../omnilatex}
\documentclass[doctype=white-paper]{../../omnilatex}

% Education documents
\documentclass[doctype=homework]{../../omnilatex}
\documentclass[doctype=exam]{../../omnilatex}
\documentclass[doctype=lecture-notes]{../../omnilatex}
\documentclass[doctype=syllabus]{../../omnilatex}
\documentclass[doctype=handout]{../../omnilatex}

% Visual documents
\documentclass[doctype=poster]{../../omnilatex}
\documentclass[doctype=presentation]{../../omnilatex}

% Reference documents
\documentclass[doctype=manual]{../../omnilatex}
\documentclass[doctype=book]{../../omnilatex}
\documentclass[doctype=standard]{../../omnilatex}
\documentclass[doctype=patent]{../../omnilatex}
\documentclass[doctype=dictionary]{../../omnilatex}
```

The doctype resolution pipeline normalises the input string to lowercase before matching against the alias lists. Unrecognised doctypes fall back to `book` with a warning in the log. Each canonical doctype has a corresponding profile file in `config/document-types/` that defines layout, spacing, font mode, and title page behaviour.

D Module Reference

OmniLaTeX's functionality is distributed across 27 library modules organised under `lib/`. Each module is a self-contained `.sty` file loaded conditionally by the main class or by other modules. The table below lists every module with its category and purpose.

Table D.1 / All 27 library modules

Module Path	Category	Description
lib/core/omnilatex-base	Core	Build-mode detection, date/time, spacing, and utility packages
lib/typography/omnilatex-fonts	Typography	Font selection via fontspec and unicode-math
lib/typography/omnilatex-math	Typography	amsmath, mathtools, siunitx, chemmacros, and custom math commands
lib/typography/omnilatex-typesetting	Typography	Text formatting, quotation marks, and microtypography
lib/typography/omnilatex-lists	Typography	Enumerate, itemize, and description list configuration
lib/layout/omnilatex-koma	Layout	KOMA-Script global options and class setup
lib/layout/omnilatex-page	Layout	Page geometry, margins, and typearea configuration
lib/layout/omnilatex-document	Layout	Document-level settings (draft, version, watermark)
lib/layout/omnilatex-floats	Layout	Float placement rules and caption formatting
lib/layout/omnilatex-boxes	Layout	tcolorbox styles and custom box environments
lib/layout/omnilatex-presentation	Layout	Beamer-style presentation support via scrreprt
lib/layout/omnilatex-accessibility	Layout	PDF/UA accessibility tags and structure
lib/graphics/omnilatex-graphics	Graphics	graphicx, SVG inclusion, and image helpers
lib/graphics/omnilatex-tikz-core	Graphics	TikZ core and common library loading
lib/graphics/omnilatex-tikz-engineering	Graphics	Engineering TikZ libraries (circuits, patterns, decorations)
lib/tables/omnilatex-tables	Tables	booktabs, tabularx, and longtable configuration
lib/references/omnilatex-hyperref	References	Hyperref setup and PDF metadata

Continued on next page

Table D.1 / All 27 library modules (Continued)

Module Path	Category	Description
lib/references/omnilatex-citations	References	biblatex configuration and citation command setup
lib/references/omnilatex-biblio	References	Bibliography back-end and bibliography environment tuning
lib/references/omnilatex-glossary	References	glossaries-extra setup and acronym/abbreviation support
lib/code/omnilatex-listings	Code	Minted code listings and syntax highlighting
lib/language/omnilatex-i18n	Language	Translation key system and locale switching
lib/language/omnilatex-cjk	Language	CJK script support via luatexja
lib/language/omnilatex-rtl	Language	Right-to-left script support (Arabic, Hebrew)
lib/utils/omnilatex-utils	Utilities	General-purpose helper macros and convenience commands
lib/utils/omnilatex-colors	Utilities	Colour theme definitions and palette management
lib/utils/omnilatex-themes	Utilities	High-level theme selection via `\usetheme`

Module loading follows a dependency graph enforced in the main class file. The `enable*` boolean class options gate modules at the top level: `enablemath`, `enabletikz`, `enableengineering`, `enablecode`, and `enabletables`.

E SI Units & Physical Quantities Reference

This appendix documents the `siunitx` v3 integration provided by OmniLaTeX, including the global configuration in `omnilatex-math.sty`, custom unit declarations, custom qualifiers, and practical usage patterns for scientific and engineering documents.

E.1 OmniLaTeX siunitx Configuration

OmniLaTeX loads **siunitx** v3 with the following global settings applied via `\sisetup`:

```
\sisetup{
    mode=match,
    propagate-math-font=true,
    reset-math-version=false,
    reset-text-family=false,
    reset-text-series=false,
    reset-text-shape=false,
    text-family-to-math=true,
    text-series-to-math=true,
    text-font-command=\unitnumberfont,
    range-units=single,
    per-mode=symbol,
    uncertainty-mode=separate,
}
```

Additionally, locale-specific settings are applied automatically based on the document language:

```
% German documents
\sisetup{locale=DE}

% English documents
\sisetup{locale=US}
```

The mode=match setting ensures that siunitx output inherits the surrounding text and math font properties (family, series, shape), so units render correctly in both serif and sans-serif contexts.

E.2 Core siunitx Commands

The three primary commands for typesetting physical quantities are `\unit`, `\qty`, and `\qtyrange`:

```
% Unit only
\unit{\kilogram\per\cubic\metre}
% → kg/m³

% Quantity with value
\qty{9.81}{\metre\per\second\squared}
% → 9.81 m/s²

% Range of quantities
\qtyrange{20}{25}{\degreeCelsius}
% → 20 °C to 25 °C
```

```
% Number only (locale-aware formatting)
\num{1234567.89}
% → 1,234,567.89 (US) or 1.234.567,89 (DE)
```

E.3 Number Formatting

The `\num` command provides extensive number formatting options. Commonly used keys are shown below.

Table E.1: Number formatting options.

Option	Example Input
Default	`\num{1234.5678}`
round-mode=places	`\num[round-mode=places, round-precisio`
round-mode=figures	`\num[round-mode=figures, round-precisi`
group-separator	`\num[group-separator={.}]{1000}`
group-minimum-digits	`\num[group-minimum-digits=4]{1000}`
exponent-mode=scientific	`\num[exponent-mode=scientific]{0.00034`
exponent-mode=engineering	`\num[exponent-mode=engineering]{0.0003`
exponent-mode=engineering, engineering-orientation=positive	`\num[exponent-mode=engineering, engine`
scientific-notation=engineering	`\num[scientific-notation=engineering]{`
sign-mantissa	`\num[sign-mantissa=+]{42}`
complex-mode=magnitude	`\num[complex-mode=magnitude]{3+4i}`

E.4 Standard SI Units

OmniLaTeX provides access to the full siunitx v3 unit library. The most commonly used units are listed below with their correct siunitx command syntax.

Table E.2 / Common SI base and derived units

Quantity	Unit	siunitx Command	Example
Length	metre	`\metre`	`\qty{1.5}{\metre}` → 1.5 m
Mass	kilogram	`\kilogram`	`\qty{2.0}{\kilogram}` → 2.0 kg
Time	second	`\second`	`\qty{60}{\second}` → 60 s

Continued on next page

Table E.2 / Common SI base and derived units (Continued)

Quantity	Unit	siunitx Command	Example
Electric current	ampere	`\ampere`	`\qty{0.5}{\ampere}` → 0.5 A
Temperature	kelvin	`\kelvin`	`\qty{293}{\kelvin}` → 293 K
Amount	mole	`\mole`	`\qty{1}{\mole}` → 1 mol
Luminous intensity	candela	`\candela`	`\qty{1}{\candela}` → 1 cd
Force	newton	`\newton`	`\qty{9.81}{\newton}` → 9.81 N
Energy	joule	`\joule`	`\qty{100}{\joule}` → 100 J
Power	watt	`\watt`	`\qty{2.5}{\kilo\watt}` → 2.5 kW
Pressure	pascal	`\pascal`	`\qty{101325}{\pascal}` → 101 325 Pa
Voltage	volt	`\volt`	`\qty{230}{\volt}` → 230 V
Frequency	hertz	`\hertz`	`\qty{50}{\hertz}` → 50 Hz
Angle	degree	`\degree`	`\qty{45}{\degree}` → 45°
Area	square metre	`\square\metre`	`\qty{25}{\square\metre}` → 25 m²
Volume	cubic metre	`\cubic\metre`	`\qty{1}{\cubic\metre}` → 1 m³

E.5 Angles, Temperature, and Frequency

Special attention is required for units that use non-standard notation:

```
% Angles: use \degree for the degree symbol
\qty{90}{\degree}              % → 90°
\qty{1.57}{\radian}            % → 1.57 rad
\qty{3600}{\degree\per\hour}   % → 3600°/h (angular velocity)

% Temperature: \degreeCelsius and \kelvin
\qty{25}{\degreeCelsius}       % → 25 °C
\qty{298}{\kelvin}             % → 298 K
\qtyrange{20}{30}{\degreeCelsius}  % → 20 °C to 30 °C

% OmniLaTeX temperature pair shorthand
\temperaturepair{45}{55}       % → 45/55 °C
```

```
% Frequency
\qty{50}{\hertz}              % → 50 Hz
\qty{2.4}{\giga\hertz}        % → 2.4 GHz
```

E.6 Custom SI Units

OmniLaTeX declares additional units beyond the standard siunitx library. All custom units are declared with `\DeclareSIUnit` and are available immediately after module load. They integrate with the standard `\unit`, `\qty`, and `\qtyrange` commands.

Table E.3: Custom SI units defined by OmniLaTeX..

Command	Symbol	Base	Description
`\volpercent`	vol %	percent	Volume percent; declared as text to preserve hyphen
`\watthour`	Wh	J/s × h	Watt-hour energy unit
`\annum`	a	year	Annum (Julian year, 365.25 days)
`\atmosphere`	atm	pressure	Standard atmosphere (101 325 Pa)
`\partspermillion`	ppm	ratio	Parts per million (10^{-6})
`\bar`	bar	pressure	Bar (10^5 Pa)
`\relhumidity`	%rh / % r.F.	ratio	Relative humidity; language-dependent label

The `\relhumidity` unit adapts to the document language. In German documents it renders as `%r.F.` (*relative Feuchte*); in English documents as `%RH`.
The Euro sign is also available as a unit via the `eurosym` package–use `\euro` directly inside `\unit{}` or `\qty{}{`.

```
\qty{350}{\watthour}              % → 350 Wh
\qty{15}{\percent}                % → 15 %
\qty{400}{\volpercent}            % → 400 vol %
\qty{1.013}{\bar}                 % → 1.013 bar
\qty{1}{\atmosphere}              % → 1 atm
\qty{350}{\partspermillion}       % → 350 ppm
\qty{65}{\relhumidity}            % → 65 %RH
\qty{2.50}{\euro\per\kilogram}    % → €2.50/kg
```

E.7 Custom SI Qualifiers

Unit qualifiers appear as subscripts and disambiguate quantities that share the same SI unit. OmniLaTeX declares three qualifiers via `\DeclareSIQualifier`, sourcing their display text from the glossary system for consistency.
Usage follows the standard siunitx qualifier syntax:

```
\unit{\kilogram\per\cubic\metre\dryair}
% → kg/m³_dry air
```

Table E.4: Custom SI qualifiers defined by OmniLaTeX..

Command	Qualifier	Description
`\dryair`	dry air	Dry air component in moist-air calculations; reads from glossary entries `sub.dry` and `sub.air`
`\water`	water	Water component; reads from glossary entry `sub.water`
`\thermal`	thermal	Thermal property qualifier; reads from glossary entry `sub.thermal`

```
\qty{1.2}{\kilo\watt\per\metre\thermal}
% → 1.2 kW/m_thermal
```

E.8 Declaring Custom Units

Users can declare additional units in their document preamble using the standard siunitx v3 interface:

```
\DeclareSIUnit\foot{ft}
\DeclareSIUnit\horsepower{hp}
\DeclareSIUnit\BTU{BTU}

\qty{6}{\foot}                    % → 6 ft
\qty{150}{\horsepower}            % → 150 hp
\qty{5000}{\BTU\per\hour}         % → 5 000 BTU/h
```

To declare a compound unit with a qualifier:

```
\DeclareSIQualifier\fuel{fuel}
\unit{\kilogram\per\second\fuel}
% → kg/s_fuel
```

E.9 Physics Examples

The following examples demonstrate typical physics and engineering calculations using OmniLaTeX's siunitx integration.

E.9.1 Mechanics

```
% Force: F = ma
The gravitational force is $F = \qty{80}{\kilogram} \times
\qty{9.81}{\metre\per\second\squared} = \qty{784.8}{\newton}$.

% Energy: kinetic energy
$E_k = \frac{1}{2} m v^2
     = \frac{1}{2} \times \qty{2.0}{\kilogram}
       \times \left(\qty{3.0}{\metre\per\second}\right)^2
     = \qty{9.0}{\joule}$

% Work: W = F * d
$W = \qty{50}{\newton} \times \qty{10}{\metre}
   = \qty{500}{\joule}$
```

E.9.2 Thermodynamics

```
% Heat transfer
The heat flux is $\dot{Q} = \qty{5.2}{\kilo\watt}$ at
$\Delta T = \temperaturepair{80}{60}$.

% Specific heat capacity
$c_p = \qty{4.18}{\kilo\joule\per\kilogram\per\kelvin}$

% Thermal conductivity
$\lambda = \qty{0.025}{\watt\per\metre\per\kelvin}$

% Using qualifiers for moist air
$\rho = \qty{1.20}{\kilogram\per\cubic\metre\dryair}$
```

E.9.3 Electrical Engineering

```
% Ohm's law
$V = I \cdot R = \qty{2}{\ampere} \times \qty{470}{\ohm}
   = \qty{940}{\volt}$

% Power
$P = V \cdot I = \qty{230}{\volt} \times \qty{10}{\ampere}
   = \qty{2.3}{\kilo\watt}$

% Energy consumption
$E = \qty{2.3}{\kilo\watt} \times \qty{8}{\hour}
   = \qty{18.4}{\kilo\watt\hour}$
```

E.9.4 Fluid Dynamics

```
% Volumetric flow rate
$\dot{V} = \qty{0.05}{\cubic\metre\per\second}$

% Pressure drop
$\Delta p = \qty{15}{\kilo\pascal} = \qty{0.15}{\bar}$

% Reynolds number (dimensionless)
$\mathrm{Re} = \frac{\rho v D}{\mu}
  = \frac{\qty{998}{\kilogram\per\cubic\metre}
          \times \qty{1.5}{\metre\per\second}
          \times \qty{0.05}{\metre}}
         {\qty{1.002e-3}{\pascal\second}}
  = 74\,625$
```

Further Reading

(GBV), Gemeinsamer Bibliotheksverbund (2013): *Gemeinsamer Verbundkatalog (GVK)*. URL: http://gso.gbv.de/DB=2.1/ (visited on 08/28/2013).

Baehr, Hans Dieter and Kabelac, Stephan (2016): *Thermodynamik: Grundlagen und technische Anwendungen*. 16., aktualisierte Auflage. Lehrbuch. Berlin: Springer Vieweg. 671 pp. ISBN: 978-3-662-49568-1.

Bird, Steven et al. (2009): *Natural Language Processing with Python*. 1st ed. Beijing ; Cambridge [Mass.]: O'Reilly. 479 pp. ISBN: 978-0-596-51649-9.

Dirac, Paul Adrien Maurice (1981): *The Principles of Quantum Mechanics*. International Series of Monographs on Physics. Clarendon Press. ISBN: 978-0-19-852011-5.

Dubbel, Heinrich et al. (2007): *Taschenbuch Für Den Maschinenbau*. 22nd ed. Berlin Heidelberg New York: Springer. 1 p. ISBN: 978-3-540-49714-1.

Einstein, Albert (1905): "Zur Elektrodynamik Bewegter Körper. (German) [On the Electrodynamics of Moving Bodies]." In: *Annalen der Physik* 322.10 (10), pp. 891–921. DOI: http://dx.doi.org/10.1002/andp.19053221004.

Example, Jane (2024): *Demonstration Citation Entry*. Placeholder reference used for OmniLaTeX citation examples.

Goossens, Michel et al. (1993): *The \LaTeX\ Companion*. Reading, Massachusetts: Addison-Wesley.

International Organization for Standardization (Mar. 2017): *ISO 8217:2017 - Petroleum Products - Fuels (Class F) - Specifications of Marine Fuels*. Standard 8217. ISO/TC 28/SC 4, p. 23. URL: https://www.iso.org/standard/64247.html (visited on 10/08/2018).

Karmasin, Matthias and Ribing, Rainer (2010): *Die Gestaltung wissenschaftlicher Arbeiten*. 5th ed. Wien: Facultas. ISBN: 9783825248222.

Knuth, Donald E. (1973): *Fundamental Algorithms*. 3rd ed. Vol. 1. 4 vols. The Art of Computer Programming. Reading, Mass.: Addison-Wesley. ISBN: 978-0-201-89683-1.

Knuth, Donald Ervin (1986): *The TeXbook*. Computers & Typesetting A. Reading, Mass: Addison-Wesley. 483 pp. ISBN: 978-0-201-13447-6.

LaTeX3 Project Team (Nov. 27, 2005): *LaTeX 2_ε font selection*. URL: http://mirrors.ctan.org/macros/latex/doc/fntguide.pdf (visited on 08/28/2013).

Mathworks (2020): *Create a Simple Class - MATLAB & Simulink*. URL: https://www.mathworks.com/help/matlab/matlab_oop/create-a-simple-class.html (visited on 04/14/2020).

McKinney, Wes (2018): *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. Second edition. Sebastopol, California: O'Reilly Media, Inc. 524 pp. ISBN: 978-1-4919-5766-0.

Mollenhauer, Klaus and Tschöke, Helmut (2007): *Handbuch Dieselmotoren*. 3., neubearb. Aufl. VDI-Buch. Berlin: Springer. 702 pp. ISBN: 978-3-540-72164-2.

Physikalisch-Technische Bundesanstalt (2007): "Das Internationale Einheitensystem (SI)." In: *PTB-Mitteilungen* 2.2007, pp. 144–180.

Ramalho, Luciano (2015): *Fluent Python*. First edition. Sebastopol, CA: O'Reilly. 743 pp. ISBN: 978-1-4919-4600-8.

Sesink, Werner (2007): *Einführung in das wissenschaftliche Arbeiten*. 7th ed. München: Oldenbourg Verlag.

TUHH Universitätsbibliothek (2013): *Katalog der TUB*. URL: https://katalog.b.tuhh.de/DB=1/LNG=DU/ (visited on 08/28/2013).

Voss, Rödiger (2011): *Wissenschaftliches Arbeiten*. 2nd ed. Konstanz: UVK Verlagsgesellschaft. ISBN: 978-3-8252-8483-1.

Wikipedia Contributors (Jan. 5, 2021): *Modelica*. In: *Wikipedia*. Ed. by Wikipedia Contributors. URL: https://en.wikipedia.org/w/index.php?title=Modelica&oldid=998516587 (visited on 02/19/2021).

Index

Terms

Names

Colophon

Build Information

This manual was typeset using OmniLaTeX with the following toolchain:

Table E.5: Build toolchain.

Component	Version / Identifier
Document class	OmniLaTeX v1.16.0
TEX engine	LuaLaTeX
KOMA-Script	Part of TeX Live 2025+
Primary font	Libertinus Serif, Sans, Mono
Math font	Libertinus Math
Code highlighting	Pygments (via **minted**)
Bibliography	Biber backend (**biblatex**)
Glossary	**glossaries-extra**
Index	**imakeidx**
Tables	**booktabs**, **tabularray**
Graphics	**tikz**, **pgfplots**
Boxes	**tcolorbox**
Hyperlinks	**hyperref**, **cleveref**

Compilation Sequence

The manual is compiled using the standard OmniLaTeX build sequence. The recommended build command is:

```
% Full build (four passes for cross-references)
lualatex manual.tex
biber manual
lualatex manual.tex
lualatex manual.tex
```

The build sequence resolves cross-references, bibliography entries, glossary entries, and index entries across multiple passes:

1. **First pass** – collects citation keys, glossary references, and index entries into auxiliary files.
2. **Biber** – processes `.bcf` to generate the bibliography database (`.bbl`).

3. **Second pass** – incorporates bibliography and resolves forward references.
4. **Third pass** – ensures all cross-references (equations, figures, tables, glossary, index) are fully resolved.

For iterative editing, two passes (lualatex + lualatex) are usually sufficient. A full four-pass build is recommended before final submission or publication.

Auxiliary Files Generated During Build

The compilation process generates several auxiliary files. These should not be manually edited and can be safely deleted to force a clean rebuild:

Table E.6: Auxiliary files produced by the build process.

File	Purpose
`.aux`	Cross-reference data for LaTeX
`.bcf`	Bibliography control file for Biber
`.bbl`	Processed bibliography output
`.run.xml`	Biber run metadata
`.glo`	Glossary entry data
`.gls`	Processed glossary output
`.idx`	Raw index entries
`.ind`	Processed index output
`.ilg`	Index log
`.lof`	List of figures data
`.lot`	List of tables data
`.toc`	Table of contents data
`.log`	Build log with warnings and errors
`.fls`	File listing from the TeX engine
`.fdb_latexmk`	Latexmk dependency cache

Reproducible Builds with Nix

OmniLaTeX supports fully reproducible builds via Nix. When using the Nix flake, all system dependencies (TeX Live, Python, Pygments) are pinned to exact versions, ensuring that the output is bit-for-bit identical across machines. See Chapter **??** for detailed Nix workflow instructions.

```
% Reproducible build via Nix
nix build
# Output: result/bin/manual.pdf
```

The Nix flake pins all dependencies using `flake.lock`. To update dependencies while maintaining reproducibility:

```
nix flake update        # Update all inputs
nix flake lock --update-input nixpkgs  # Update specific input
```

Repository and License

Table E.7: Repository information.

Field	Value
Repository	https://github.com/WyattAu/OmniLaTeX-template
License	MIT
Issue tracker	GitHub Issues

Bibliography Details

The bibliography in this manual is managed by **biblatex** with the Biber backend. Two bibliography sections are printed:

1. **Cited Works** – entries referenced via `\cite{}` in the text. These appear with numbered labels.
2. **Further Reading** – all other entries in the `.bib` file that were not explicitly cited. These appear without numbering (`env=bibnonum`) and are intended as supplementary references.

The `\nocite{*}` command ensures that all entries from the bibliography database are included, regardless of whether they were cited in the text. Remove this command to list only cited works.

Glossary and Index

The glossary is generated by **glossaries-extra** and collects three categories of entries:

Symbols Mathematical symbols with their notation, description, and units (e.g. velocity, pressure, temperature).

Abbreviations Acronyms and initialisms that are expanded on first use (e.g. CJK, RTL, DOI).

General terms Technical terms that benefit from a concise definition.

The index is generated by **imakeidx** and uses the `bookindex` style for two-level entries. Index entries are embedded in the source using `\index{term}` and `\index{term!subterm}` commands. The `printunsrtindex` command renders the index in unsorted order (as written in the source).

Final Notes

OmniLaTeX is a living project. New doctypes, institutional profiles, and module features are added on a rolling basis. Consult the git log for detailed changelog entries between releases.
To check the version of OmniLaTeX you are using, inspect the LaTeX log file for the line:

```
Document Class: omnilatex 2026/05/06 v1.16.0
```

If you encounter any issues with this manual or with OmniLaTeX itself, please file a report on the GitHub issue tracker with a minimal working example and the version information from the log.